

Part I

Exact Riemann solvers

Chapter 1

Introduction

1.1 ■ What is a Riemann problem?

In 1860, Bernhard Riemann published a study titled “On the Propagation of Plane Waves of Finite Amplitude” [14], the first mathematical analysis of the Euler equations of gas dynamics. In that work, among many contributions, is a study of what came to be known as the **Riemann problem**: a hyperbolic partial differential equation (PDE) with initial data consisting of two constant states separated by a discontinuity. The idea of discontinuous solutions to hyperbolic conservation laws was first suggested by Stokes [16], but experts at the time shied away from this idea; Stokes’ suggestion was criticized by Lords Kelvin and Rayleigh. In focusing on a problem with discontinuous data, Riemann not only accepted this idea but placed it at the center of our understanding of hyperbolic PDEs. This history is recounted in more detail by Dafermos (see [1]). As stated there, “Riemann’s remarkable paper provides the foundation for the general theory of hyperbolic systems of conservation laws in one space dimension.” That theory revolves, to a great extent, around the Riemann problem and its solution.

For a hyperbolic system of m PDEs, each of the states in the Riemann solution is a vector with m components. We will generally use $q(x, t) \in \mathbb{R}^m$ to denote the generic solution to a hyperbolic problem in one space dimension and denote the Riemann initial data by q_ℓ for the left state (i.e., $q(x, 0) = q_\ell$ for $x < 0$) and by q_r for the right state (i.e., $q(x, 0) = q_r$ for $x \geq 0$).

We will formally define what we mean by a hyperbolic system of PDEs shortly, but for now it suffices to know that these are the PDEs that naturally arise whenever we model wave propagation. The work of Riemann and Stokes was motivated by the Euler equations of compressible gas dynamics; in this introduction we use examples involving the motion of water waves. Hyperbolic equations also arise in modeling waves in solids (e.g., seismic waves), in traffic, and in many other applications.

In a model of small-amplitude waves, the equations may be linear (e.g., acoustics or linear elasticity, modeling seismic waves). These PDEs arise from a linear approximation to more general nonlinear hyperbolic systems that are also valid for

larger-amplitude waves (e.g., the Euler equations of compressible gas dynamics, or nonlinear elastic wave equations).

The plots shown in this chapter are created using a function that is defined in the Python module

- `exact_solvers/shallow_water.py` ... on GitHub.¹⁷

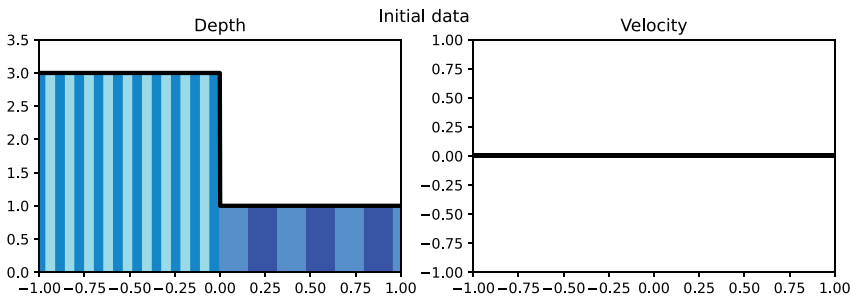
In all of the notebooks, much of the Python code that is used behind the scenes is imported from modules in the `exact_solvers` subdirectory, including the grungy details to make nice plots. Much of the code for actually solving the Riemann problems is also in these modules, so if you want to dive into the implementations, you will need to examine that code. Near the top of each notebook there is a link to the relevant modules for that chapter. The versions on GitHub may be nicer to read since the syntax is automatically highlighted.

Note also that the import statements are hidden in the printed version of each notebook, to reduce clutter.

1.2 ■ An example: The dam break problem

To get a feel for what a Riemann problem and its solution look like, consider a *dam break* problem in which a dam initially separates water with constant depth h_ℓ to the left of the dam from water with depth h_r to the right of the dam. Suppose the water is initially stationary, with velocity 0 on both sides of the dam. This situation is depicted in the figure below, with water behind the dam shaded light blue and water in front of the dam shaded dark blue.

In [3]: `make_demo_plot_function(h_l=3., h_r=1., u_l=0., u_r=0.)(t=0)`



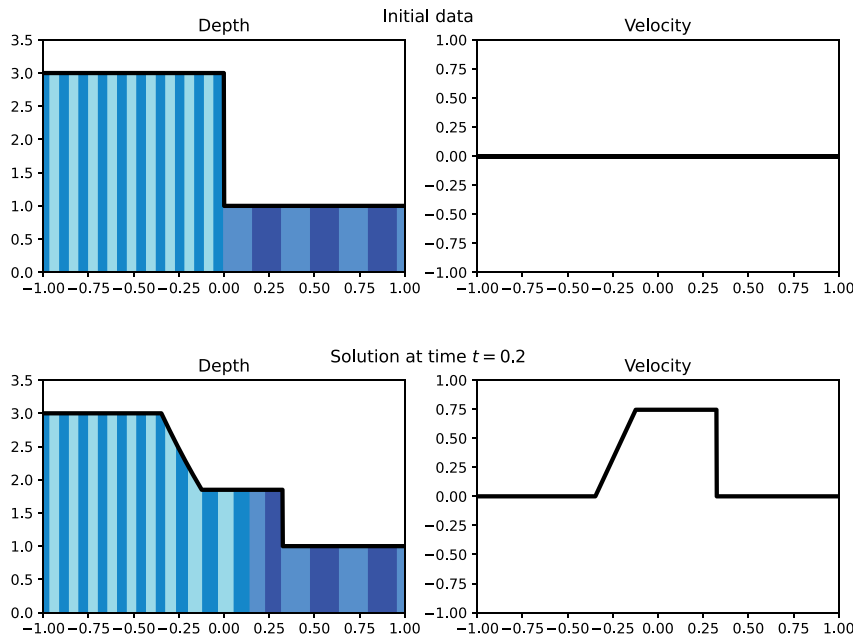
Now suppose we remove the dam at time $t = 0$. What happens for $t > 0$?

If the water to the left of the dam is deeper than the water to the right (i.e., $h_\ell > h_r$), then we expect water to flow from left to right in some manner. In reality we might expect the flow to be turbulent and multidimensional, but for now let us idealize this with a one-dimensional model in which the velocity is purely horizontal and is constant throughout the depth of the fluid, so for $t > 0$ the velocity varies only with x and t , given by some function $u(x, t)$, and the depth is given by $h(x, t)$. In Chapter 7 we will discuss the one-dimensional **shallow water equations** that can be used to model this idealized situation.

¹⁷https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/shallow_water.py

The figure below shows the solution to the shallow water equations in the situation just described (as an animation in the live notebook or html version). Again, the water that is initially to the left of the dam is colored light blue, and the water that is initially to the right is colored dark blue. The stripes with different shading are to help you visualize the flow; each stripe represents the same volume of water. Think of the color as a dye that is simply carried along with the water as it flows. In Chapter 8 we will see that we can in fact model this by adding another equation to the shallow water equations for the density of a tracer.

```
In [4]: interact(make_demo_plot_function(h_l=3., h_r=1., u_l=0., u_r=0),
                 t=widgets.FloatSlider(min=0., max=0.6, step=0.1,
                                       value=0.), fig=fig(0));
```



1.2.1 ■ Things to note about this animation

- Initially the fluid velocity $u(x,0) \equiv 0$ at $t = 0$. For $t > 0$, the water remains stationary far away from the dam. The region where the water is not stationary grows linearly outward from $x = 0$ as t increases.
- The fluid is accelerated to the right as water starts to flow.
- Water upstream from the dam (for $x < 0$) starts to accelerate slowly as the depth of the water starts to fall, and both the depth and the velocity remain continuous functions of x in this region for all time. The depth and velocity vary through a **rarefaction wave** that moves upstream.
- The water downstream from the dam is stationary until a **shock wave** passes by, which instantaneously accelerates the fluid from $u_r = 0$ to some constant velocity $u_m > 0$. At the same time, the depth increases from h_r to h_m . The resulting *middle-state* depth and velocity are the same as the depth and

velocity that the light blue water upstream from the dam also reaches once it has passed through the rarefaction wave and been fully accelerated.

- The shock wave moves at a constant velocity $s_r > u_m$.
- The rarefaction wave spreads out as time evolves, and the left and right edges move at constant velocities λ_{l1} and λ_{m1} . We will see later that these speeds are eigenvalues of a certain Jacobian matrix.

A **shock wave** is a type of wave often seen when solving nonlinear hyperbolic equations. The solution is discontinuous across a shock wave, which means that special mathematical techniques must be used to make sense of functions like this as solutions of a differential equation. It also means that special algorithmic techniques must be used to compute accurate numerical solutions to such equations. Simply replacing derivatives by finite differences typically does not work well.

In the context of water waves, this shock wave is often called a **hydraulic jump**. In a real dam break problem, the solution would not be discontinuous, but there might be a turbulent bore that, from far enough away, looks like a discontinuity. The nonlinear shallow water equations can be used to model such flows in the sense that the jump in depth and velocity and the propagation speed of the shock wave closely approximate the hydraulic jump and speed of the bore.

The nonlinear shallow water equations are usually solved in terms of the variables $q = [h, hu]$, where h is the depth (mass per unit length) and hu is the density of momentum (per unit length). For flow on a flat bottom these are conserved quantities, and the hyperbolic problem in this case is called a **hyperbolic conservation law**. For nonlinear hyperbolic equations, conservation is the key to understanding the propagation of shock waves, as we will explore in several chapters.

1.3 ■ Why is the Riemann problem important?

Why study the simple case of piecewise-constant initial data? Here are some motivations:

- As we have just seen, even this simple initial data can lead to complicated solutions.
- However, the solution is simple enough that there is some hope we can determine it exactly, even for nonlinear systems of PDEs.
- In the animation above we noted that the shock wave and the two edges of the rarefaction wave are all moving at *constant speeds*. In fact if we plot contours of the depth (or the velocity) in the x - t plane, we find that the value of each variable is constant along any ray $x = at$ from the origin. In other words, $h(x, t) = H(x/t)$ and $u(x, t) = U(x/t)$ for some functions H and U of a single variable. The solution is said to be a **similarity solution**, and this structure greatly facilitates solving the Riemann problem. This is true even for nonlinear hyperbolic problems provided certain conditions are satisfied.
- Developing an understanding of how the Riemann solution behaves for all possible choices of left and right states for a hyperbolic problem is essential for understanding the behavior of more complicated solutions.
- The development of the mathematical theory of hyperbolic problems has relied extensively on the understanding that comes from studying Riemann problems.

- Many robust numerical methods for approximating the solution with more general initial data use the Riemann solution as an essential building block. The most basic method of this type is **Godunov's method**, described below. Numerical methods typically make use of *approximate Riemann solvers*, which are often computationally cheaper than an exact solver.

While we usually think of the Riemann problem as describing the solution on a small scale near a discontinuity, it is also relevant to describing the large-scale, long-time behavior of solutions. Let us consider the general one-dimensional Cauchy problem and suppose that

$$\lim_{x \rightarrow -\infty} q(x, 0) = q_\ell,$$
$$\lim_{x \rightarrow \infty} q(x, 0) = q_r.$$

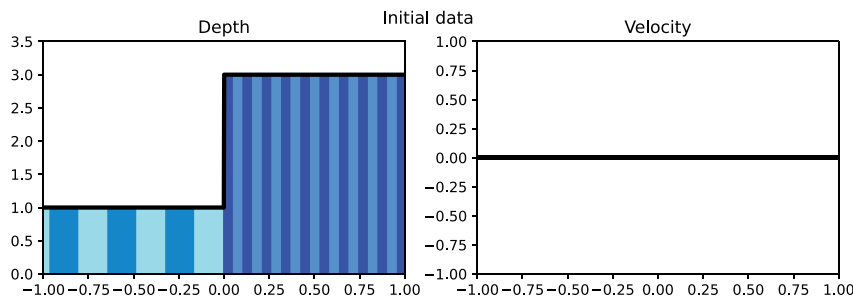
Under some additional technical assumptions it turns out that, regardless of the initial data between these asymptotic states, the solution will eventually tend toward the solution of the Riemann problem with states q_ℓ, q_r (see, e.g., [11] or [1]). Thus the solution of the Riemann problem can be used to understand both the small-scale and the large-scale behavior of solutions.

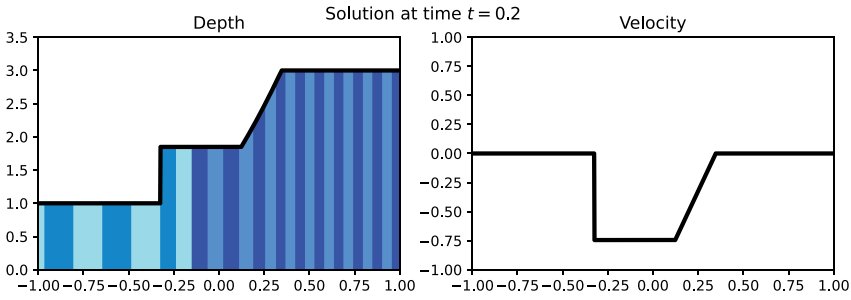
1.4 ■ The Riemann problem for the shallow water equations

The more general Riemann problem for the shallow water equations has arbitrary depths $h_\ell, h_r \geq 0$ and also arbitrary velocities u_ℓ and u_r to the left and right of $x = 0$. The dam break problem considered above is a special case where $u_\ell = u_r = 0$.

In the dam break problem considered above with $h_\ell > h_r$, the solution consists of a left-going rarefaction wave and a right-going shock wave. If we instead have $h_\ell < h_r$ with $u_\ell = u_r = 0$, then the water flows to the left and the Riemann solution has a left-going shock wave and a right-going rarefaction wave.

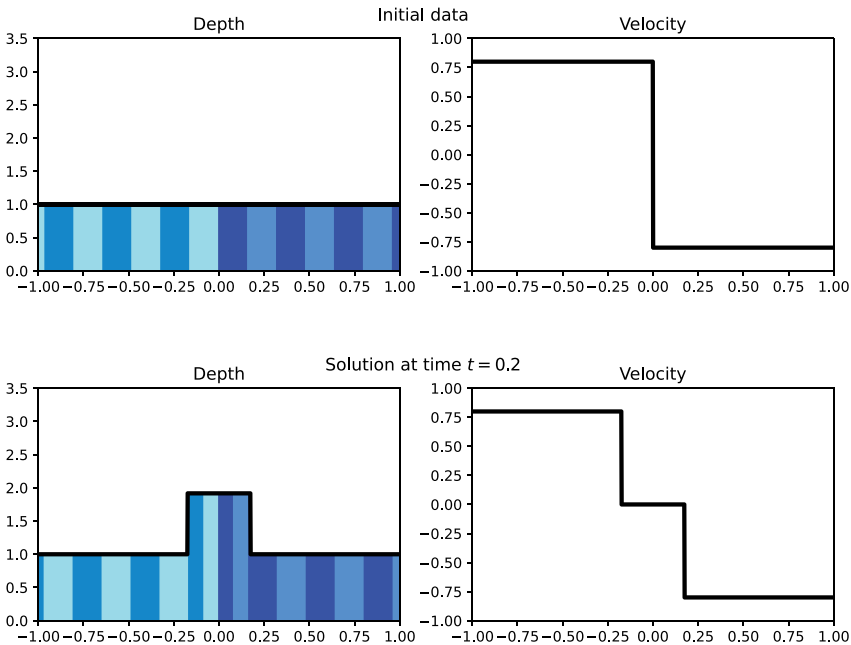
```
In [5]: interact(make_demo_plot_function(h_l=1., h_r=3., u_l=0., u_r=0.),
                 t=widgets.FloatSlider(min=0.0, max=0.6, step=0.1,
                                       value=0.), fig=fig)
```





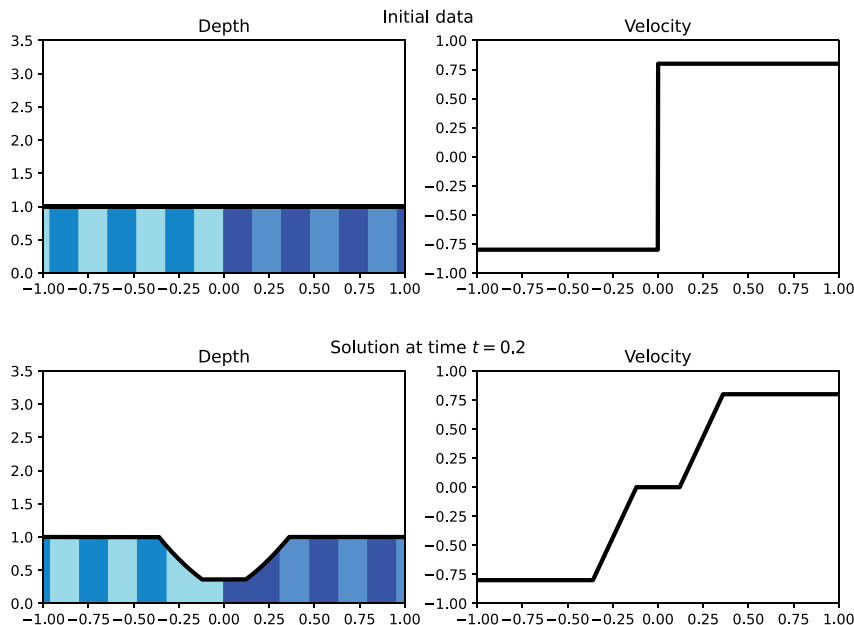
By choosing nonzero velocities in the initial data, we can also obtain other combinations of shock and rarefaction waves. For example, if the depth is initially the same everywhere ($h_\ell = h_r$) while the velocities satisfy $u_\ell > 0$ and $u_r < 0$ (modeling two opposing streams of water colliding at $x = 0$), then the Riemann solution consists of two shock waves. Note that the water is deeper between the shock waves, and in the symmetric case $u_r = -u_\ell$ the water is stationary ($u = 0$) in the region between the shocks:

```
In [6]: interact(make_demo_plot_function(h_l=1., h_r=1., u_l=0.8, u_r=-0.8),
                 t=widgets.FloatSlider(min=0., max=0.6, step=0.1,
                                     value=0.), fig=fig_fixed(0));
```



On the other hand, if $u_\ell < 0$ and $u_r > 0$ (so that the water is moving away from $x = 0$ on both sides), then the solution consists of two rarefaction waves:

```
In [7]: interact(make_demo_plot_function(h_l=1., h_r=1., u_l=-0.8, u_r=0.8),
                  t=widgets.FloatSlider(min=0.0, max=0.6, step=0.1,
                                         value=0.), fig=fig(0));
```



In Chapter 7 we will show how to solve the Riemann problem for arbitrary left and right states and in particular how to determine whether each wave is a shock or a rarefaction wave.

1.5 ■ What is a hyperbolic PDE?

In this book we consider only first-order hyperbolic equations, meaning equations that involve only first derivatives in space and time. Many important hyperbolic PDE models can be written as a system of conservation laws:

$$q_t + \nabla \cdot F(q) = 0. \quad (1.1)$$

Here $q(x, t)$ is a vector of conserved quantities (e.g., mass, momentum, energy), and each component of $F(q)$ is the corresponding flux. In practice (and in most of this book), we focus on one-dimensional problems, for which (1.1) can be written as

$$q_t + f(q)_x = 0. \quad (1.2)$$

If the flux in (1.2) is a linear function $f(q) = Aq$ for some matrix A , then we have

$$q_t + Aq_x = 0. \quad (1.3)$$

We say that (1.3) is **hyperbolic** if A is diagonalizable with real eigenvalues; similarly, we say that the nonlinear system (1.2) is hyperbolic at some state q if the Jacobian matrix $f'(q)$ is diagonalizable with real eigenvalues.

A common property of these equations is that *information travels at finite speed* and the equations model *waves*. As we will see in more detail later, the eigenvalues of $f'(q)$ correspond to *wave speeds*, while the eigenvectors tell us about the relation between the components of q in each wave.

Some examples of physical problems that can be modeled with hyperbolic PDEs (and are included as examples in this book) are

- sound waves (acoustics),
- gravity waves in shallow water (e.g., tsunami modeling),
- traffic flow,
- compressible gas dynamics, and
- elastic waves (e.g., seismology).

1.6 ■ Godunov's method

The role of the Riemann problem in the numerical solution of hyperbolic conservation laws can be traced back to the work of Godunov [3]. He proposed a method of approximation based on the following steps:

1. Approximate the solution by a piecewise-constant function, e.g., by averaging it over a set of intervals (or *grid cells*).
2. Solve the Riemann problem arising at each point of discontinuity (i.e., at the cell interfaces).
3. Using the Riemann solution, evolve the overall solution forward for a short time (before waves from neighboring Riemann problems can interact). Then re-average to obtain a piecewise-constant approximation, and repeat.

Godunov's method remains the basis for many modern numerical approaches. A number of refinements have been made. First, instead of representing the solution by a piecewise-constant function, solvers may use a piecewise-polynomial representation of higher degree. Second, instead of solving the Riemann problem exactly, most modern methods use **approximate Riemann solvers**. Approximate solutions can often be obtained at much lower computational cost. Furthermore, the approximate Riemann solver can be designed to make the overall numerical solver more robust by avoiding unphysical solution states. In Part II of this book we explore some approximate solvers.

1.7 ■ A linear example: Acoustic waves

We end this introduction with a sample linear system. Acoustic waves in one dimension can be modeled by the system of PDEs

$$\begin{aligned} p_t + Ku_x &= 0, \\ u_t + \frac{1}{\rho} p_x &= 0, \end{aligned} \tag{1.4}$$

where p represents the deviation from ambient pressure, u is the velocity, K is the bulk modulus, and ρ is the density of the medium. This first-order system is equivalent to the familiar wave equation $u_{tt} = c^2 u_{xx}$ with $c = \sqrt{K/\rho}$, which can

be derived from (1.4) by differentiating the first equation with respect to x and the second with respect to t , and then eliminating $p_{tx} = p_{xt}$. Recall that c represents the speed of propagation of waves in the wave equation. Coming back to the first-order form (1.4), we can write it in the canonical form (1.3) by setting

$$q = \begin{bmatrix} p \\ u \end{bmatrix}, \quad A = \begin{bmatrix} 0 & K \\ 1/\rho & 0 \end{bmatrix}.$$

The eigenvalues of A are $\pm\sqrt{K/\rho} = \pm c$; it is true in general that the speed of propagation of waves in (1.3) is given by the eigenvalues of A ; this hints at the importance of the condition that A be diagonalizable with real eigenvalues.

1.7.1 ■ The Riemann problem for acoustics

The Riemann problem consists of (1.4) together with initial data

$$(p_0, u_0) = \begin{cases} (p_\ell, u_\ell) & \text{for } x < 0, \\ (p_r, u_r) & \text{for } x > 0. \end{cases}$$

The solution of the Riemann problem for this simple linear system consists of two jumps, one proportional to each of the eigenvectors of A and each moving with velocity equal to the corresponding eigenvalue of A . Between these two jumps lies a middle state different from the left and right states; we can think of solving the Riemann problem by finding the intersection (in the p - u plane, called the *phase plane*) of lines passing through the left and right states that are each parallel to the appropriate eigenvector of A . This example is explored in more detail in Chapter 3.

In the notebook version, and on this webpage,¹⁸ we show the phase plane and the location of q_m relative to q_ℓ and q_r . You can drag the left and right states around to see how this changes the solution. Also shown in the other two plots are the pressure $p(x, t)$ and velocity $u(x, t)$ at one particular time $t = 1$ for this Riemann solution.

1.8 ■ Phase plane plot for the shallow water equations

The phase plane plot for the nonlinear shallow water equations is more interesting. Since the eigenvectors of the Jacobian matrix vary with q , the curves connecting states separated by a single wave are not straight lines. There is also a distinction between the curves connecting states by a shock wave and those connecting states by a rarefaction wave. This is discussed in much more detail in Chapter 7.

In the notebook, and on this webpage,¹⁹ we provide an interactive view that allows you to see how the solution changes as you move the left and right states around in the phase plane. Note that for this nonlinear system the situation is much more complicated than in the linear acoustics case.

¹⁸http://www.clawpack.org/riemann_book/phase_plane/acoustics_small.html

¹⁹http://www.clawpack.org/riemann_book/phase_plane/shallow_water_small.html

Chapter 2

Advection

We start our study with the scalar, linear hyperbolic PDE: the advection equation. The solution to this equation simply consists of the initial condition propagating with constant velocity. We first describe the physical origin of this equation and then investigate the solution of the corresponding Riemann problem. This equation is studied in more detail, e.g., in Chapter 2 of [9].

To examine the Python code for this chapter, see

- `exact_solvers/advection.py` ... on GitHub.²⁰

2.1 ■ Conservation of mass

Imagine a fluid flowing in a long, narrow tube. We'll use q to indicate the density of the fluid and u to indicate its velocity. Both of these are functions of space and time: $q = q(x, t)$; $u = u(x, t)$. The total mass in the section of tube $[x_1, x_2]$ is

$$\int_{x_1}^{x_2} q(x, t) dx.$$

This total mass changes over time due to flow in or out of this section of the tube. We call the rate of flow the *flux* and denote it by $f(q)$. Thus the net rate of flow of mass into (or out of) the interval $[x_1, x_2]$ at time t is

$$f(q(x_1, t)) - f(q(x_2, t)).$$

We just said that this rate of flow must equal the time rate of change of total mass; i.e.,

$$\frac{d}{dt} \int_{x_1}^{x_2} q(x, t) dx = f(q(x_1, t)) - f(q(x_2, t)).$$

²⁰https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/advection.py

Now since $\int_{x_1}^{x_2} \frac{\partial}{\partial x} f(q) dx = f(q(x_2, t)) - f(q(x_1, t))$, we can rewrite this as

$$\frac{d}{dt} \int_{x_1}^{x_2} q(x, t) dx = - \int_{x_1}^{x_2} \frac{\partial}{\partial x} f(q) dx.$$

If q is sufficiently smooth, we can move the time derivative inside the integral. We'll also put everything on the left side, to obtain

$$\int_{x_1}^{x_2} \left(\frac{\partial}{\partial t} q(x, t) + \frac{\partial}{\partial x} f(q) \right) dx = 0.$$

Since this integral vanishes for *any* choice of x_1, x_2 , it must be that the integrand vanishes everywhere. Therefore we can write the **differential conservation law**

$$q_t + f(q)_x = 0.$$

This equation expresses the fact that the total mass is conserved, since locally the mass can change only due to a net inflow or outflow.

2.2 ■ Advection equation

In order to solve the conservation law above, we need an expression for the flux, f . The rate of flow is just velocity times density: $f = uq$. Thus we obtain the **continuity equation**

$$q_t + (uq)_x = 0.$$

In general, we need another equation to determine the velocity $u(x, t)$. In later chapters we'll examine models in which the velocity depends on the density (or other properties), but for now let's consider the simplest case, in which the flow is characterized by a single, constant velocity $u(x, t) = a$. Then the continuity equation becomes the **advection equation**

$$q_t + aq_x = 0. \quad (2.1)$$

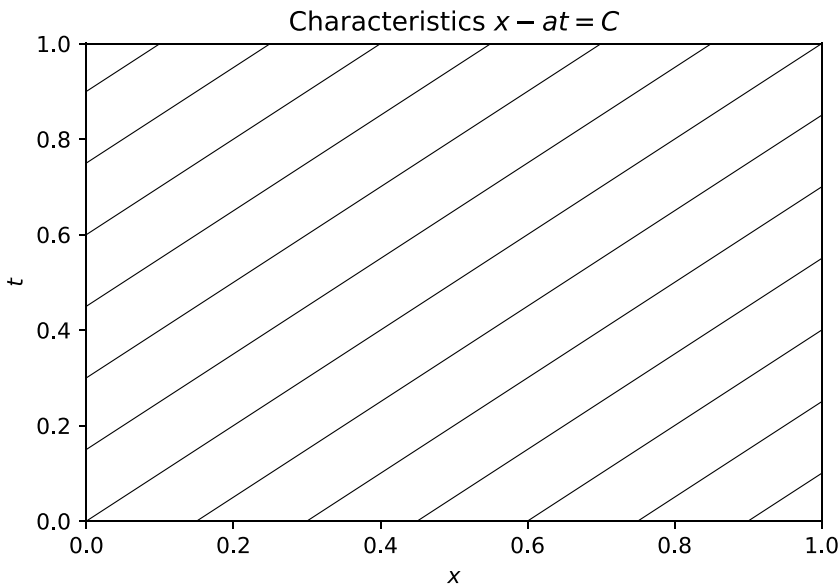
This equation has a very simple solution. Given the initial density $q(x, 0) = q_0(x)$, the solution is simply

$$q(x, t) = q_0(x - at). \quad (2.2)$$

2.3 ■ Characteristics

Notice that the solution is constant along the line $x - at = C$ for each value of C . These are parallel, straight lines in the x - t plane, with slope $1/a$, as shown in the figure below.

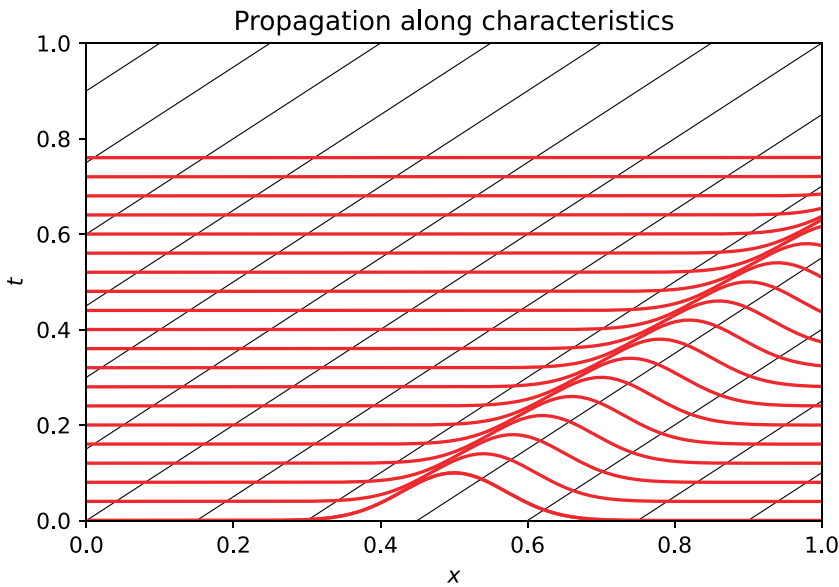
In [3]: `interact(advection.characteristics);`



We can think of the initial values $q_0(x)$ being transmitted along these lines; we sometimes say that *information* is transmitted along characteristics.

The next figure shows how an initial hump propagates to the right along characteristics.

In [4]: `interact(advection.solution);`



For more complicated hyperbolic problems, we may have multiple sets of characteristics, they may not be parallel, and the solution may not be constant along them. But it will still be the case that information is propagated along characteristics. The idea that information propagates at finite speed is an essential property of hyperbolic PDEs.

2.4 ■ The Riemann problem

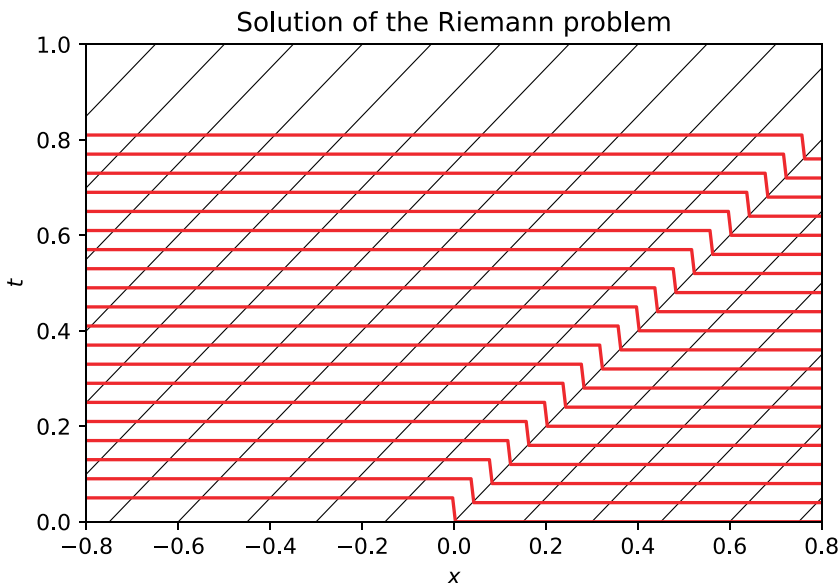
The Riemann problem consists of a hyperbolic PDE, such as (2.1), together with piecewise-constant initial data consisting of two states. For convenience, we place the interface (or jump) at $x = 0$ and refer to the left state (for $x < 0$) as q_ℓ and the right state (for $x > 0$) as q_r . We thus have

$$q_0(x) = \begin{cases} q_\ell, & x < 0, \\ q_r, & x > 0. \end{cases}$$

For the advection equation, the solution to the Riemann problem is immediately obvious; it is simply a special case of (2.2). The discontinuity initially at $x = 0$ moves at speed a . We have

$$q(x, t) = \begin{cases} q_\ell, & x - at < 0, \\ q_r, & x - at > 0. \end{cases}$$

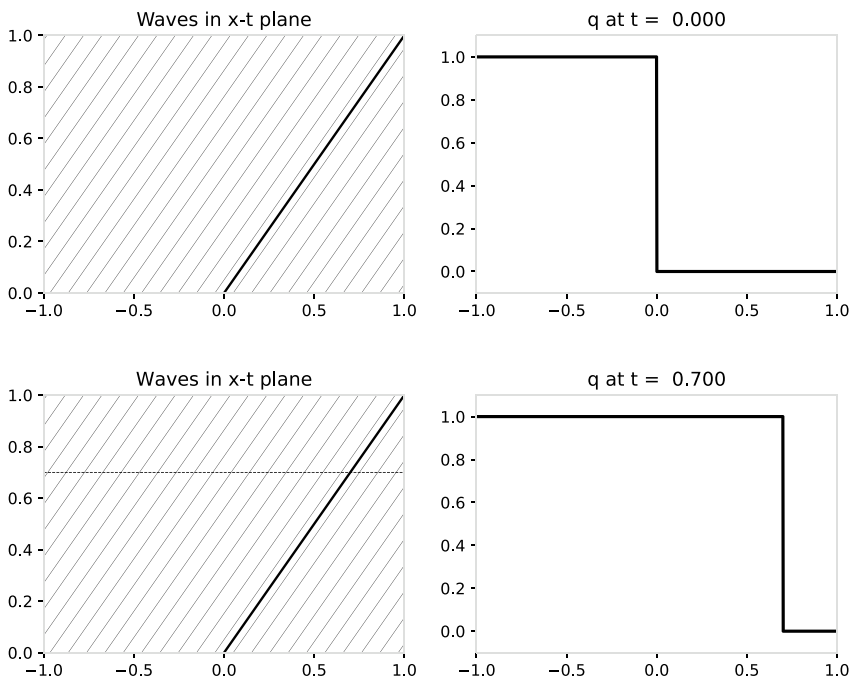
In [5]: `interact(advection.riemann_demo);`



Notice how the initial discontinuity follows the characteristic coming from $x = 0$ at $t = 0$. *In the live notebook, you can adjust the advection speed a and see how this changes the solution.*

Here’s another way to look at the Riemann solution that will be very useful for more complicated equations.

```
In [6]: q_l = 1.  
        q_r = 0.  
        advection.plot_riemann_solution(q_l, q_r, a=1.);
```



The left plot shows (again) the characteristics in the $x-t$ plane; the characteristic along which the discontinuity propagates is darker and thicker. The right plot shows the solution. *In the live notebook you can advance a time slider and see how the wave propagates.* Try modifying the initial states and the velocity a to see how this simple solution changes.

Chapter 3

Acoustics

In this chapter we consider our first *system* of hyperbolic conservation laws. We study the acoustics equations that were introduced briefly in Chapter 1. We first describe the physical context of this system and then investigate its characteristic structure and the solution to the Riemann problem. This system is described in more detail in Chapter 3 of [9].

If you wish to examine the Python code for this chapter, please see

- `exact_solvers/acoustics.py` ... on GitHub,²¹
- `exact_solvers/acoustics_demos.py` ... on GitHub.²²

3.1 ■ Physical setting

The linear acoustic equations describe the propagation of small perturbations in a fluid. In Chapter 2 we derived the one-dimensional continuity equation, which describes mass conservation:

$$\rho_t + (\rho u)_x = 0. \quad (3.1)$$

For more realistic fluid models, we need another equation that determines the velocity u . This typically takes the form of a conservation law for the momentum ρu . Momentum, like density, is transported by fluid motion with corresponding flux ρu^2 . Additionally, any difference in pressure will also lead to a flux of momentum that is proportional to the pressure difference. Thus the momentum equation takes the form

$$(\rho u)_t + (\rho u^2 + P(\rho))_x = 0, \quad (3.2)$$

where the pressure P is given by the equation of state $P(\rho)$; here we have assumed the pressure depends only on the density. A more general equation of state will be considered, along with fully nonlinear fluid motions, in Chapter 9. The linear

²¹https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/acoustics.py

²²https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/acoustics_demos.py

acoustics equations focus on the behavior of small perturbations in the system above.

In order to derive the equations of linear acoustics, observe that equations (3.1)–(3.2) form a hyperbolic system $q_t + f(q)_x = 0$ with

$$q = \begin{bmatrix} \rho \\ \rho u \end{bmatrix}, \quad f(q) = \begin{bmatrix} \rho u \\ \rho u^2 + P(\rho) \end{bmatrix}.$$

We will make use of the quasilinear form of a hyperbolic system:

$$q_t + f'(q)q_x = 0.$$

Here $f'(q)$ denotes the Jacobian of the flux f with respect to the conserved variables q . In the present system, as is often the case, f is most naturally written in terms of so-called primitive variables (in this case ρ and u) rather than in terms of the conserved variables q . To find the flux Jacobian (and thus the quasilinear form), we first write f in terms of the conserved variables $(q_1, q_2) = (\rho, \rho u)$:

$$f(q) = \begin{bmatrix} q_2 \\ q_2^2/q_1 + P(q_1) \end{bmatrix}. \quad (3.3)$$

Now we can differentiate to find the flux Jacobian:

$$\begin{aligned} f'(q) &= \begin{bmatrix} \partial f_1 / \partial q_1 & \partial f_1 / \partial q_2 \\ \partial f_2 / \partial q_1 & \partial f_2 / \partial q_2 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 1 \\ -q_2^2/q_1^2 + P'(q_1) & 2q_2/q_1 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 1 \\ P'(\rho) - u^2 & 2u \end{bmatrix}. \end{aligned}$$

Thus small perturbations to an ambient fluid state ρ_0, u_0 evolve according to the linearized equations $q_t + f'(q_0)q_x = 0$ or, more explicitly,

$$\begin{aligned} \rho_t + (\rho u)_x &= 0, \\ (\rho u)_t + (P'(\rho_0) - u_0^2)\rho_x + 2u_0(\rho u)_x &= 0. \end{aligned}$$

As we are only interested in small perturbations of equation (3.2), we expand the perturbations $\rho - \rho_0$ and $\rho u - \rho_0 u_0$ as functions of a small parameter ϵ and then we discard terms of order ϵ^2 and higher. This results in the linear hyperbolic system

$$\begin{aligned} p_t + u_0 p_x + P'(\rho_0)u_x &= 0, \\ u_t + \frac{1}{\rho_0}p_x + u_0 u_x &= 0, \end{aligned}$$

where $p(x, t)$ is the pressure as a function of x and t . If the ambient fluid is at rest (i.e., $u_0 = 0$) and the pressure is directly proportional to the density, then this simplifies to

$$\begin{bmatrix} p \\ u \end{bmatrix}_t + \underbrace{\begin{bmatrix} 0 & K_0 \\ 1/\rho_0 & 0 \end{bmatrix}}_A \begin{bmatrix} p \\ u \end{bmatrix}_x = 0, \quad (3.4)$$

where $K_0 = P'(\rho_0)$ is referred to as the bulk modulus of compressibility. The system of equations (3.4) is called the linear acoustics equations.

For the rest of this chapter we work with (3.4) and let $q = [p, u]^T$. Then we can write (3.4) as $q_t + Aq_x = 0$. For simplicity, we also drop the subscripts on K, ρ . Direct calculation reveals that the eigenvectors of A are

$$\lambda_1 = -c, \quad \lambda_2 = c, \quad (3.5)$$

where $c = \sqrt{K/\rho}$ is the speed of sound in a medium with a given density and bulk modulus. The right eigenvectors of A are given by

$$r_1 = \begin{bmatrix} -Z \\ 1 \end{bmatrix}, \quad r_2 = \begin{bmatrix} Z \\ 1 \end{bmatrix},$$

where $Z = \rho c$ is called the acoustic impedance. Defining $R = [r_1 r_2]$ and $\Lambda = \text{diag}(\lambda_1, \lambda_2)$, we have $AR = R\Lambda$, or $A = R\Lambda R^{-1}$. Substituting this into (3.4) yields

$$\begin{aligned} q_t + Aq_x &= 0, \\ q_t + R\Lambda R^{-1}q_x &= 0, \\ R^{-1}q_t + \Lambda R^{-1}q_x &= 0, \\ w_t + \Lambda w_x &= 0, \end{aligned}$$

where we have introduced the *characteristic variables* $w = R^{-1}q$. The last system above is simply a pair of decoupled advection equations for w_1 and w_2 , with velocities λ_1 and λ_2 , a system we already know how to solve. Thus we see that the eigenvalues of A are the velocities at which information propagates in the solution.

3.2 ■ Solution by characteristics

The discussion above suggests a strategy for solving the Cauchy problem:

1. Decompose the initial data $(p(x, 0), u(x, 0))$ into characteristic variables $w(x, 0) = (w_1^0(x), w_2^0(x))$ using the relation $w = R^{-1}q$.
2. Evolve the characteristic variables: $w_p(x, t) = w_p^0(x - \lambda_p t)$.
3. Transform back to the physical variables: $q = Rw$.

The first step in this process amounts to expressing the vector q in the basis given by r_1, r_2 . Solving the system $Rw = q$ yields

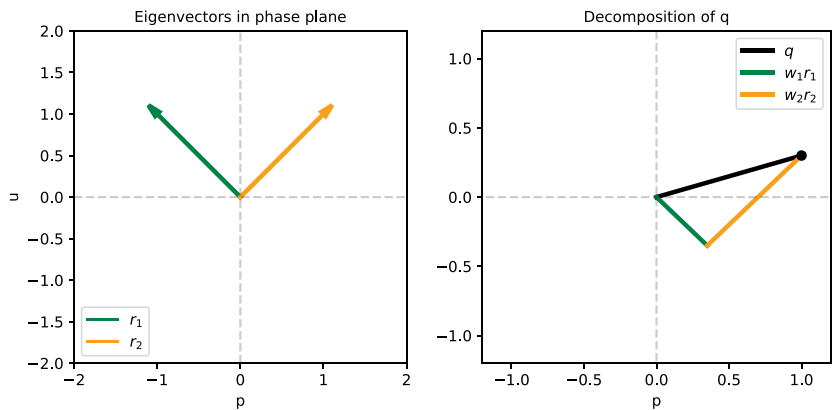
$$q = w_1 r_1 + w_2 r_2,$$

where

$$w_1 = \frac{-p + Zu}{2Z}, \quad w_2 = \frac{p + Zu}{2Z}.$$

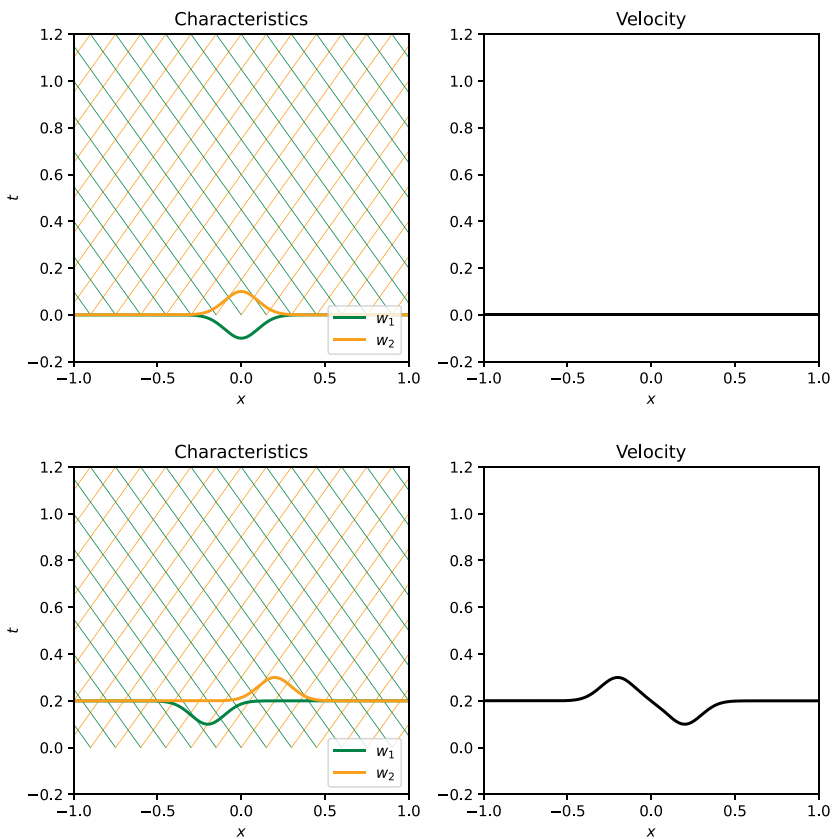
We visualize this below, where the first plot shows the two eigenvectors and the second plot shows how q can be expressed as a linear combination of the two eigenvectors r_1 and r_2 . *In the live notebook you can adjust the left and right states or the material parameters to see how this affects the construction of the Riemann solution.*

In [5]: `acoustics_demos.decompose_q_interactive()`



In the second and third steps, we evolve the characteristic variables w and then transform back to the original variables. We take as initial pressure a Gaussian with zero initial velocity. We visualize this below, where the time evolution in the characteristic variables is shown in the first plot and the time evolution of the velocity is shown in the second plot.

In [6]: `acoustics_demos.char_solution_interactive()`



In the live notebook, you can advance the above solutions in time and select which of the two characteristic variables to display. Notice how in the characteristic variables w (plotted on the left) each part of the solution simply advects (translates) since each of the characteristic variables simply obeys an uncoupled advection equation.

3.3 ■ The Riemann problem

Now that we know how to solve the Cauchy problem, solution of the Riemann problem is merely a special case. We have the special initial data

$$q(x, 0) = \begin{cases} q_\ell & \text{if } x \leq 0, \\ q_r & \text{if } x > 0. \end{cases}$$

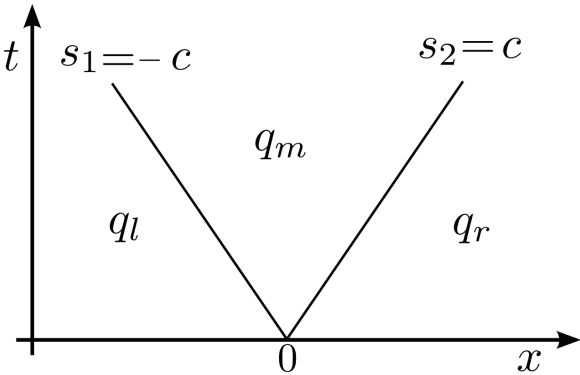
We can proceed as before by decomposing into characteristic components, advecting, and then transforming back. But since we know the solution will be constant almost everywhere, it's even simpler to just decompose the jump $\Delta q = q_r - q_\ell$ in terms of the characteristic variables and advect the two resulting jumps Δw_1 and Δw_2 :

$$\Delta q = \Delta w_1 r_1 + \Delta w_2 r_2.$$

Since $R\Delta w = \Delta q$, we have

$$\Delta w_1 = \frac{-\Delta p + Z\Delta u}{2Z}, \quad \Delta w_2 = \frac{\Delta p + Z\Delta u}{2Z}.$$

Thus the solution has the structure depicted below:



The three constant states are related by the jumps

$$q_m = q_\ell + \Delta w_1 r_1 = q_r - \Delta w_2 r_2. \tag{3.6}$$

The jumps in pressure and velocity for each propagating discontinuity are related in a particular way, since each jump is a multiple of one of the eigenvectors of A . More generally, the eigenvectors of the coefficient matrix of a linear hyperbolic system reveal the relation between jumps in the conserved variables across a wave propagating with speed given by the corresponding eigenvalue. For acoustics, the impedance is the physical parameter that determines this relation.

3.3.1 ■ A simple solution

Here we provide some very simple initial data and determine the Riemann solution, which consists of three states, q_ℓ , q_m , and q_r , and the speeds of the two waves.

```
In [7]: # Initial data for Riemann problem
rho = 0.5          # density
bulk = 2.          # bulk modulus
ql = np.array([3,2]) # Left state
qr = np.array([3,-2]) # Right state
# Calculated parameters
c = np.sqrt(bulk/rho) # calculate sound speed
Z = np.sqrt(bulk*rho) # calculate impedance
print("With density rho = %g, bulk modulus K = %g" \
      % (rho,bulk))
print("We compute: sound speed c = %g, impedance Z = %g \n" \
      % (c,Z))
```

With density rho = 0.5, bulk modulus K = 2
 We compute: sound speed c = 2, impedance Z = 1

```
In [8]: # Call and print Riemann solution
states, speeds, reval = \
    acoustics.exact_riemann_solution(ql ,qr, [rho, bulk])

print("The states ql, qm and qr are: ")
print(states, "\n")
print("The left and right wave speeds are:")
print(speeds)
```

The states q_ℓ , q_m and q_r are:

```
[[ 3.  5.  3.]
 [ 2.  0. -2.]
```

The left and right wave speeds are:

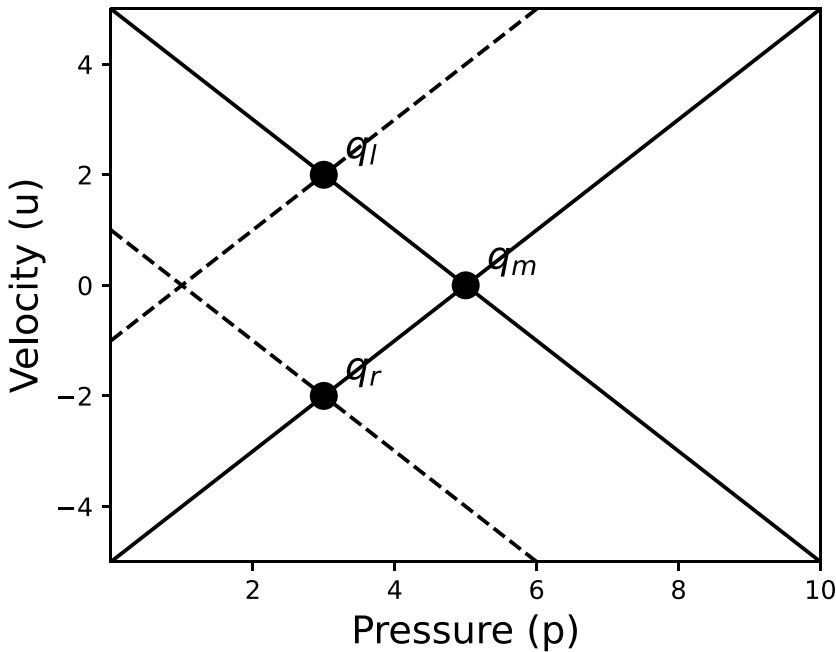
```
[-2.  2.]
```

One way to visualize the Riemann solution for a system of two equations is by looking at the p - u phase plane. In the figure below, we show the two initial conditions of the Riemann problem, q_ℓ and q_r , as points in the phase space; the lines passing through these points correspond to the eigenvectors r_1 and r_2 .

The middle state, q_m , is simply the intersection of the line in the direction r_1 passing through q_ℓ and the line in the direction r_2 passing through q_r . The structure of this solution becomes evident from equation (3.6). The dashed lines correspond to a line in the direction r_2 passing through q_ℓ and a line in the direction r_1 passing through q_r ; these also intersect but cannot represent a Riemann solution since they would involve a wave going to the right but connected to q_ℓ and a wave going to the left but connected to q_r .

In the live notebook, the cell below allows you to interactively adjust the initial conditions, the material parameters, and the plot range, so that you can explore how the structure of the solution in the phase plane is affected by these quantities.

```
In [9]: acoustics_demos.interactive_phase_plane(q_l,q_r,rho,bulk)
```



Note that the eigenvectors are given in terms of the impedance Z , which depends on the density ρ and the bulk modulus K . Therefore, when ρ and K are modified, the eigenvectors change, and consequently the slope of the lines changes as well.

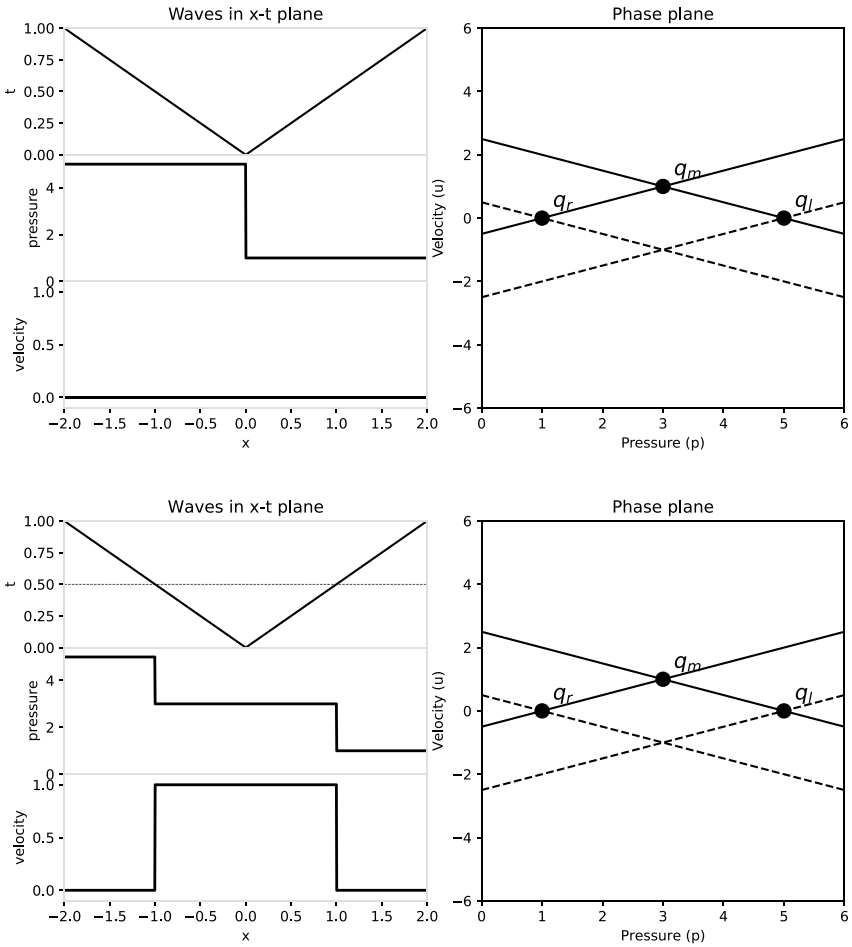
3.4 ■ Examples

We will use the exact solver in `exact_solvers/acoustics.py` and the functions in `exact_solvers/acoustics_demos.py` to plot interactive solutions for a few examples.

3.4.1 ■ Shock tube

If there is a jump in pressure and the velocity is zero in both initial states (the shock tube problem), then the resulting Riemann solution consists of pressure jumps of equal magnitude propagating in both directions, with equal and opposite jumps in velocity. This is the linearized version of what is known in fluid dynamics as a shock tube problem, since it emulates what would happen inside a shock tube, where the air is initially stationary and a separate chamber at the end of the tube is pressurized and then released.

```
In [10]: q1 = np.array([5,0])
qr = np.array([1,0])
rho = 1.0
bulk = 4.0
acoustics_demos.riemann_plot_pplane(q1,qr,rho,bulk)
```

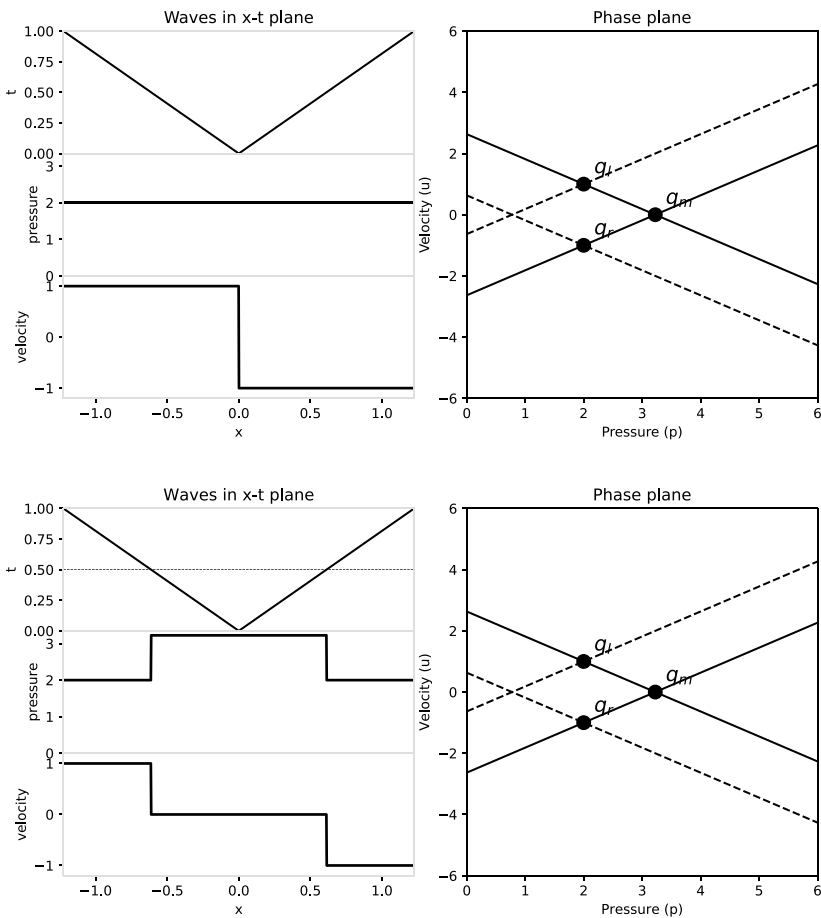


We can also observe the structure of the solution in the phase plane, which is shown in the second plot above.

3.4.2 ■ Reflection from a wall

As another example, suppose the pressure is initially the same in the left and right states, while the velocities are nonzero with $u_r = -u_l > 0$. The flow is converging from both sides, and, because of the symmetry of the initial states, the result is a middle state q_m in which the velocity is 0 (and the pressure is higher than on either side).

```
In [11]: q1 = np.array([2,1])
qr = np.array([2,-1])
rho = 1.0
bulk = 1.5
acoustics_demos.riemann_plot_pplane(q1,qr,rho,bulk)
```



We again show the Riemann solution in space and in the phase plane, where the symmetry is also evident.

Disregarding the left half of the domain ($x < 0$), one can view this as a solution to the problem of an acoustic wave impacting a solid wall. The result is a reflected wave that moves away from the wall; notice that the velocity vanishes at the wall, as it must. This type of Riemann solution is important when simulating waves in a domain with reflecting boundaries. The reflecting condition can be imposed by using fictitious *ghost cells* that lie just outside the domain and whose state is set by reflecting the interior solution with the symmetry just described (equal pressure, negated velocity).

In reality, at a material boundary only part of a wave is reflected, while the rest is transmitted. This can be accounted for by including the spatial variation in ρ, K and solving a variable-coefficient Riemann problem.

3.4.3 ■ Interactive phase plane with solution at fixed time

For a more general exploration of the solution to the acoustics equation, we now show an interactive solution (in the notebook version or on this webpage²³) of the acoustics equations. The initial states q_ℓ and q_r can be modified by dragging and dropping the points in the phase plane plot.

3.4.4 ■ Gaussian initial condition

In this example, we use the first example described near the beginning of this chapter. The initial condition is a Gaussian pressure perturbation, while the initial velocity is zero. Reflecting boundary conditions are imposed at $x = -2$ and $x = 2$, so the wave is fully reflected back, and we can see how it interacts with itself. This animation is produced using a numerical method from PyClaw²⁴ and can be viewed in the interactive notebook or on this webpage.²⁵

²³http://www.clawpack.org/riemann_book/phase_plane/acoustics_small.html

²⁴<http://www.clawpack.org/pyclaw/>

²⁵http://www.clawpack.org/riemann_book/html/acoustics_bump_animation.html

Chapter 4

Burgers' equation

In this chapter, we study a simple scalar nonlinear conservation law: Burgers' equation. Burgers' equation models momentum transport in a fluid of uniform density and pressure, and it is the simplest equation that captures some key features of gas dynamics or water waves.

To examine the Python code for this chapter, see

- `exact_solvers/burgers.py` ... on GitHub,²⁶
- `exact_solvers/burgers_demos.py` ... on GitHub.²⁷

Burgers' equation has been used extensively for developing both theory and numerical methods, and it will allow us to explore the Riemann problem for a nonlinear conservation law. Burgers' equation is a scalar conservation law with flux $f(q) = q^2/2$:

$$q_t + \left(\frac{1}{2}q^2\right)_x = 0. \quad (4.1)$$

The quasilinear form is obtained by applying the chain rule to the flux term:

$$q_t + qq_x = 0.$$

This equation looks very similar to the advection equation, with the difference that the advection speed at each point is given by the solution q . Burgers' equation is often viewed as a simplified version of equations in fluid dynamics or water waves that also have nonlinear fluxes. Our study of the dynamics of equation (4.1) is a step toward understanding these more complex nonlinear systems in Chapter 7 and Chapter 9. In Chapter 5 we consider another scalar nonlinear conservation law that has a similar structure to Burgers' equation and a simpler physical interpretation.

²⁶https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/burgers.py

²⁷https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/burgers_demos.py

The *characteristic speed* for Burgers' equation is $f'(q) = q$. As long as the solution is smooth, the solution is constant along characteristic curves $X(t)$ satisfying $X'(t) = f'(q(X(t), t))$ since then

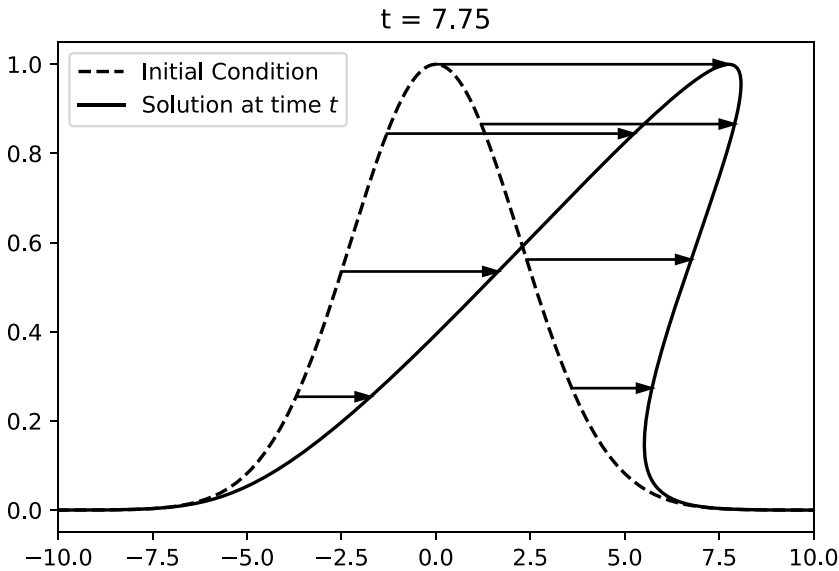
$$\frac{d}{dt}q(X(t), t) = q_x(X(t), t)X'(t) + q_t(X(t), t) = 0.$$

Because of this, the characteristics are straight lines. However, since the characteristic speed depends on the solution, these lines are not parallel and characteristics may converge or spread out.

4.1 ■ Shock formation

In the figure below we consider Burgers' equation with a Gaussian hump as the initial data. Since the characteristic speed in Burgers' equation is given by q itself, the peak of the hump travels faster than the rest, and characteristics converge at the front of the traveling wave (where $f'(q)$ decreases with x) while they spread out behind the peak (where $f'(q)$ increases with x). The dashed line shows the initial condition, while the solid lines show the solution at later times.

```
In [5]: interact(burgers_demos.multivalued_solution,
                 t=FloatSlider(min=0.,max=9.,value=7.75), fig=fixed(0));
```

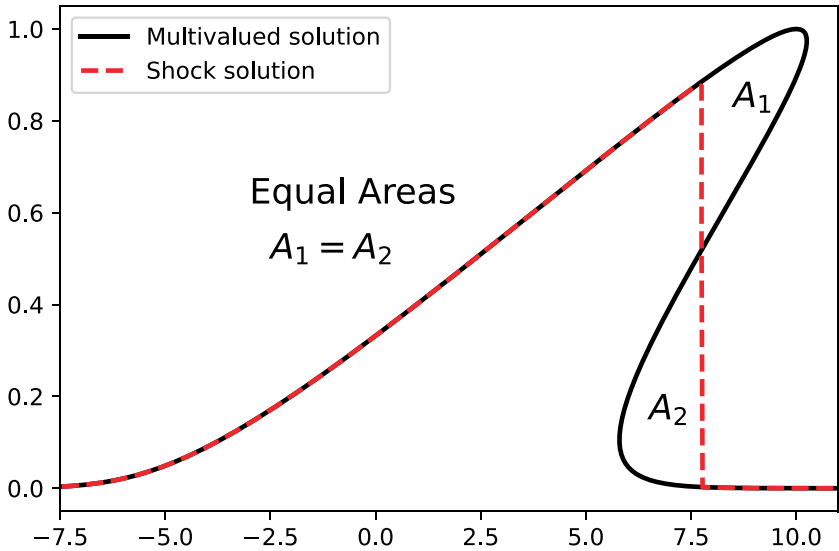


Notice that at first q remains single valued for every x . However, after some time the crest of the wave overtakes the leading edge. After this time, we obtain a triple-valued solution for certain values of x . The first time this overtaking happens is referred to as the *breaking time*—a reference to waves breaking on the beach. It is also the point where the conservation law, in its differential form, breaks down and where the characteristics cross each other for the first time. When characteristics cross, a shock wave, or discontinuity, forms. Mathematically, replacing the

triple-valued region with a discontinuity will avoid the problem of the solution being multivalued. Where should the shock be located?

If we replace part of the multivalued solution interval with a shock, some mass will be removed (area A_1 in the figure below) and some mass will be added (area A_2 in the figure below). In the live notebook, the figure below shows possible locations for the shock at a given time; to maintain conservation of the integral of q , the shock must be placed so that these two areas are equal.

```
In [6]: interact(burgers_demos.shock_location,
                xshock = FloatSlider(min=6,max=10,step=0.25,value=7.75),
                fig=fixed(0));
```



This geometric reasoning provides a nice intuition for the shock location but is cumbersome in practice. To determine the location of the shock we can use the *Rankine–Hugoniot jump condition*, which we will derive in Chapter 5 and which requires that the jump in flux across a propagating shock be related to the jump in q by

$$s(q_r - q_\ell) = f(q_r) - f(q_\ell)$$

at each instant in time, where s is the shock speed at this time.

Since the flux for Burgers' equation is $f(q) = q^2/2$, this gives

$$\begin{aligned} s(q_r - q_\ell) &= \frac{1}{2}(q_r^2 - q_\ell^2) \\ \Rightarrow s &= \frac{1}{2}(q_\ell + q_r). \end{aligned}$$

In general (as in the image above) the states q_ℓ and q_r just to the left and right of the shock will not be constant and the speed of a shock will change in time.

4.1.1 ■ Shock solution

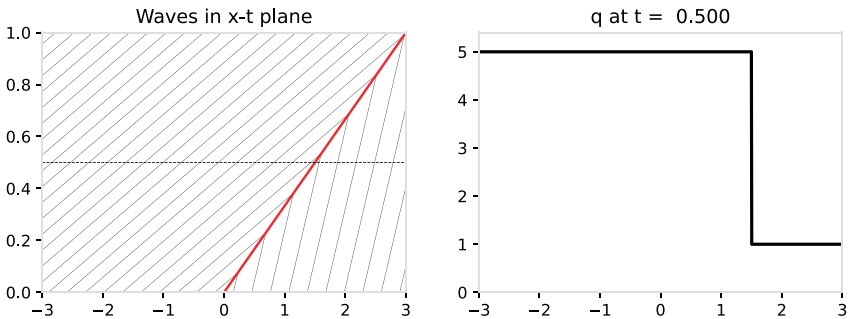
For Burgers' equation (or any scalar hyperbolic conservation law with a *convex* flux function, such as the traffic flow problem considered in Chapter 5), the solution of the Riemann problem consists of a single wave, which may be either a shock or a rarefaction, separating regions of constant value q_ℓ and q_r . Here convexity of the flux $f(q)$ means that $f'(q)$ is either monotonically increasing or monotonically decreasing as q varies. If $f(q)$ is not convex, then the solution can be more complicated; see Section 4.3.3 below and Chapter 6 for further discussion.

The analysis above already yields the solution to the Riemann problem in the case that the resulting wave is a shock. The entropy condition, as we will explain below, indicates that a shock will form when $f'(q_\ell) > f'(q_r)$, i.e., when $q_\ell > q_r$. In this case, the solution is

$$q(x,t) = \begin{cases} q_\ell & \text{if } x/t < s, \\ q_r & \text{if } x/t > s. \end{cases}$$

Below, we plot the solution of Burgers' equation for an initial condition that leads to a shock (i.e., with $q_\ell > q_r$).

```
In [7]: interact(burgers_demos.shock(),
                t=widgets.FloatSlider(min=0,max=1.0,value=0.5),
                which_char=widgets.Checkbox(value=True,
                                             description='Show characteristics'));
```



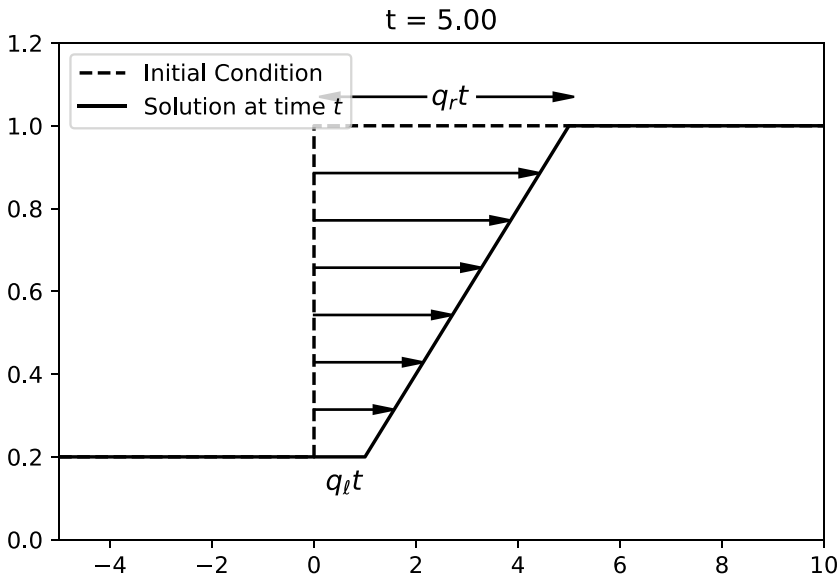
4.2 ■ Rarefaction wave

In the previous figure with the hump as the initial condition, we observed that a shock formed on the right side of the hump. However, on the left side, the characteristics spread out and will never cross. This part of the solution is called a rarefaction wave. This is the kind of behavior we will observe in the solution of the Riemann problem when $q_\ell < q_r$.

In the next figure, we consider such a Riemann problem. Although the initial data is discontinuous at $x = 0$, we can think of smearing it out slightly to a continuous function so that all values between q_ℓ and q_r are taken over a very narrow region around $x = 0$. As time evolves, each value of q propagates according to the

advection speed q given by the quasi-linear equation. Therefore, after time t , each value q must propagate a distance $x = qt$, so the solution along the rarefaction is then $q = x/t$. Because $q_\ell < q_r$, the smallest and largest displacements are given by $q_\ell t$ and $q_r t$, respectively:

```
In [8]: interact(burgers_demos.rarefaction_figure,
                t = FloatSlider(min=0,max=9,value=5));
```



The full rarefaction solution for the Burgers' Riemann problem is then simply given by

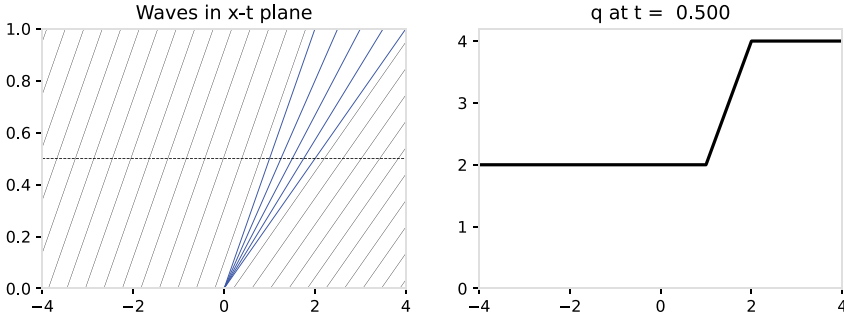
$$q(x,t) = \begin{cases} q_\ell & \text{for } x < q_\ell t, \\ x/t & \text{for } q_\ell t \leq x \leq q_r t, \\ q_r & \text{for } x > q_r t. \end{cases}$$

As we will see in Chapter 5, the rarefaction solution is always a self-similar solution. This means that it can be expressed as a function of the ratio between position and time $q(x,t) = \tilde{q}(x/t)$, so it remains the same when rescaling both x and t by the same factor. In Burgers' equation, the form of the rarefaction is particularly simple since the advection speed is simply q , and so the rarefaction wave is linear in x with slope $1/t$ at time t .

4.2.1 ■ Rarefaction solution

In the figure below, we plot a solution of the Riemann problem with $q_\ell < q_r$, exhibiting a rarefaction.

```
In [9]: interact(burgers_demos.rarefaction(),
                 t=widgets.FloatSlider(min=0,max=1.0,value=0.5),
                 which_char=widgets.Checkbox(value=True,
                                              description='Show characteristics'));
```



4.3 ■ Weak solutions

As we mentioned before, the differential form of the equation breaks down in the presence of shocks/discontinuities. However, the integral form of the conservation law remains valid. Let's integrate the general conservation law $q_t + f(q)_x = 0$ from $x = x_1$ to $x = x_2$ and $t = t_1$ to $t = t_2$:

$$\int_{t_1}^{t_2} \int_{x_1}^{x_2} [q_t + f(q)_x] dx dt = 0.$$

This integral can be rewritten in terms of an indicator function $\phi(x, t)$ in $[x_1, x_2] \times [t_1, t_2]$ (defined to be 1 in this region and 0 elsewhere):

$$\int_0^\infty \int_{-\infty}^\infty [q_t + f(q)_x] \phi(x, t) dx dt = 0. \quad (4.2)$$

We can further replace $\phi(x, t)$ by a smooth function with compact support on some region of the x - t plane (zero outside a closed and bounded region). Assuming $t = 0$ is in the support of $\phi(x, t)$, integration by parts yields

$$\int_0^\infty \int_{-\infty}^\infty [q \phi_t + f(q) \phi_x] dx dt = - \int_{-\infty}^\infty q(x, 0) \phi(x, 0) dx, \quad (4.3)$$

where now the derivatives are on $\phi(x, t)$ and not on q or $f(q)$, so the equation still makes sense for discontinuous q . Note that we only obtain one boundary term along $t = 0$ since $\phi(x, t)$ vanishes at infinity. The function $q(x, t)$ is called a *weak solution* of the conservation law if (4.3) holds for all functions $\phi(x, t)$ that are continuously differentiable and have compact support (bump functions). However, the function $\phi(x, t)$ chosen in (4.2) does not satisfy these conditions since it is not smooth. Nonetheless, we can approximate this function arbitrarily well by a smooth function. Note that any weak solution is a solution of the integral conservation law and vice versa.

Weak solutions are thus allowed to include discontinuities. The shock solution presented above, for instance, is a weak solution of Burgers' equation. After characteristics cross, there is no strong solution of the conservation law, and we must resort to weak solutions.

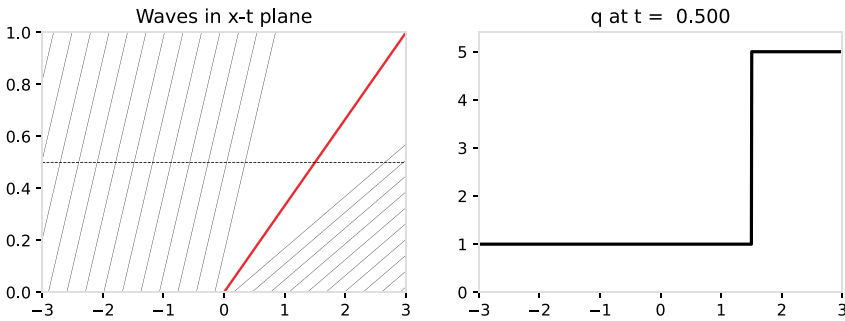
4.3.1 ■ An unphysical weak solution

A potential problem with the notion of weak solutions is that in some cases the weak solution is not unique. For example, consider the Riemann problem for Burgers' equation with $q_\ell < q_r$. As mentioned already, we expect a rarefaction to be the correct solution. This is because if we smeared out the initial data just slightly, then there would be smoothly varying characteristics emerging from each point x and these characteristics would spread out. However, for exactly discontinuous initial data, there exists another weak solution in which the initial discontinuity propagates as a shock wave with the Rankine–Hugoniot speed $(q_\ell + q_r)/2$:

$$q(x, t) = \begin{cases} q_\ell & \text{if } x/t < s, \\ q_r & \text{if } x/t > s. \end{cases}$$

This unphysical solution is also referred to as an expansion shock, and it is also a weak solution to Burgers' equation. In the next figure, we plot this weak solution.

```
In [10]: interact(burgers_demos.unphysical(),
                  t=widgets.FloatSlider(min=0,max=1.0,value=0.5),
                  which_char=widgets.Checkbox(value=True,
                  description='Show characteristics'));
```



Note the behavior of the characteristics with respect to the shock. The fact that the characteristics are spreading away from the shock rather than converging on it indicates that the correct solution should instead be a rarefaction. In order to be able to specify which of the weak solutions is physically correct, we need to derive a mathematical condition from our physical intuition gained from observing the behavior of the characteristics. This condition is referred to as the entropy condition.

4.3.2 ■ The entropy condition

A given initial value problem for a hyperbolic PDE may have many weak solutions. A condition that selects a unique physically correct solution out of these weak solutions is called an *admissibility condition* or more often an *entropy condition*. This name comes from gas dynamics, where the physical entropy must increase in the gas passing through a shock, according to the second law of thermodynamics. A discontinuity that violates this is nonphysical. A mathematical "entropy

function" with a similar property can often be defined for other conservation laws. More discussion of this and several other formulations of admissibility conditions, with more detailed explanations, is available in the literature, e.g., in many of the books cited in the preface.

In the context of the Riemann problem, the entropy condition allows us to determine whether the physical solution should involve a shock or a rarefaction. In the case of scalar conservation laws, the entropy condition is quite simple and can be formulated in terms of the flux function of the conservation law as follows: the solution of a scalar Riemann problem will consist of a shock only if

$$f'(q_\ell) > f'(q_r).$$

In other words, the solution is a shock only if nearby characteristics from the left and right approach the shock as time progresses. This is often called the *Lax entropy condition*. If nearby characteristics are spreading out, as in the last example, the correct solution is instead a rarefaction. In the case of Burgers' equation, the flux function is $f(q) = q^2/2$, so the correct solution is a shock only if

$$q_\ell > q_r,$$

which can be clearly observed in the interactive solution below (in the notebook). In later chapters, we will see how this condition generalizes to systems of conservation laws.

4.3.3 ■ Convexity

The flux $f(q) = q^2/2$ for Burgers' equation is a convex function, since $f''(q) = 1 > 0$ for all values of q . This means that as q varies between q_ℓ and q_r , the characteristic speed $f'(q) = q$ is either monotonically increasing (if $q_\ell < q_r$) or monotonically decreasing (if $q_\ell > q_r$). Hence for any Riemann problem the characteristic speeds for intermediate states are either purely converging or purely diverging, and the Riemann solution is always either a single rarefaction wave or a single shock wave.

The flux $f(\rho) = \rho(1 - \rho)$ of the simple traffic flow model that we will consider in Chapter 5 also has the property that $f'(\rho)$ varies monotonically with ρ (in the context of hyperbolic PDEs, a flux function is referred to as *convex* if either $f''(q) \geq 0$ for all q or $f''(q) \leq 0$ for all q). Since f'' is always negative for the traffic flow model, the correct Riemann solution is a shock if $\rho_\ell < \rho_r$ or a rarefaction wave if $\rho_\ell > \rho_r$.

The solution to the Riemann problem can be much more complicated if $f'(q)$ is not monotonically varying between q_ℓ and q_r , i.e., if $f''(q)$ changes sign. In this case the Riemann solution can consist of multiple shock and rarefaction waves. The nonconvex case is explored further in Chapter 6.

4.4 ■ Interactive solution and examples

In the live notebook, we include an interactive solution of the Riemann problem for Burgers' equation. The values of the initial conditions and the time can be modified to observe their effect on the characteristic structure and the solution.

4.5 ■ Animations of more complex solutions

We can now explore a few more examples that are representative of phenomena we will observe in more complicated systems, with more interesting initial data than the single jump discontinuity of a Riemann problem. For the animations, the solution is computed numerically using PyClaw.²⁸

4.5.1 ■ Bump initial conditions

The animation, found in the notebook or on this webpage,²⁹ shows the solution of Burgers' equation for an initial Gaussian hump, as discussed at the beginning of this chapter. A shock forms on the leading edge, and the trailing edge spreads out as a rarefaction.

4.5.2 ■ Three-state data with merging shocks

For an initial condition that is piecewise constant with three values, one can solve Burgers' equation by solving a Riemann problem locally for each discontinuity and then considering how the resulting waves interact. For instance, consider the initial condition

$$q_0(x) = \begin{cases} q_\ell & \text{if } x < -1, \\ q_m & \text{if } -1 \leq x \leq 1, \\ q_r & \text{if } x > 1. \end{cases}$$

We can decompose this into two Riemann problems: one at $x = -1$ and another at $x = 1$. Each of these two Riemann problems will yield either a shock or a rarefaction. The interesting part is what happens when the generated shocks or rarefactions interact. In the scenario shown below, let us assume that both Riemann problems produce a right-propagating shock. However, the shock generated at $x = -1$ propagates faster, so it will eventually reach the shock originally generated at $x = 1$. At the point in time when the shocks collide, one can simply restate the problem as a Riemann problem and solve the whole problem analytically. In this animation, the plot on the left shows the solution $q(x)$ evolving through time, while the one on the right shows the characteristic structure in the x - t plane, with the shocks shown as wide lines and the characteristics as thin lines, and with time marked by a horizontal dashed line. In this animation, one can clearly see the two shocks merge to become a single shock at later times. This animation can be viewed in the live notebook or on this webpage.³⁰

4.5.3 ■ Three-state data with interacting shock and rarefaction

As one would expect, this can become more complicated when one or both of the waves generated are given by rarefactions. The animation (see the notebook or this webpage³¹) shows how a right-propagating shock can overtake a rarefaction.

²⁸<http://www.clawpack.org/pyclaw/>

²⁹http://www.clawpack.org/riemann_book/html/burgers_animation0.html

³⁰http://www.clawpack.org/riemann_book/html/burgers_animation1.html

³¹http://www.clawpack.org/riemann_book/html/burgers_animation2.html

The shock speed changes as it passes through the rarefaction, since the state just to the left of the shock is changing. This is seen clearly in the x - t plane, where the slope of the shock is changed after interacting with the rarefaction. Note that the shock is again marked as a wider line.

Analogously, one can also have a rarefaction overtaking a shock. As seen in the animation (see the notebook or this webpage³²), this can even change the direction in which the shock is propagating. This happens because the velocity of the shock depends on the slopes of the impinging characteristics. When a shock interacts with a rarefaction, the varying slopes of the characteristic curves produce a constant change in the shock propagation velocity.

In the notebook, try modifying the values of q_ℓ , q_m , and q_r to see what other behavior can be observed. Is it possible to observe two interacting rarefactions?

³²http://www.clawpack.org/riemann_book/html/burgers_animation3.html

Chapter 5

Traffic flow: The Lighthill–Whitham–Richards model

In this chapter we investigate a conservation law that models the flow of traffic. This model is sometimes referred to as the Lighthill–Whitham–Richards (LWR) traffic model (see [10] and [13]). This model and the corresponding Riemann problem are discussed in many places; the discussion here is most closely related to that in Chapter 11 of [9].

This nonlinear scalar problem is similar in many ways to Burgers' equation, which we discussed in Chapter 4, since both involve a quadratic (and hence convex) flux function. In this chapter we repeat some of the discussion from Chapter 4 to reinforce essential concepts that will be important throughout the remainder of the book.

If you wish to examine the Python code for this chapter, please see

- `exact_solvers/traffic_LWR.py` ... on GitHub,³³
- `exact_solvers/traffic_demos.py` ... on GitHub.³⁴

5.1 ■ The LWR model

Recall the continuity equation for any density that is advected with a flow:

$$\rho_t + (u\rho)_x = 0.$$

In this chapter, ρ represents the density of cars on a road, traveling with velocity u . Note that we're not keeping track of the individual cars but just of the average number of cars per unit length of road. Thus $\rho = 0$ represents an empty stretch of road, and we can choose the units so that $\rho = 1$ represents bumper-to-bumper traffic.

³³https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/traffic_LWR.py

³⁴https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/traffic_demos.py

We'll also choose units so that the speed limit is $u_{\max} = 1$ and assume that drivers never go faster than this (yeah, right!). If we assume that drivers always travel at a single uniform velocity, we obtain once again the advection equation that we studied in Chapter 2. But we all know that's not accurate in practice—cars go faster in light traffic and slower when there is congestion. The simplest way to incorporate this effect is to make the velocity a linearly decreasing function of the density:

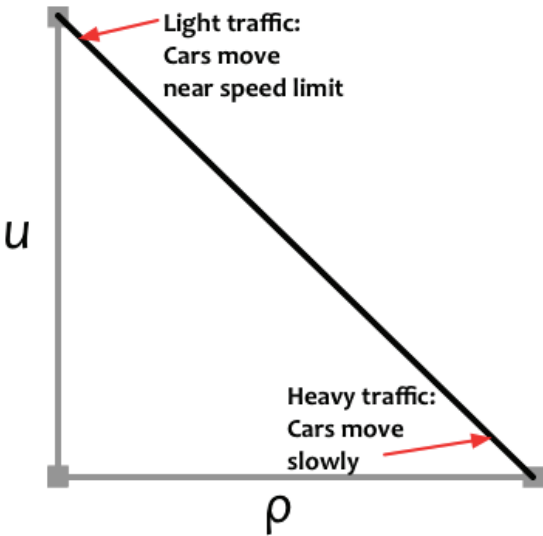
$$u(\rho) = 1 - \rho.$$

Notice that u goes to zero as ρ approaches the maximum density of 1, while u goes to the maximum value of 1 as traffic density goes to zero. Obviously, both ρ and u should always stay in the interval $[0, 1]$.

Here is a plot of this velocity function:

```
In [3]: Image('figures/LWR-Velocity.png', width=350)
```

Out[3]:



Combining the two equations above, our conservation law says

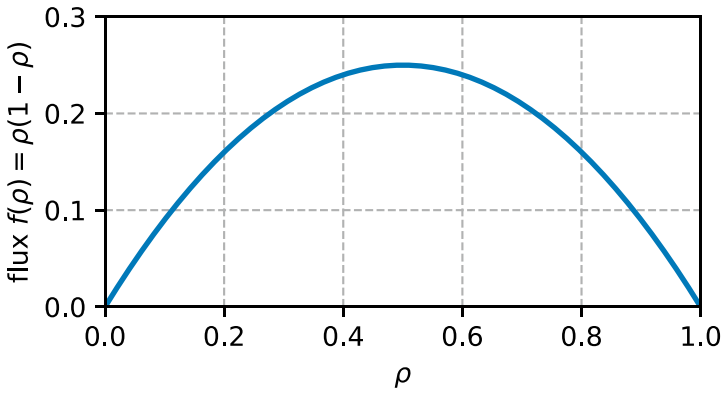
$$\rho_t + (\rho(1 - \rho))_x = 0,$$

with the flux function

$$f(\rho) = \rho(1 - \rho)$$

giving the rate of flow of cars. Notice how the flux is zero when there are no cars ($\rho = 0$) and also when the road is completely full ($\rho = 1$). The maximum flow of traffic actually occurs when the road is half full, as the plot below shows.

```
In [4]: f = lambda rho: rho*(1-rho)
        traffic_demos.plot_flux(f)
```



Like the flux in Burgers' equation, the LWR flux is **nonlinear**, so we again expect to see shock waves and rarefaction waves in the solution. We can superficially make this equation look like the advection equation by using the chain rule to write it in quasilinear form:

$$f(\rho)_x = f'(\rho)\rho_x = (1 - 2\rho)\rho_x.$$

Then we have

$$\rho_t + (1 - 2\rho)\rho_x = 0.$$

This is like the advection equation, but with a velocity $1 - 2\rho$ that depends on the density of cars. The value $f'(\rho) = 1 - 2\rho$ is referred to as the *characteristic speed*. This characteristic speed is not the speed at which cars move (notice that it can even be negative, whereas cars only drive to the right in our model). Rather, it is the speed at which *information* is transmitted along the road. Notice that the LWR flux is not convex but concave; because of this, the characteristic speed is a decreasing function of the density.

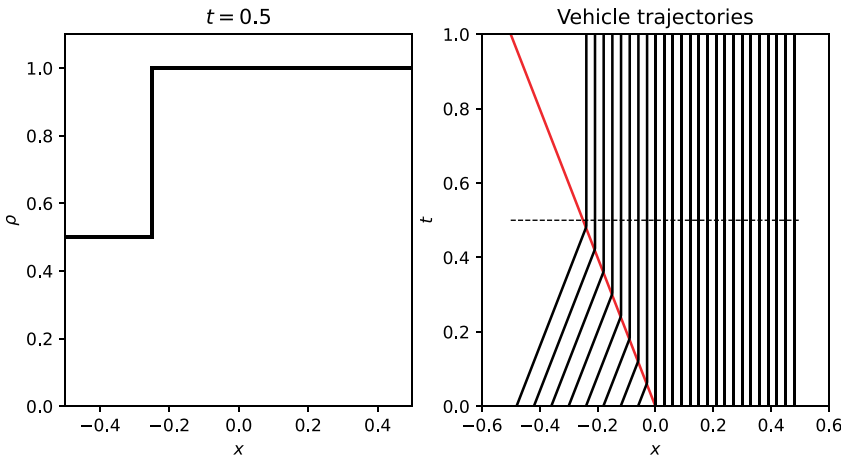
5.2 ■ Example: Traffic jam

What does our model predict when traffic approaches a totally congested ($\rho = 1$) area? This might be due to construction, an accident, or a red light somewhere to the right; upstream of the obstruction, cars will be bumper-to-bumper, so we set $\rho = 1$ for $x > 0$ (supposing that traffic has backed up to that point). For $x < 0$ we'll assume a lower density, $\rho_\ell < 1$. This is another example of a Riemann problem: two constant states separated by a discontinuity. We have

$$\rho(x, t = 0) = \begin{cases} \rho_\ell, & x < 0, \\ 1, & x > 0. \end{cases}$$

What will happen as time goes forward? Intuitively, we expect traffic to continue backing up to the left, so the region with $\rho = 1$ will extend further and further to the left. This corresponds to the discontinuity (or shock wave) moving to the left. How quickly will it move? The example below shows the solution (on the left) and individual vehicle trajectories in the x - t plane (on the right).

```
In [5]: interact(traffic_demos.jam,
               rho_l=FloatSlider(min=0.1,max=0.9,value=0.5,
                                description=r'\rho_l$'),
               t=FloatSlider(min=0.,max=1.,value=0.5), fig=fixed(0));
```



Note that the vehicle trajectory plot above shows the motion of cars moving at velocity $u_\ell = 1 - \rho_\ell > 0$ as they approach the traffic jam and at speed $u_r = 1 - \rho_r = 0$ to the right of the shock, where the cars are stationary.

Unlike the case of linear advection, the *characteristic speeds* are different, with $f'(\rho_\ell) = 1 - 2\rho_\ell$, which could be either negative or positive depending on ρ_ℓ , and $f'(\rho_r) = -1$.

5.3 ■ Speed of a shock wave: The Rankine–Hugoniot condition

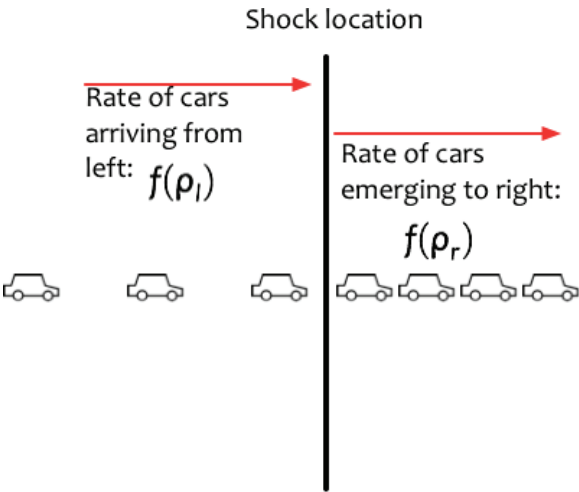
In the plot above, we see a shock wave (i.e., a discontinuity) that moves to the left as more and more cars pile up behind the traffic jam. How quickly does this discontinuity move to the left?

We can figure it out by putting an imaginary line at the location of the shock, as shown in the next figure.

Let ρ_ℓ be the density of cars just to the left of the line, and let ρ_r be the density of cars just to the right. Imagine for a moment that the line is stationary. Then the rate of cars reaching the line from the left is $f(\rho_\ell)$ and the rate of cars departing from the line to the right is $f(\rho_r)$. If the line really were stationary, we would need to have $f(\rho_\ell) - f(\rho_r) = 0$ to avoid cars accumulating at the line.

In [6]: `Image('figures/shock_diagram_traffic_a.png', width=350)`

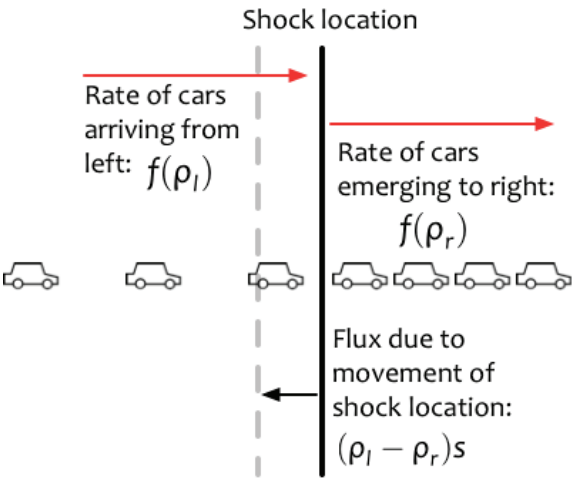
Out[6]:



However, the shock is not stationary, so the line is moving. Let s be the speed of the shock. Then as the line moves to the left, some cars that were to the left are now to the right of the line. The rate of cars removed from the left is $s\rho_l$ and the rate of cars added on the right is $s\rho_r$, as shown in this figure:

In [7]: `Image('figures/shock_diagram_traffic_b.png', width=350)`

Out[7]:



So to avoid an infinite density of cars at the shock, these two effects need to be balanced:

$$f(\rho_l) - f(\rho_r) = s(\rho_l - \rho_r).$$

This same condition was used for Burgers' equation in Chapter 4 and is known as the **Rankine–Hugoniot condition**. It holds for any shock wave in the solution

of any hyperbolic PDE (even systems of equations, where the corresponding vector version gives even more information about the structure of allowable shock waves).

Returning to our traffic jam scenario, we set $\rho_r = 1$. Then we find that the Rankine–Hugoniot condition gives the shock speed

$$s = \frac{f(\rho_\ell) - f(\rho_r)}{\rho_\ell - \rho_r} = \frac{f(\rho_\ell)}{\rho_\ell - 1} = -\rho_\ell.$$

This makes sense: the traffic jam propagates back along the road, and it does so more quickly if there is a greater density of approaching cars.

5.4 ■ Example: Green light

What about when a traffic light turns green? At $t = 0$, when the light changes, there will be a discontinuity, with traffic backed up behind the light but little or no traffic after the light. With the light at $x = 0$, this takes the form of another Riemann problem:

$$\rho(x, t = 0) = \begin{cases} 1, & x < 0, \\ \rho_r, & x > 0, \end{cases}$$

with $\rho_r = 0$, for example. In this case we don't expect the discontinuity in density to propagate. Physically, the reason is clear: after the light turns green, the cars in front accelerate and spread out; then the cars behind them accelerate, and so forth. This kind of expansion wave is referred to as a *rarefaction wave* because the drivers experience a decrease in density (a rarefaction) as they pass through this wave. Initially, the solution is discontinuous, but after time zero it becomes continuous.

5.4.1 ■ Similarity solutions

The exact form of the solution at a green light can be determined by assuming that the solution $\rho(x, t)$ depends only on x/t . A solution with this property is referred to as a *similarity solution* because it remains the same if we rescale both x and t by the same factor. The solution of any Riemann problem is, in fact, a similarity solution. Writing $\rho(x, t) = \tilde{\rho}(x/t)$, we have (with $\xi = x/t$)

$$\rho_t = -\frac{x}{t^2} \tilde{\rho}'(\xi), \quad f(\rho)_x = \frac{1}{t} \tilde{\rho}'(\xi) f'(\tilde{\rho}(\xi)).$$

Thus

$$\rho_t + f(\rho)_x = -\frac{x}{t^2} \tilde{\rho}'(\xi) + \frac{1}{t} \tilde{\rho}'(\xi) f'(\tilde{\rho}(\xi)) = 0. \quad (5.1)$$

This can be solved to find

$$f'(\tilde{\rho}(\xi)) = \frac{x}{t} \quad (5.2)$$

or, since $f'(\tilde{\rho}) = 1 - 2\tilde{\rho}$,

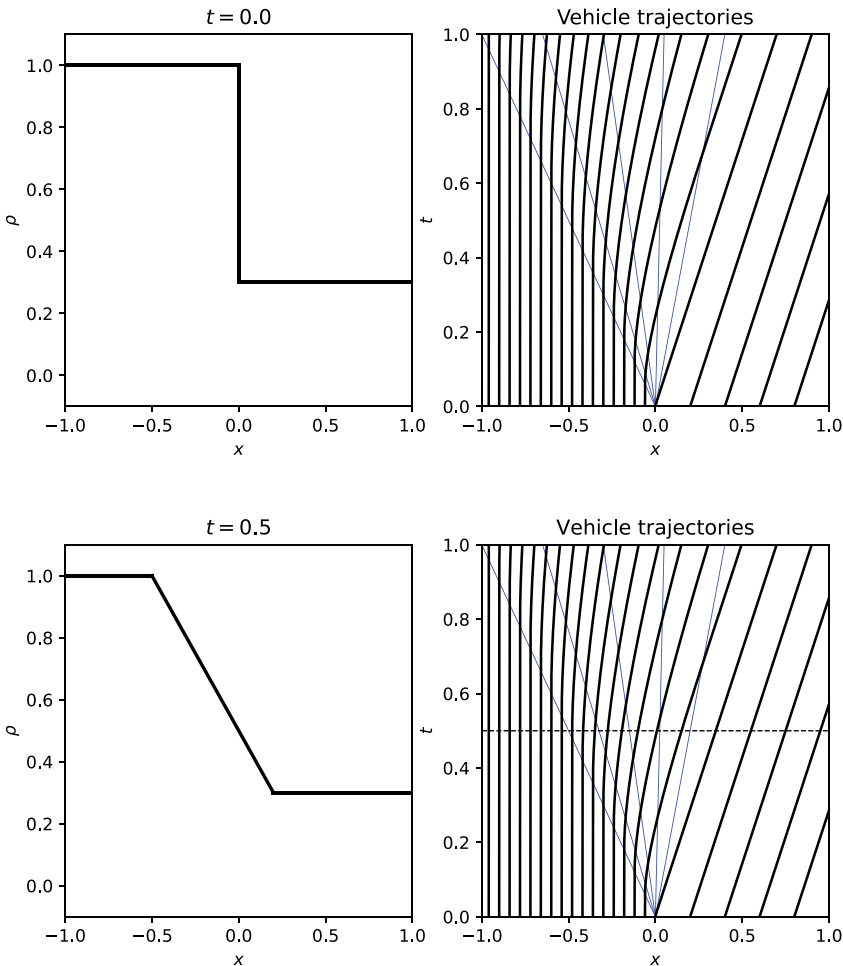
$$\tilde{\rho}(\xi) = \frac{1}{2} \left(1 - \frac{x}{t} \right). \quad (5.3)$$

We know that the solution far enough to the left is just $\rho_\ell = 1$, and far enough to the right it is ρ_r . The formula above gives the solution in the region between these constant states. For instance, if $\rho_r = 0$ (i.e., the road beyond the light is empty at time zero), then

$$\rho(x,t) = \begin{cases} 1, & x/t \leq -1, \\ \frac{1}{2}(1 - x/t), & -1 < x/t < 1, \\ 0, & 1 \leq x/t. \end{cases} \tag{5.4}$$

The plot below shows the solution density and vehicle trajectories for a green light at $x = 0$.

```
In [8]: interact(traffic_demos.green_light,
               rho_r=FloatSlider(min=0.,max=0.9,value=0.3,
                                description=r'$\rho_r$'),
               t=FloatSlider(min=0.,max=1.), fig=fixed(0));
```



In the x - t plane plot above, the black curves show vehicle trajectories, while the blue rays are characteristics corresponding to values of ρ between $\rho_\ell = 1$ and ρ_r , with the leftmost characteristic following $x = f'(q_\ell)t$ and the rightmost characteristic following $x = f'(q_r)t$.

5.5 ▪ Entropy condition

How can we determine whether an initial discontinuity will lead to a shock or a rarefaction? We have already addressed this for scalar equations in Chapter 4. Recall the Lax entropy condition introduced there:

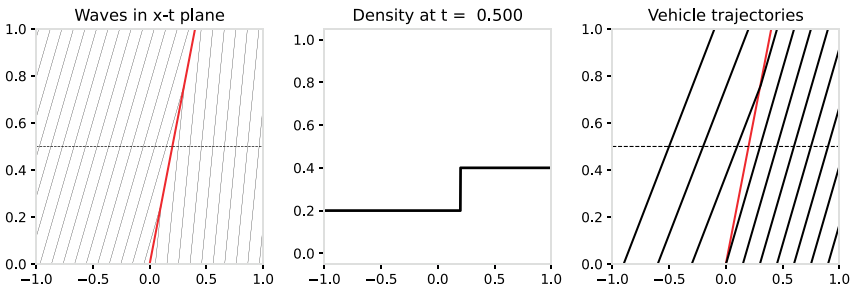
- Shocks appear in regions where characteristics converge, as in the traffic jam example above.
- Rarefactions appear in regions where characteristics are spreading out, as in the green light example.

More precisely, if the solution is a shock wave with left state ρ_ℓ and right state ρ_r , then it must be that $f'(\rho_\ell) > f'(\rho_r)$. In fact the shock speed must lie between these characteristic speeds:

$$f'(\rho_\ell) > s > f'(\rho_r).$$

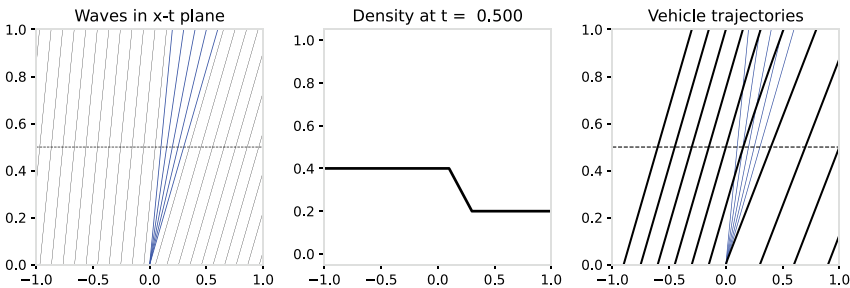
We say that the characteristics *impinge* on the shock.
Here is an example showing the characteristics near a shock:

```
In [9]: traffic_LWR.plot_riemann_traffic(0.2,0.4,t=0.5)
```



On the other hand, if $f'(\rho_\ell) < f'(\rho_r)$, then a rarefaction wave results and the initial discontinuity immediately spreads out. Here is an example showing the characteristics in this case:

```
In [10]: traffic_LWR.plot_riemann_traffic(0.4,0.2,t=0.5)
```



5.6 ■ Other resources

Many other traffic flow models exist. Some of them are *continuum models*, like the one presented here, that model traffic density and velocity as an aggregate. Others are *particle* or *agent* models that simulate individual vehicles. You can see simulations of the latter kind on this webpage.³⁵ Shock waves and rarefaction waves naturally appear in such agent-based models too, and viewing some of these simulations may give you better intuition for the development of shocks and rarefactions in the context of traffic jams.

³⁵<http://www.traffic-simulation.de/routing.html>

Chapter 6

Nonconvex scalar conservation laws

The scalar nonlinear conservation law $q_t + f(q)_x = 0$ is said to be *genuinely nonlinear* over some range of states between q_ℓ and q_r if $f''(q) \neq 0$ for all q between these values. This is true in particular if $f(q)$ is any quadratic function; for example, the flux function for Burgers' equation and the LWR traffic flow model are genuinely nonlinear for all q . This is important because it means that the characteristic speed $f'(q)$ is either monotonically increasing or monotonically decreasing over the interval between q_ℓ and q_r . As discussed already in Chapters 4 and 5, if $f'(q)$ is increasing as q varies from q_ℓ to q_r , then the initial discontinuity of a Riemann problem spreads out into a smooth single-valued rarefaction wave. On the other hand, if $f'(q)$ is decreasing, then the discontinuity propagates as a single shock wave with speed given by the Rankine–Hugoniot condition. Hence the Riemann solution for a genuinely nonlinear scalar equation always consists of either a single rarefaction wave or a single shock.

The solution can be much more complicated if $f''(q)$ vanishes somewhere between q_ℓ and q_r , since this means that the characteristic speed may not vary monotonically. As we will illustrate below, for a scalar conservation law of this type the solution to the Riemann problem might consist of multiple shocks and rarefaction waves. These waves all originate from $x = 0$, and the Riemann solution is still a similarity solution $q(x, t) = Q(x/t)$ for some single-valued function $Q(\xi)$, but determining the correct set of waves is more challenging.

In this chapter, we focus on *scalar nonconvex* problems and present results computed using an elegant form of the exact solution for a general scalar conservation law due to Osher. At the end of the chapter we comment briefly on implications for systems of equations.

6.1 ■ Osher's solution

In this chapter we use Osher's general solution to the scalar nonlinear Riemann problem (valid also for nonconvex fluxes), using the formula from [12]. The Riemann solution is always a similarity solution $q(x, t) = Q(x/t)$ for all $t > 0$ (constant on any ray $x = \alpha t$ emanating from the origin, for any constant α).

The function $Q(\xi)$ is given by

$$Q(\xi) = \begin{cases} \operatorname{argmin}_{q_\ell \leq q \leq q_r} [f(q) - \xi q] & \text{if } q_\ell \leq q_r, \\ \operatorname{argmax}_{q_r \leq q \leq q_\ell} [f(q) - \xi q] & \text{if } q_r \leq q_\ell. \end{cases}$$

Recall that $\operatorname{argmin}_{q_\ell \leq q \leq q_r} G(q)$ returns the value of q for which $G(q)$ is minimized over the indicated interval, while argmax returns the value of q where $G(q)$ is maximized. For more discussion, see also Section 16.1 of [9].

If you wish to examine the Python code for this chapter, please see

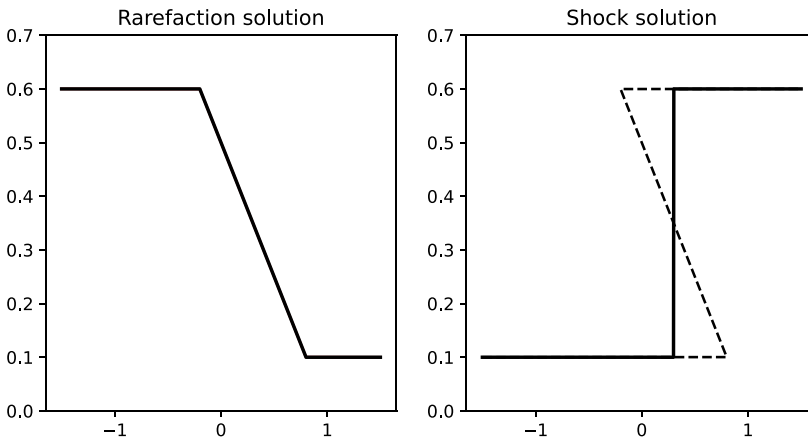
- `exact_solvers/nonconvex.py` ... on GitHub,³⁶
- `exact_solvers/nonconvex_demos.py` ... on GitHub.³⁷

Note that in `exact_solvers/nonconvex.py` we use the function `osher_solution` to define a function `nonconvex_solutions` that evaluates this solution at a set of $\mathbf{x} = \mathbf{x}/t$ values. It also computes the possibly multivalued solution that would be obtained by tracing characteristics, for plotting purposes. In `exact_solvers/nonconvex_demos.py`, an additional function `make_plot_function` returns a plotting function for use in the interactive widgets below.

6.2 ■ Traffic flow

First we recall that the Riemann solution for a problem with convex flux consists of a single shock or rarefaction wave. For example, consider the flux function $f(q) = q(1 - q)$ from traffic flow (with q now representing the density ρ used in Chapter 5).

In [3]: `nonconvex_demos.demo1()`



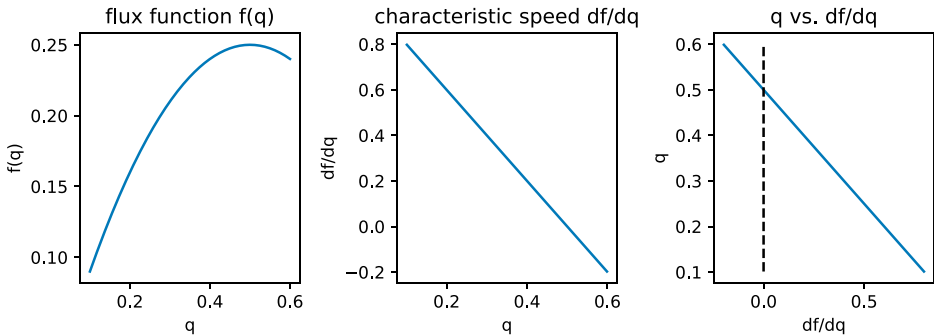
The plot on the left above shows a case where the solution is a rarefaction wave that can be computed by tracing characteristics. On the right we see the case for which tracing characteristics would give a multivalued solution (as a dashed line), whereas the correct Riemann solution consists of a shock wave (solid line). As discussed in Chapter 5, the shock location is determined by the Rankine–Hugoniot condition or, equivalently, by the equal-area condition discussed in Chapter 4.

³⁶https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/nonconvex.py

³⁷https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/nonconvex_demos.py

For comparison with later examples, we also plot the quadratic flux function $f(q)$ and the linear characteristic speed $f'(q)$ for this range of q values. Plotting q versus the characteristic speed shows how we can interpret each value of q in the jump discontinuity (represented by the dashed vertical line in the plot on the right below) as propagating to the left or right at its characteristic speed. Since $f'(q)$ is linear in q , the rarefaction wave shown above is piecewise linear.

```
In [4]: f = lambda q: q*(1-q)
        q_left = 0.1
        q_right = 0.6
        nonconvex_demos.plot_flux(f, q_left, q_right)
```



6.3 ▪ Buckley–Leverett equation

The Buckley–Leverett equation for two-phase flow is described in Section 16.1.1 of [9]. It has the nonconvex flux function

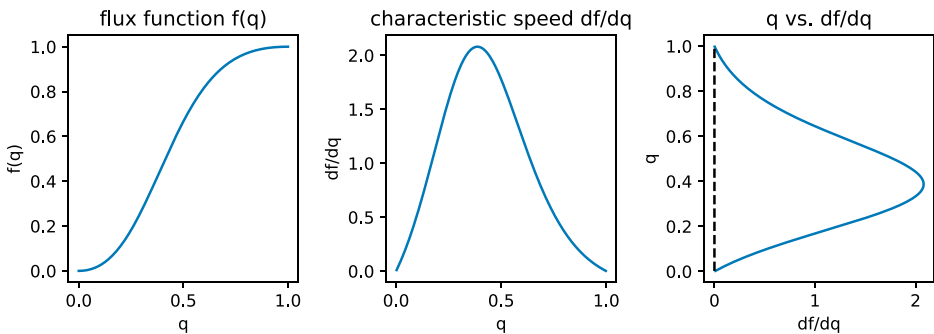
$$f(q) = \frac{q^2}{q^2 + a(1 - q)^2},$$

where a is some constant and $q = 1$ corresponds to pure water and $q = 0$ to pure oil in a saturated porous medium.

Here we make plots like the ones above, but for the Buckley–Leverett flux.

```
In [5]: a = 0.5
        f_buckley_leverett = lambda q: q**2 / (q**2 + a*(1-q)**2)
        q_left = 1.
        q_right = 0.

        nonconvex_demos.plot_flux(f_buckley_leverett, q_left, q_right)
```



Again the third plot above shows q on the vertical axis and $f'(q)$ on the horizontal axis (it's the middle figure turned sideways). You can think of this as showing the characteristic velocity for each point on a jump discontinuity from $q = 0$ to $q = 1$ (indicated by the dashed line), and hence a triple-valued solution of the Riemann problem at $t = 1$ when each q value has propagated this far.

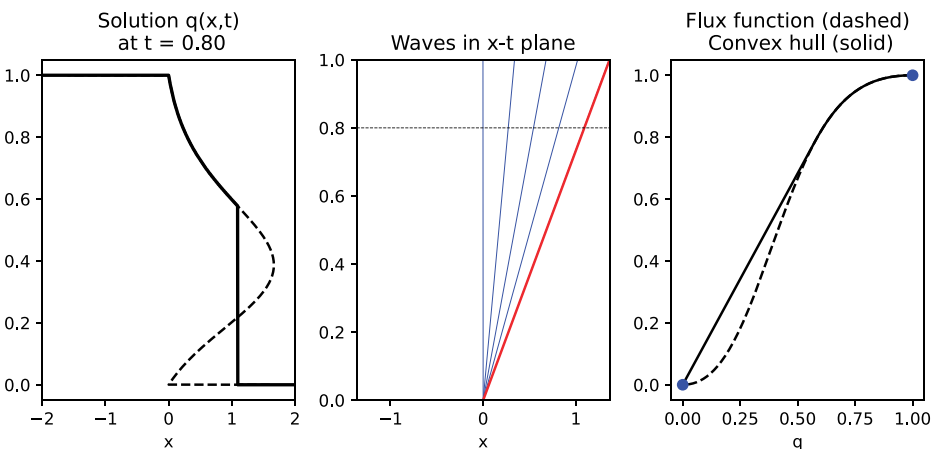
6.3.1 ■ The correct Riemann solution

Below we show this triple-valued solution together with the correct solution to the Riemann problem, with a shock wave inserted at the appropriate point (as computed using the Osher solution defined above). Note that for this nonconvex flux function the Riemann solution contains both a rarefaction wave and a shock wave.

In the plot on the right, we also show the flux function $f(q)$ as a red curve and the upper boundary of the convex hull of the set of points below the graph for $q_r \leq q \leq q_\ell$. Note that the convex hull boundary follows the flux function for the set of q values corresponding to the rarefaction wave and then jumps from $q \approx 0.6$ to $q = 0$, corresponding to the shock wave. See Section 16.1 of [9] for more discussion of this construction of the Riemann solution.

```
In [6]: q_left = 1.
        q_right = 0.
        plot_function = nonconvex_demos.make_plot_function(f_buckley_leverett,
                                                            q_left, q_right, xi_left=-2, xi_right=2)

        interact(plot_function,
                  t=widgets.FloatSlider(value=0.8,min=0,max=.9),
                  fig=fig.fixed(0));
```



Note from the plot on the left above that the triple-valued solution suggested by tracing characteristics (the dashed line) has been partially replaced by a shock wave. By conservation, the areas of the two regions cut off by the shock must

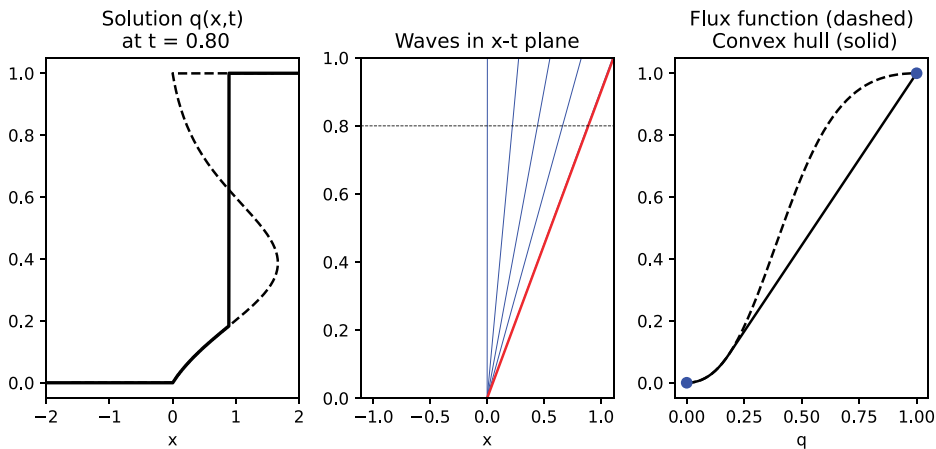
cancel out. Moreover, the shock speed coincides with the characteristic speed at the edge of the rarefaction wave that ends at the shock. In terms of the flux function shown by the dashed curve in the rightmost figure above, we see that the shock wave connects $q_r = 0$ to the point where the slope of the linear segment of the solid line (which is the shock speed, by the Rankine–Hugoniot condition) agrees with the slope of the flux function (which is the characteristic speed at this edge of the rarefaction wave).

Note that the correct Riemann solution lies along the upper *convex hull* of the flux function (i.e., the convex hull of the region below the curve $f(q)$). We will see below that this is true more generally, even for flux functions with many local maxima or minima. If $q_\ell > q_r$, then the upper convex hull of the flux function between these points illustrates the Riemann solution. On the other hand, if $q_\ell < q_r$, then the lower convex hull gives the Riemann solution, as illustrated in the next example.

6.3.2 ■ Swapping left and right states

Note what happens if we switch q_{left} and q_{right} in this problem. This now corresponds to oil on the left pushing into water on the right.

```
In [7]: q_left = 0.  
        q_right = 1.  
        plot_function = nonconvex_demos.make_plot_function(f_buckley_leverett,  
                                                            q_left, q_right, xi_left=-2, xi_right=2)  
  
        interact(plot_function,  
                  t=widgets.FloatSlider(value=0.8,min=0,max=.9),  
                  fig=fixed(0));
```



Again the shock replaces the triple-valued part of the solution in such a way that mass is conserved (equal areas are cut off), and the shock speed again agrees with the limiting characteristic speed at the edge of the adjacent rarefaction wave.

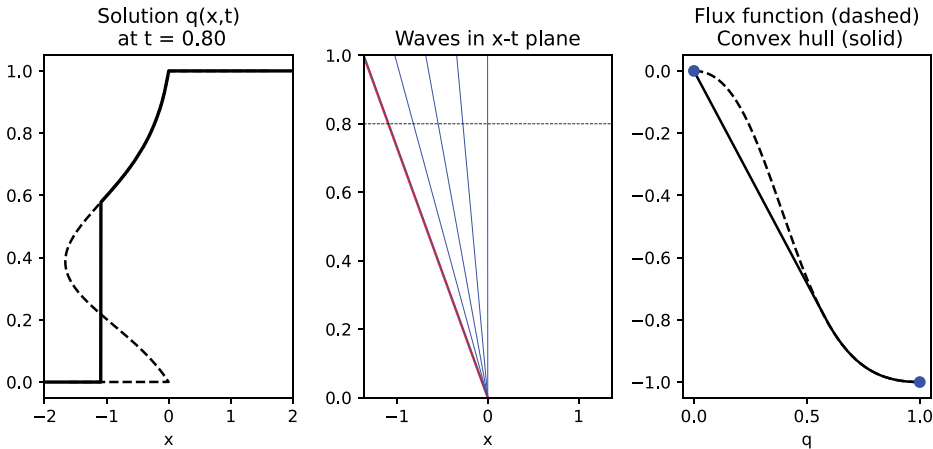
Since $q_\ell < q_r$ in this case, the correct Riemann solution corresponds to the lower convex hull of the flux function, i.e., the convex hull of the set of points lying above $f(q)$.

6.3.3 ■ Leftward flow

The Buckley–Leverett equation as written above simulates flow from left to right. So if we want to model water on the right pushing leftward into oil on the left, we must negate the flux function, as is done in the next cell. Note that this gives the same solution as our original Riemann problem, but flipped in x .

```
In [8]: f_BL_leftward = lambda q: -f_buckley_leverett(q)
        q_left = 0.
        q_right = 1.
        plot_function = nonconvex_demos.make_plot_function(f_BL_leftward,
                                                             q_left, q_right, xi_left=-2, xi_right=2)

        interact(plot_function,
                  t=widgets.FloatSlider(value=0.8,min=0,max=.9),
                  fig=fixed(0));
```

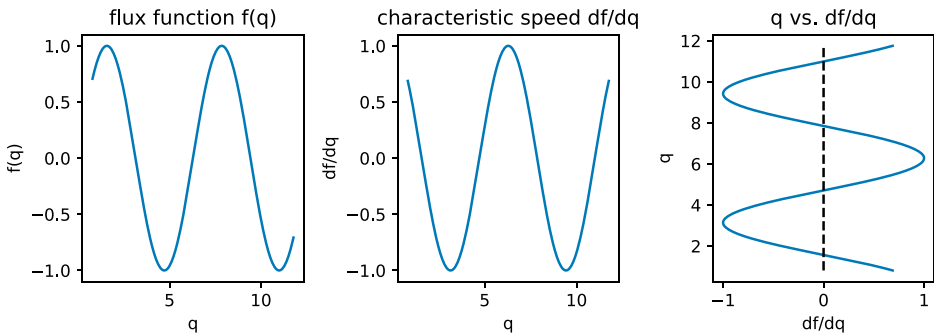


6.4 ■ Sinusoidal flux

As another test, we consider the flux function $f(q) = \sin(q)$, which is also used in Example 16.1 of [9]. First we plot the flux function $f(q)$ and also the characteristic speed df/dq . We also plot q as a function of df/dq . This again helps us visualize whether the characteristic speed is positive or negative for each value of q along the jump discontinuity in the Riemann data (again indicated by the vertical dashed line) and shows that trying to solve the Riemann problem by tracing characteristics alone would lead to a multivalued solution.

```
In [9]: f_sin = lambda q: np.sin(q)
        q_left = np.pi/4.
        q_right = 15*np.pi/4.

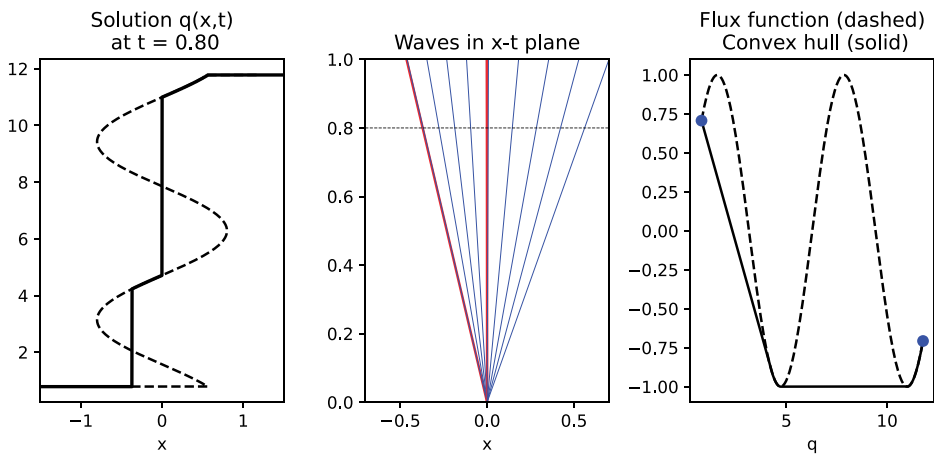
        nonconvex_demos.plot_flux(f_sin, q_left, q_right)
```



Below is a recreation of Figure 16.4 of [9], illustrating where shocks must be inserted to make the Riemann solution single valued. In the live notebook you can see how the solution evolves with time.

```
In [10]: plot_function = \
        nonconvex_demos.make_plot_function(f_sin, q_left, q_right, -1.5, 1.5)

        interact(plot_function,
                  t=widgets.FloatSlider(value=0.8,min=0.,max=.9),
                  fig=fixed(0));
```



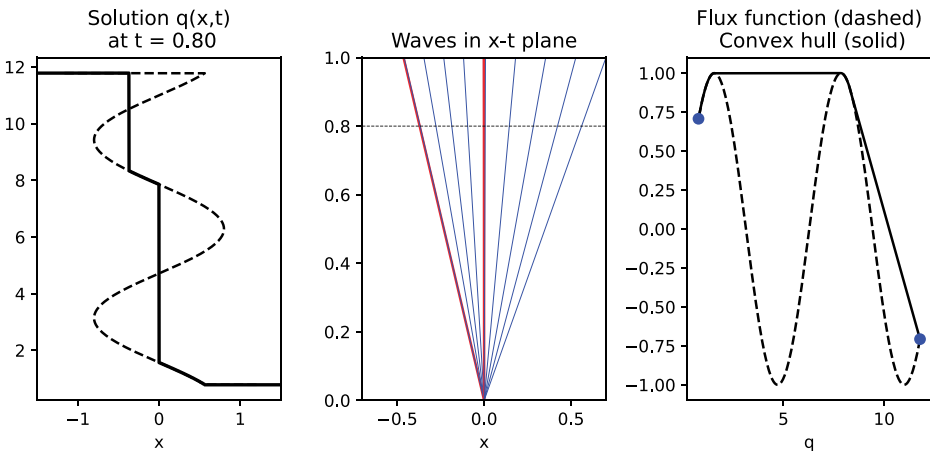
In the figure above, note that the shocks in the Riemann solution correspond to linear segments of the lower convex hull of the flux function $f(q)$. This is because we chose $q_\ell < q_r$ in this example.

If we switch the states so that $q_\ell > q_r$, then the Riemann solution corresponds to the upper convex hull of the flux function.

```
In [11]: f_sin = lambda q: np.sin(q)
         q_left = 15*np.pi/4.
         q_right = np.pi/4.

         plot_function = \
             nonconvex_demos.make_plot_function(f_sin, q_left, q_right,
                                                -1.5, 1.5)

         interact(plot_function,
                  t=widgets.FloatSlider(value=0.8,min=0.,max=.9),
                  fig=fixed(0));
```



6.5 ■ Yet another example

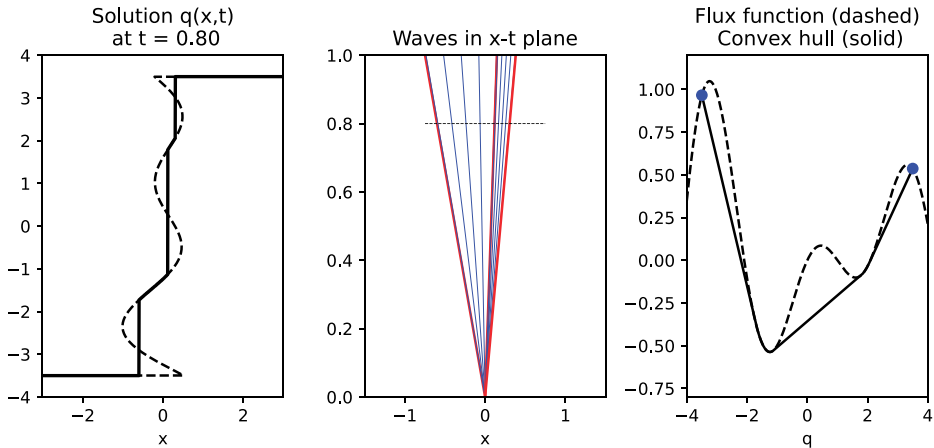
Here's another example that you can run in the live notebook. Observe the collection of shock and rarefaction waves that result from this. *In the notebook you can also adjust q_ℓ and q_r .* Notice how the structure of the solution changes as you vary them. What do you think the Riemann solution would look like if you switched the left and right states? Check whether you are right.

```
In [12]: f = lambda q: 0.25*(1. - q)*np.sin(1.5*q)

         plot_function = nonconvex_demos.make_plot_function_qsliders(f)

         interact(plot_function,
                  t=widgets.FloatSlider(value=0.8,min=0.,max=.9),
                  q_left=widgets.FloatSlider(value=-3.5,min=-4,max=4),
```

```
q_right=widgets.FloatSlider(value=3.5,min=-4,max=4),
fig=fig_fixed(0));
```



Experiment with this and other flux functions in this notebook! But be warned that the plotting functions, as written, may break down if you try an example with too many oscillations in the flux.

6.6 ■ Implications for systems of equations

For hyperbolic systems of m equations, there are m different characteristic fields. For classical nonlinear systems such as the shallow water equations or the Euler equations of gas dynamics, the Riemann solution generally consists of m waves, each of which is either a discontinuity or a rarefaction wave. There is only one wave in each family because, for these systems, each characteristic speed either varies monotonically through the wave (the field is said to be *genuinely nonlinear*) or else is constant across the wave (in which case the field is called *linearly degenerate*). In the former case the wave is either a single shock or rarefaction wave, and in the latter case the wave is a *contact discontinuity*, as discussed further in Chapters 8 and 9. As in the case of scalar advection, the characteristic speed is constant across a contact discontinuity ($f'(q) \equiv 0$ between the left and right states). The definition of genuine nonlinearity and linear degeneracy for systems of equations is given in Chapter 9.

The nonconvex equations studied in this chapter illustrate that even for a *scalar* problem the Riemann solution becomes much more complicated if the problem fails to be genuinely nonlinear (e.g., Burgers' equation or the LWR model for traffic flow) or linearly degenerate (the advection equation). *Systems* of equations that lack the corresponding properties are more difficult still and are beyond the scope of this book, although they do arise in some important applications, such as magnetohydrodynamics (MHD).

Chapter 7

The shallow water equations

In this chapter we study a model for shallow water waves in one dimension:

$$h_t + (hu)_x = 0, \quad (7.1)$$

$$(hu)_t + \left(hu^2 + \frac{1}{2}gh^2 \right)_x = 0. \quad (7.2)$$

Here h is the depth, u is the velocity, and g is a constant representing the force of gravity. These equations are *depth averaged* and neglect vertical velocity and any vertical variations in the horizontal velocity. Viscosity and compressibility are neglected, and the pressure is assumed to be hydrostatic. Nevertheless, this is a surprisingly effective model for many applications, particularly when the wavelength is long compared to the fluid depth.

Previously we have looked at a *linear hyperbolic system* (Chapter 3) and *scalar hyperbolic equations* (Chapters 2, 4, and 5). The shallow water system is our first example of a *nonlinear hyperbolic system*; solutions of the Riemann problem for this system consist of two waves (since it is a system of two equations), each of which may be a shock or a rarefaction (since it is nonlinear).

7.1 ■ Hyperbolic structure

We can write (7.1)–(7.2) in the canonical form $q_t + f(q)_x = 0$ if we define

$$q = \begin{pmatrix} h \\ hu \end{pmatrix}, \quad f = \begin{pmatrix} hu \\ hu^2 + \frac{1}{2}gh^2 \end{pmatrix}. \quad (7.3)$$

In terms of the conserved quantities, the flux is

$$f(q) = \begin{pmatrix} q_2 \\ q_2^2/q_1 + \frac{1}{2}gq_1^2 \end{pmatrix}. \quad (7.4)$$

Thus the flux Jacobian is

$$f'(q) = \begin{pmatrix} 0 & 1 \\ -(q_2/q_1)^2 + gq_1 & 2q_2/q_1 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -u^2 + gh & 2u \end{pmatrix}. \quad (7.5)$$

Its eigenvalues are

$$\lambda_1 = u - \sqrt{gh}, \qquad \lambda_2 = u + \sqrt{gh}, \qquad (7.6)$$

with corresponding eigenvectors

$$r_1 = \begin{bmatrix} 1 \\ u - \sqrt{gh} \end{bmatrix}, \qquad r_2 = \begin{bmatrix} 1 \\ u + \sqrt{gh} \end{bmatrix}. \qquad (7.7)$$

Notice that—unlike acoustics, but similar to the LWR traffic model—the eigenvalues and eigenvectors depend on q . Because of this, the waves appearing in the Riemann problem move at different speeds and may be either jump discontinuities (shocks) or smoothly varying rarefaction waves.

If you wish to examine the Python code for this chapter, see

- `exact_solvers/shallow_water.py` ... on GitHub,³⁸
- `exact_solvers/shallow_demos.py` ... on GitHub.³⁹

For convenience, we will work in units such that the gravitational constant $g = 1$.

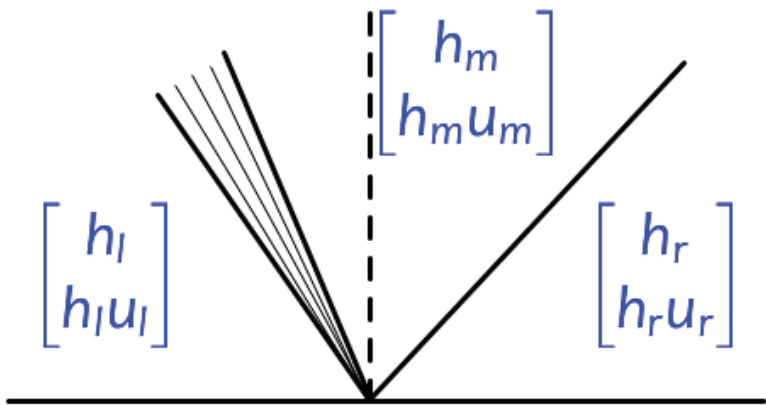
In [3]: `g = 1`.

7.2 ■ The Riemann problem

Consider the Riemann problem with left and right states

$$q_\ell = \begin{bmatrix} h_\ell \\ h_\ell u_\ell \end{bmatrix}, \qquad q_r = \begin{bmatrix} h_r \\ h_r u_r \end{bmatrix}.$$

Typically the Riemann solution consists of two waves, one related to each of the eigenvectors in (7.7). We refer to the wave related to r_1 (the first characteristic family) as a 1-wave and the wave related to r_2 (the second characteristic family) as a 2-wave. Each wave may be a shock or a rarefaction. There is a constant intermediate state q_m between them. In the figure below, we illustrate a typical situation in which the 1-wave happens to be a rarefaction and the 2-wave a shock:



³⁸https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/shallow_water.py

³⁹https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/shallow_demos.py

In the figure we have one wave going in each direction, but since the wave speeds depend on q and can have either sign, it is possible to have both waves going left or both going right. In these cases, the flow is said to be *supercritical*.

To solve the Riemann problem, we must find q_m . To do so we must find a state that can be connected to q_ℓ by a 1-shock or a 1-rarefaction and to q_r by a 2-shock or a 2-rarefaction. We must also ensure that the resulting waves satisfy the entropy condition.

7.3 ■ Shock waves

Let us now consider a shock wave connecting a known state $q_* = (h_*, h_* u_*)$ to an unknown state $q = (h, hu)$. In the context of the Riemann problem, q_* will be the known left or right state and q will be the unknown middle state. The Rankine–Hugoniot jump conditions $s(q_* - q) = f(q_*) - f(q)$ for a shock wave moving at speed s are

$$s(h_* - h) = h_* u_* - hu, \quad (7.8)$$

$$s(h_* u_* - hu) = h_* u_*^2 - hu^2 + \frac{g}{2}(h_*^2 - h^2). \quad (7.9)$$

It is convenient to define $\alpha = h - h_*$. Then we can combine the two equations above to show that the momenta are related by

$$hu = h_* u_* + \alpha \left[u_* \pm \sqrt{gh_* \left(1 + \frac{\alpha}{h_*}\right) \left(1 + \frac{\alpha}{2h_*}\right)} \right] \quad (7.10)$$

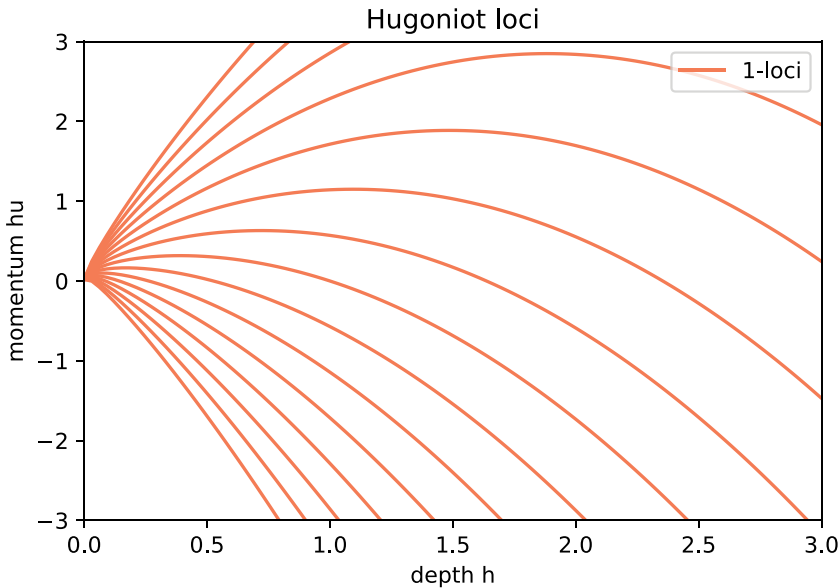
for a shock moving with speed

$$s = \frac{h_* u_* - hu}{h_* - h} = u_* \mp \sqrt{gh_* \left(1 + \frac{\alpha}{h_*}\right) \left(1 + \frac{\alpha}{2h_*}\right)} = u_* \mp \sqrt{gh \frac{h_* + h}{2h_*}}. \quad (7.11)$$

Choosing the plus sign in (7.10) (which yields the minus sign in (7.11)) gives a 1-shock. Choosing the minus sign in (7.10) (which yields the plus sign in (7.11)) gives a 2-shock. Notice from the last expression in (7.11) that as $h \rightarrow h^*$ the shock speed approaches the corresponding characteristic speed (7.6), while the ratio of the jumps in momentum and depth approaches that suggested by the eigenvectors (7.7).

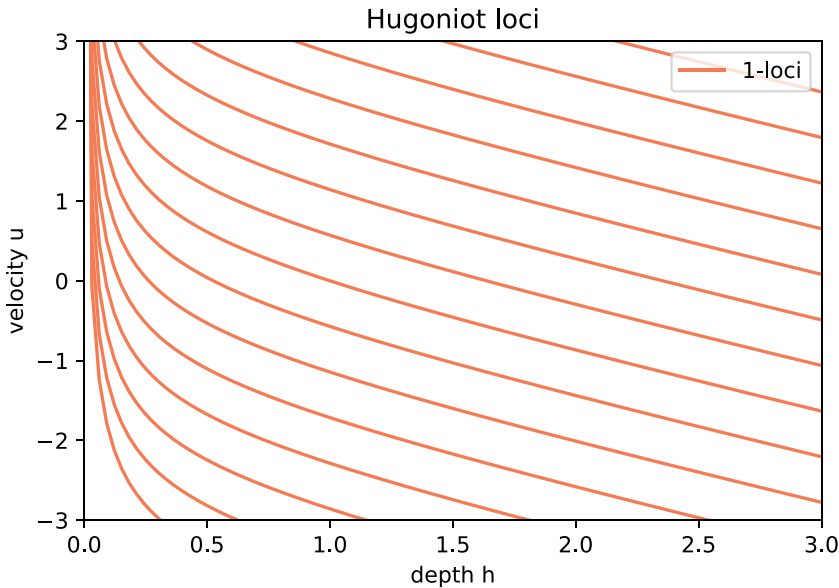
We can now consider the set of all possible states (h, hu) satisfying (7.10). This curve in the h - hu plane is known as the Hugoniot locus; there is a family of Hugoniot loci for 1-shocks and another for 2-shocks. Below we plot some members of the 1-loci of curves. *In the interactive notebook you can select the 2-loci instead:*

```
In [4]: interact(shallow_water.plot_hugoniot_loci, y_axis=widgets.fixed('hu'),
                plot_1=widgets.Checkbox(description='Plot 1-loci',value=True),
                plot_2=widgets.Checkbox(description='Plot 2-loci',value=False));
```



The plot above shows the Hugoniot loci in the h - hu plane, the natural phase plane in terms of the two conserved quantities. Of course, they all approach the origin as $h \rightarrow 0$. Alternatively, we can plot these same curves in the h - u plane:

```
In [5]: interact(shallow_water.plot_hugoniot_loci, y_axis=widgets.fixed('u'),
                plot_1=widgets.Checkbox(description='Plot 1-loci',value=True),
                plot_2=widgets.Checkbox(description='Plot 2-loci',value=False));
```



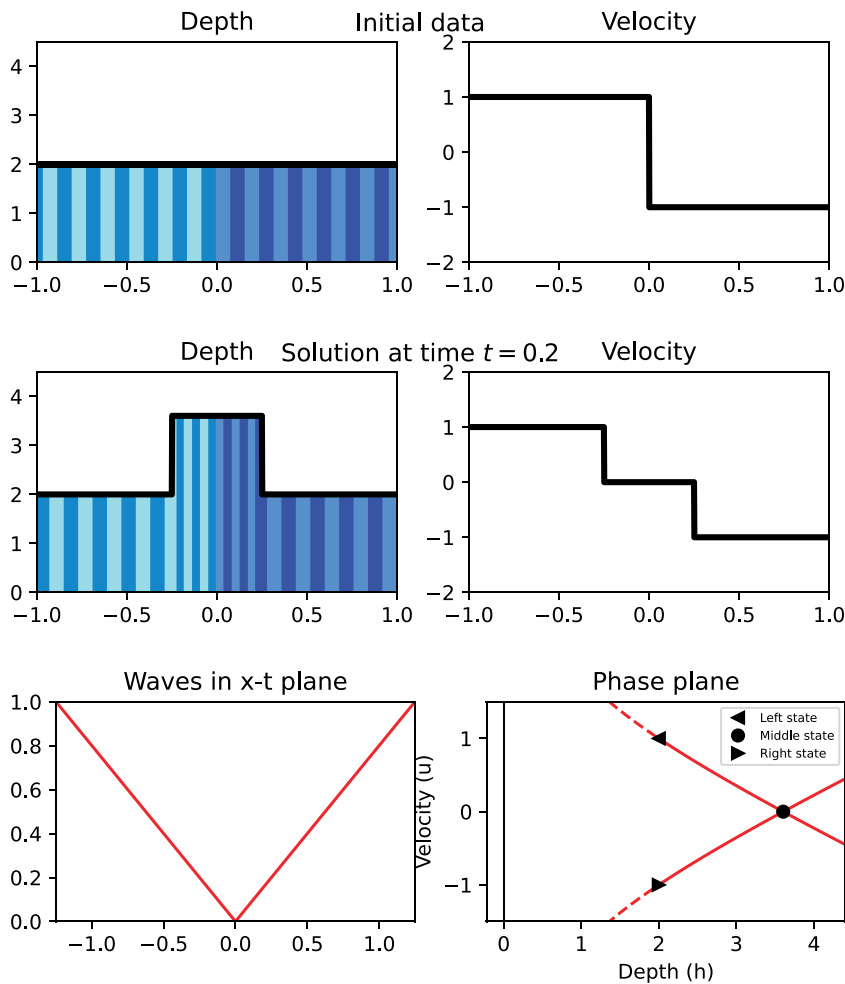
Note that in this plane the curves in each family are simply vertical translates of one another, and all curves asymptote to $\pm\infty$ as $h \rightarrow 0$. This means that it is impossible to have a shock with an adjacent dry ($h = 0$) state.

7.3.1 ■ All-shock Riemann solutions

If we knew a priori that both waves in the Riemann solution were shock waves, we could find q_m by finding the intersection of the 1-Hugoniot locus passing through q_ℓ with the 2-Hugoniot locus passing through q_r . The Riemann solution would then consist simply of two discontinuities propagating at the speeds given by (7.11).

Here is an example for which this is the correct solution: The initial depth is the same everywhere, $h_\ell = h_r$, but the velocity $u_\ell > 0$ while $u_r < 0$, so the flow is toward $x = 0$ from both sides. Note that in this plot the dark and light blue stripes show a passive tracer advected with the flow, which is shown only to help you visualize the fluid velocity in these plots.

In [6]: `shallow_water.plot_riemann_SW(h_l=2, h_r=2, u_l=1, u_r=-1)`

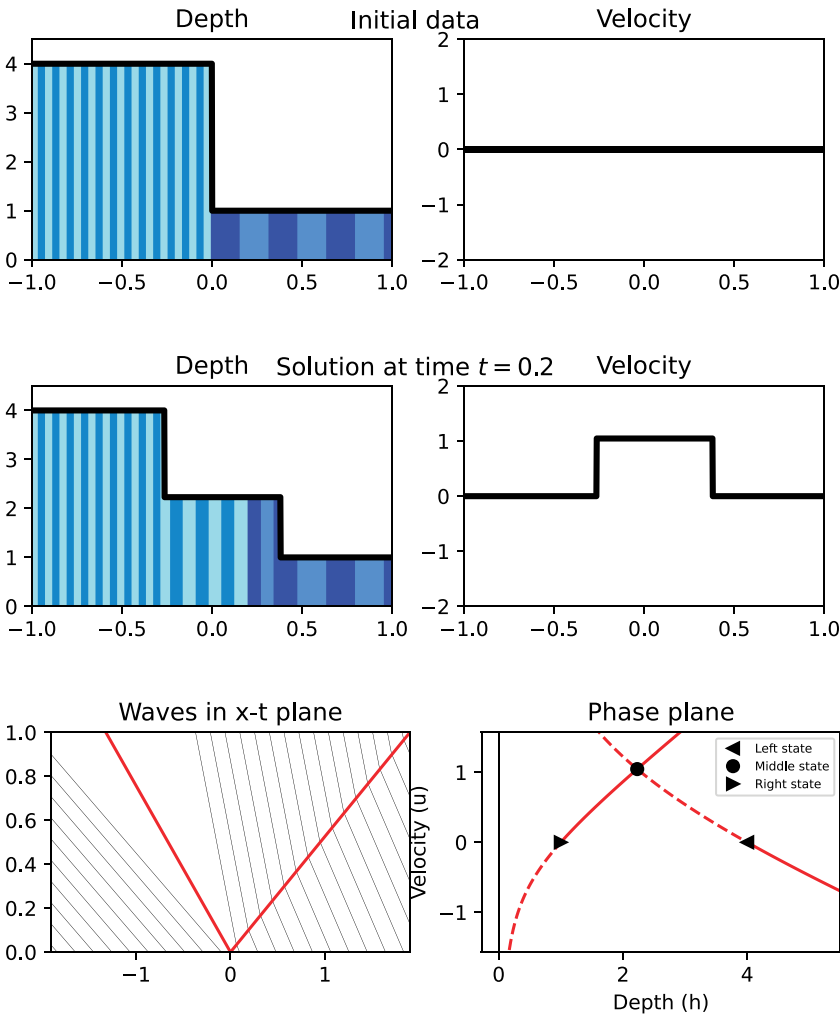


In the interactive notebook you can check the boxes to plot the characteristic families; notice that the 1-characteristics impinge on the 1-shock and the 2-characteristics impinge on the 2-shock. Thus these shocks satisfy the entropy condition. You can also check a box to show the particle paths, which show how the water is decelerated to 0 speed as it goes through each shock.

Dam break problem: All-shock solution

Here's another example, which you might think of as modeling a dam that suddenly breaks. The water is initially at rest on both sides, but much deeper on the left. We can again construct a 2-shock Riemann solution, but *this is not physically correct in this case!*

```
In [7]: shallow_water.plot_riemann_SW(h_l=4, h_r=1, u_l=0, u_r=0, plot1=True,
                                         force_waves='shock', particle_paths=False);
```



Notice that the 1-characteristics (which are plotted as thin lines) don't impinge on the 1-shock; instead, characteristics are diverging away from it. This shock does not satisfy the entropy condition and should be replaced with a rarefaction. The corresponding part of the Hugoniot locus is plotted with a dashed line to remind us that it is unphysical.

Now check the box in the interactive notebook to plot the 2-characteristics. Notice that these *do* impinge on the 2-shock; this is an entropy-satisfying shock, and the Hugoniot locus is plotted as a solid line.

7.3.2 ■ The entropy condition

For the 2-shock Riemann solution to be physically correct, each of the waves must satisfy the Lax entropy condition, with the characteristic speed to the left of the wave greater than that to the right. Notice that the shock speed (7.11) can be written as

$$s = u_* \pm \sqrt{gh \frac{h_* + h}{2h_*}}. \quad (7.12)$$

Thus the 1-shock entropy condition is

$$u_\ell - \sqrt{gh_\ell} > u_\ell - \sqrt{gh_\ell \frac{h_\ell + h_m}{2h_\ell}} = u_m - \sqrt{gh_m \frac{h_m + h_\ell}{2h_m}} > u_m - \sqrt{gh_m}.$$

The second expression for the shock speed is obtained by noticing that the shock speed is invariant under transposition of the left and right states. The first and second inequalities both simplify to

$$h_m > h_\ell. \quad (7.13)$$

Similarly, it can be shown that the entropy condition for a 2-shock connecting q_m and q_r is satisfied if and only if

$$h_m > h_r. \quad (7.14)$$

If (7.13) or (7.14) is violated, then the corresponding wave must be a rarefaction rather than a shock.

Go back and look at the phase plane plots above. You'll notice that the 1-Hugoniot locus passing through q_ℓ is plotted with a solid line for $h > h_\ell$ and with a dashed line for $h < h_\ell$ (and the same is done for the 2-Hugoniot locus passing through q_r). This is to remind us that states connected to q_ℓ by the dashed part are unphysical.

7.4 ■ Rarefaction waves

7.4.1 ■ Integral curves

In the LWR traffic flow model, we saw that a rarefaction wave consisted of a smoothly varying density. For the shallow water system, both the depth h and the momentum hu will vary smoothly through a rarefaction wave. A given rarefaction wave is associated with just one characteristic family, and the variation within the wave is at all points proportional to the corresponding eigenvector r_p . In other words, all values $\tilde{q} = (h, hu)$ in the rarefaction lie on a curve defined by

$$\tilde{q}'(\xi) = r_p(\tilde{q}(\xi)). \quad (7.15)$$

Here $\tilde{q}(\xi)$ is a parameterization of the solution. For the shallow water system, (7.15) reads as

$$h'(\xi) = q'_1(\xi) = 1, \quad (7.16)$$

$$(hu)'(\xi) = q'_2(\xi) = u \pm \sqrt{gh} = \tilde{q}_2/\tilde{q}_1 - \sqrt{g\tilde{q}_1}, \quad (7.17)$$

where we take the minus sign for 1-waves and the plus sign for 2-waves. We can think of (7.16)–(7.17) as an initial value ordinary differential equation (ODE); fixing the value of $\tilde{q}$ at one point in the rarefaction wave determines the whole solution of (7.16)–(7.17). We refer to each of these solutions as an *integral curve*.

For the shallow water system, these equations can be integrated exactly. If we fix one point (h_*, h_*u_*) on the curve, the whole integral curve is defined by

$$hu = hu_* \pm 2h(\sqrt{gh_*} - \sqrt{gh}), \quad (7.18)$$

where now the plus sign is for 1-waves and the minus sign is for 2-waves. This can be rewritten as

$$u \pm 2\sqrt{gh} = u_* \pm 2\sqrt{gh_*}. \quad (7.19)$$

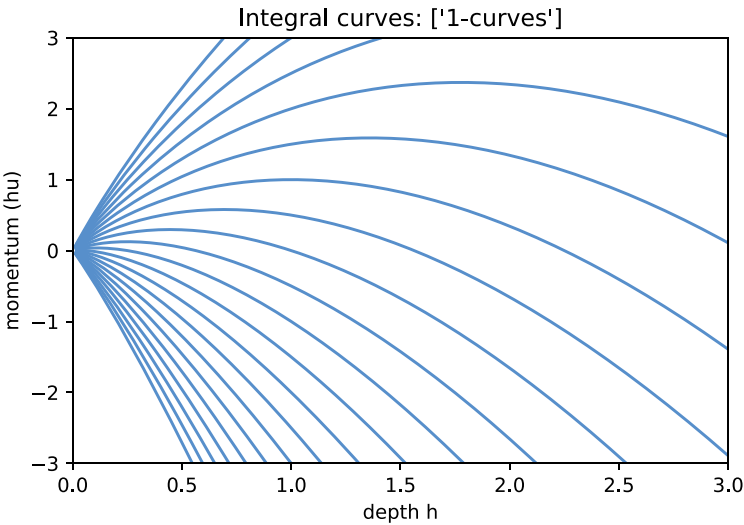
From (7.19) we see that the value $u + 2\sqrt{gh}$ is constant along any 1-integral curve, while the value $u - 2\sqrt{gh}$ is constant along any 2-integral curve. We refer to these quantities as *Riemann invariants* for the shallow water system:

$$w_1(q) = u + 2\sqrt{gh}, \quad (7.20)$$

$$w_2(q) = u - 2\sqrt{gh}. \quad (7.21)$$

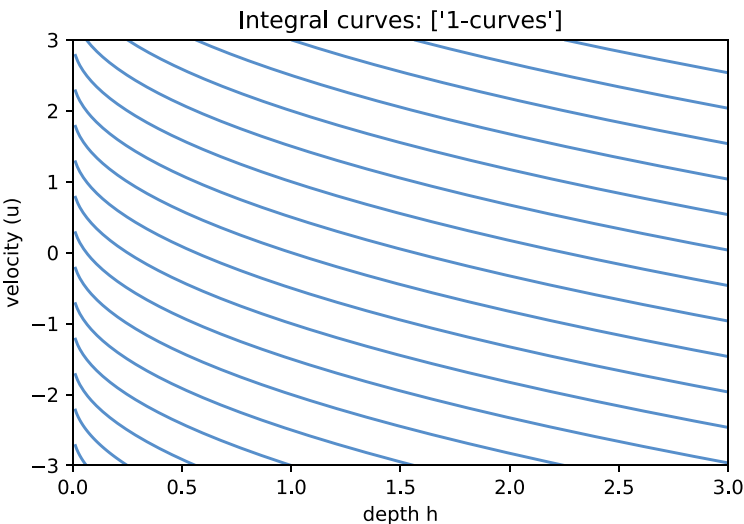
In other words, the trajectories plotted above are just the level curves of w_1 and w_2 . This allows us to easily plot the integral curves as a contour plot of w_1 or w_2 . In the notebook you can select which family of curves to display.

```
In [8]: interact(shallow_demos.plot_int_curves, y_axis=widgets.fixed('hu'),
                plot_1=widgets.Checkbox(description='1-wave curves',
                                         value=True),
                plot_2=widgets.Checkbox(description='2-wave curves',
                                         value=False));
```



We can also plot the integral curves in the h - u plane:

```
In [9]: interact(shallow_demos.plot_int_curves, y_axis=widgets.fixed('u'),
                plot_1=widgets.Checkbox(description='1-wave curves',
                                         value=True),
                plot_2=widgets.Checkbox(description='2-wave curves',
                                         value=False));
```



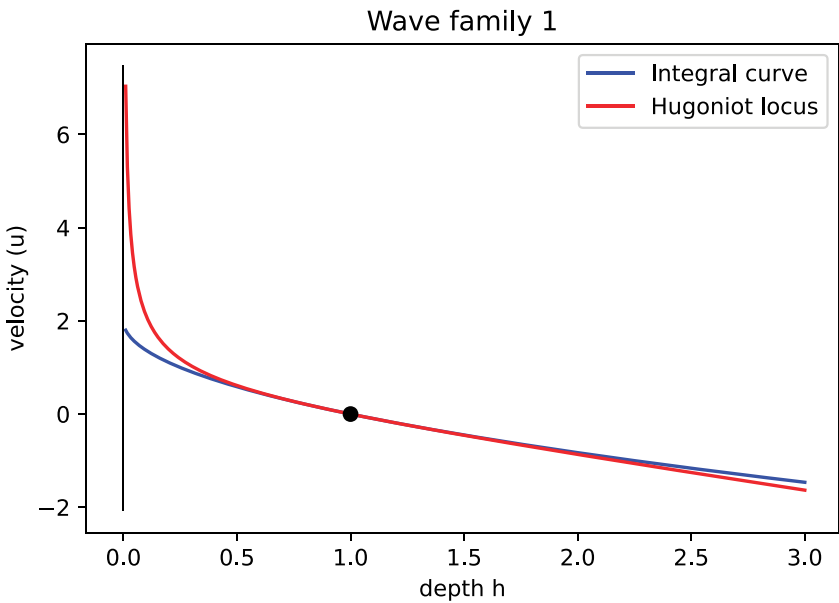
Note that in this plane the integral curves of each family are simply vertical translates of one another due to the form of the functions w_1 and w_2 . Unlike the Hugoniot loci, the integral curves do not asymptote to $\pm\infty$ as $h \rightarrow 0$ and instead each approaches a finite value.

7.4.2 ■ Comparison of integral curves and Hugoniot loci

You may have noticed that the integral curves look very similar to the Hugoniot loci, especially when plotted in the h - hu plane. Below we emphasize the differences by plotting the curve from each of these families that passes through a specified point. The difference is less noticeable in the h - hu plane since all of the curves approach the origin.

In the notebook you can select which family to display.

```
In [10]: interact(shallow_demos.compare_curves,
                  wave_family=RadioButtons(options=[1,2],
                                             description='Wave family:'),
                  y_axis=RadioButtons(options=['u','hu'],
                                           description='Vertical axis:'),
                  h0=FloatSlider(min=1.e-1,max=3.,value=1.,
                                  description='$h_0$'),
                  u0=FloatSlider(min=-3,max=3,description='$u_0$'));
```



Near the point of intersection, the curves are very close; indeed, they must be tangent at this point since their direction is parallel to the corresponding eigenvector there (and in fact they also have the same curvature). Far from this point they diverge; for small depths they must diverge greatly, since the Hugoniot locus never reaches $h = 0$ at any finite velocity.

7.4.3 ■ The structure of centered rarefaction waves

Rarefactions appearing in the Riemann solution are also similarity solutions; that is, they are constant along any ray $\xi = x/t$. For a p -rarefaction connecting states q_ℓ and q_r , the rarefaction thus has the form

$$q(x, t) = \begin{cases} q_\ell & \text{if } x/t \leq \lambda_p(q_\ell), \\ \tilde{q}(x/t) & \text{if } \lambda_p(q_\ell) \leq \lambda_p(q_r), \\ q_r & \text{if } x/t \geq \lambda_p(q_r). \end{cases} \quad (7.22)$$

It remains to determine how $\tilde{q}$ varies as a function of x/t . This can be done by noting that each solution value $\tilde{q}$ propagates at the characteristic speed $\lambda_p(\tilde{q})$. Letting $\xi = x/t$, this gives us the condition

$$\lambda_p(\tilde{q}(\xi)) = \xi. \quad (7.23)$$

By combining (7.23) with the Riemann invariant condition, we can find $\tilde{h}$ and $\tilde{u}$ within the rarefaction. For concreteness, let us consider a 1-rarefaction connecting q_ℓ and q_m . Then (7.23) reads $\tilde{u} - \sqrt{g\tilde{h}} = \xi$, which simplifies to

$$\tilde{h} = \frac{(\tilde{u} - \xi)^2}{g}. \quad (7.24)$$

Meanwhile, the fact that $w_1(\tilde{q})$ is constant means that

$$\tilde{u} + 2\sqrt{g\tilde{h}} = u_\ell + 2\sqrt{gh_\ell}. \quad (7.25)$$

Combining (7.24) with (7.25), we find

$$\tilde{h}(\xi) = \frac{(u_\ell + 2\sqrt{gh_\ell} - \xi)^2}{9g} = \frac{(w_1(q_\ell) - \xi)^2}{9g}, \quad (7.26)$$

$$\tilde{u}(\xi) = \frac{1}{3}w_1(q_\ell) + \frac{2}{3}\xi. \quad (7.27)$$

We see that h varies quadratically through the rarefaction, while u varies linearly.

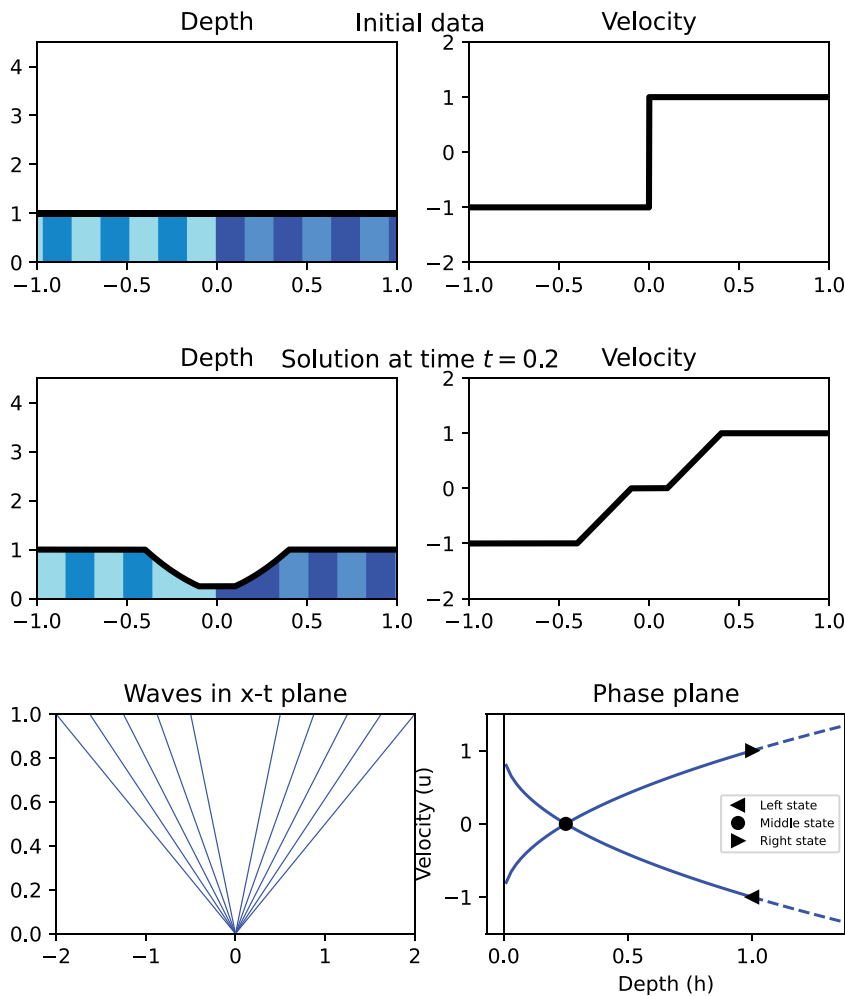
7.4.4 ■ Connecting states

Suppose we know that the middle state, q_m , is connected to the left and right states by rarefactions. Then we can find q_m by finding the intersection of the corresponding integral curves. Here is an example for which this is the physically correct solution:

$$q_\ell = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad q_r = \begin{bmatrix} 1 \\ 1 \end{bmatrix}. \quad (7.28)$$

In this case the water is flowing *away* from $x = 0$ on both sides.

In [11]: `shallow_water.plot_riemann_SW(h_l=1, h_r=1, u_l=-1., u_r=1.)`



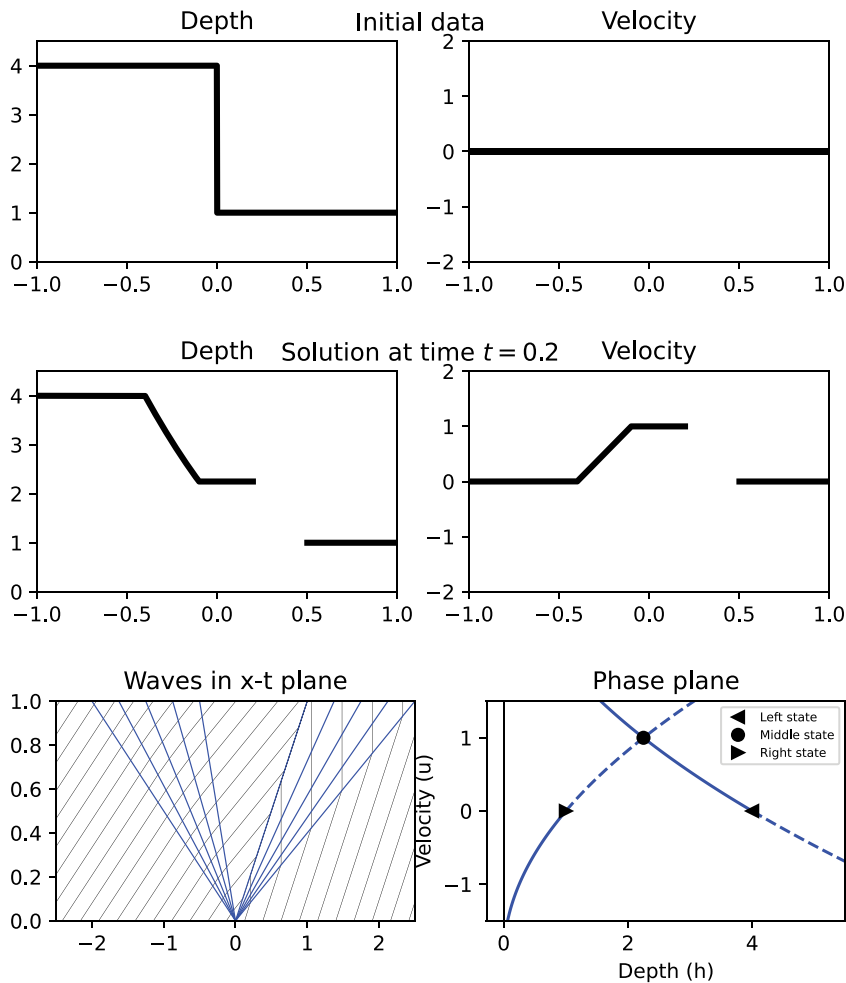
Notice that the segment of each integral curve that connects to states with a smaller depth is plotted as a solid line, while the segment connecting to states with greater depth is plotted with a dashed line. This is to remind us again that states connected by a rarefaction through the solid part are physical (entropy satisfying), while states connected by the dashed part are unphysical (entropy violating).

In the interactive notebook, try changing the velocities to $(1.9, -1.9)$, and notice that the Riemann solution has a state with almost zero depth in the middle. What happens if you use even larger outflow velocities? We'll discuss this later, in the section on dry states (Section 7.6).

7.4.5 ■ Dam break problem: All-rarefaction solution

Next let us revisit the problem from Section 7.3.1, where we found that the all-shock solution was incorrect. This time we'll try to find an all-rarefaction solution.

```
In [12]: shallow_water.plot_riemann_SW(h_l=4, h_r=1, u_l=0, u_r=0, plot2=True,
                                         force_waves='raref', particle_paths=False)
```



Notice that the 2-characteristics (plotted as thin lines) impinge on the 2-rarefaction; in fact they intersect at the left edge of the rarefaction. This means that the solution we constructed is triple valued and nonsensical as a solution to this one-dimensional conservation law, so this portion of the solution is omitted in the plots of depth and momentum. In this case, a rarefaction wave is not physical and should be replaced with a shock; the corresponding part of the integral curve is hence shown as a dashed line.

7.5 ■ The Riemann solution

To determine q_m , we need to know whether each of the two waves is a shock or a rarefaction, so that we can use the appropriate Hugoniot locus or integral curve. But to determine the nature of each wave, we need to check (7.13) or (7.14), which requires knowledge of h_m . To deal with this we define a piecewise function $\phi_\ell(h)$ that gives

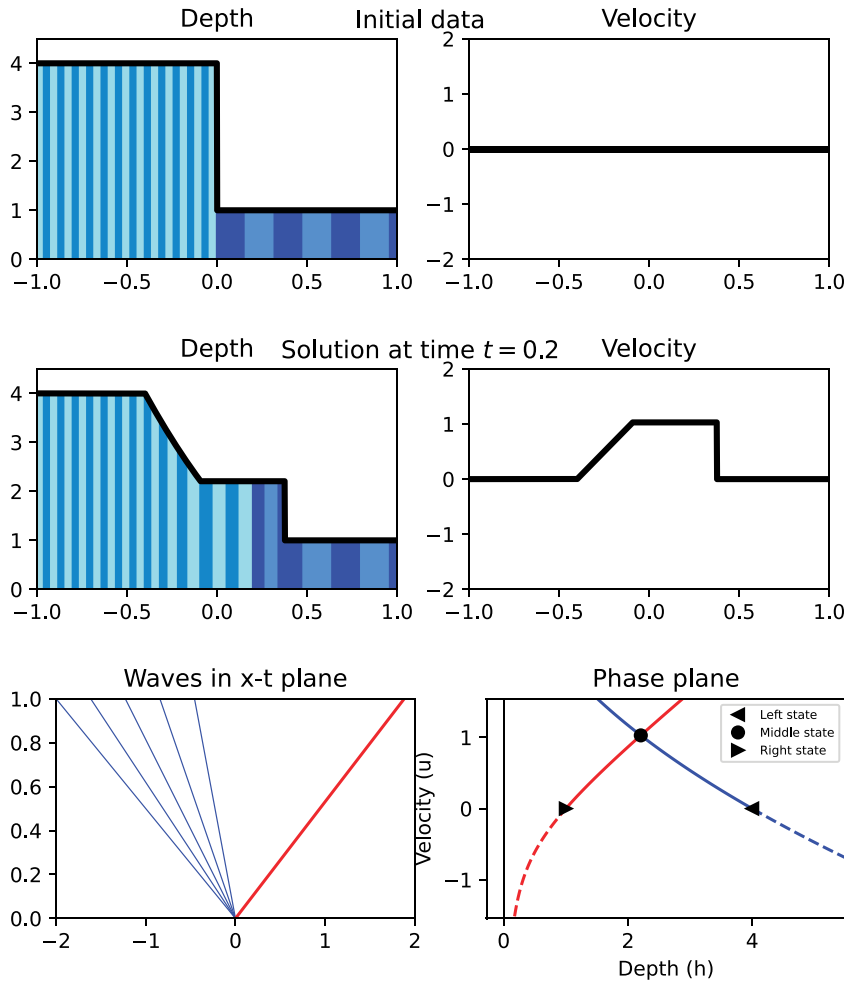
- for $h > h_\ell$, the momentum connected to q_ℓ by a 1-shock;
- for $h < h_\ell$, the momentum connected to q_ℓ by a 1-integral curve.

Thus ϕ is partly a 1-Hugoniot locus and partly a 1-integral curve; it gives the value of each precisely in the regime where it is physically correct. We can similarly define a function $\phi_r(h)$ related to the 2-Hugoniot locus and the 2-integral curve. The middle-state depth h_m is the value for which $\phi_\ell(h) = \phi_r(h)$. This value can be found computationally via a rootfinding algorithm.

7.5.1 ■ Dam break problem: Correct solution

Below we plot (at last) the correct solution to the dam break problem, involving a 1-rarefaction and a 2-shock. Plot the characteristics and notice how each family behaves correctly with respect to the corresponding wave in the Riemann solution.

```
In [13]: shallow_water.plot_riemann_SW(h_l=4, h_r=1, u_l=0, u_r=0)
```



In the interactive notebook you can confirm that 1-characteristics spread out across the rarefaction fan while 2-characteristics converge on the shock. View the particle paths, and

note that between the two waves the fluid velocity is constant, with the fluid accelerating across both the rarefaction and the shock to the same intermediate value u_m .

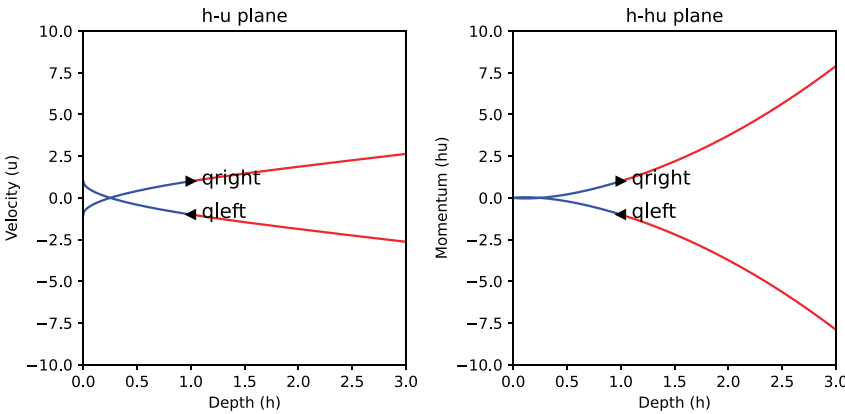
7.6 ■ Dry states

The approach just described will yield the correct Riemann solution (provided that the rootfinding algorithm converges) in most situations. However, it will fail if the solution involves dry ($h = 0$) states.

7.6.1 ■ Dry middle state

Sometimes it seems that there is no way to connect a given pair of left and right states via entropy-satisfying shocks or rarefactions. In the cell below we plot only the physically relevant parts of the integral curves and Hugoniot loci passing through the left and right states. Both the h - u and h - hu planes are shown. With the interactive widget you can try to adjust these states to find a combination for which none of the curves intersect.

```
In [14]: interact(shallow_demos.connect_states,
                  h_l=widgets.FloatSlider(min=0.001,max=2,value=1),
                  u_l=widgets.FloatSlider(min=-5,max=5,value=-1),
                  h_r=widgets.FloatSlider(min=0.001,max=2,value=1),
                  u_r=widgets.FloatSlider(min=-5,max=5,value=1));
```



You should find that by making the initial states flow sufficiently fast away from each other, there is no intersection in the h - u plane. In the h - hu plane, the integral curves always intersect at the origin. This ambiguity arises because for zero depth it isn't meaningful to assign a velocity. Thus in the h - u plane we could think of the entire u axis as being part of every integral curve. This means that we can always connect the left and right states via an intermediate state with depth $h = 0$ (a dry state).

Since the 1-integral curve through q_ℓ reaches $h = 0$ at $u = u_\ell + 2\sqrt{gh_\ell}$ and the 2-integral curve reaches $h = 0$ at $u = u_r - 2\sqrt{gh_r}$, we see that a dry middle state occurs if and only if $u_\ell + 2\sqrt{gh_\ell} < u_r - 2\sqrt{gh_r}$.

It is clear in this case that the solution must consist of two centered rarefaction waves, connecting the left and right states to the middle dry state. These centered

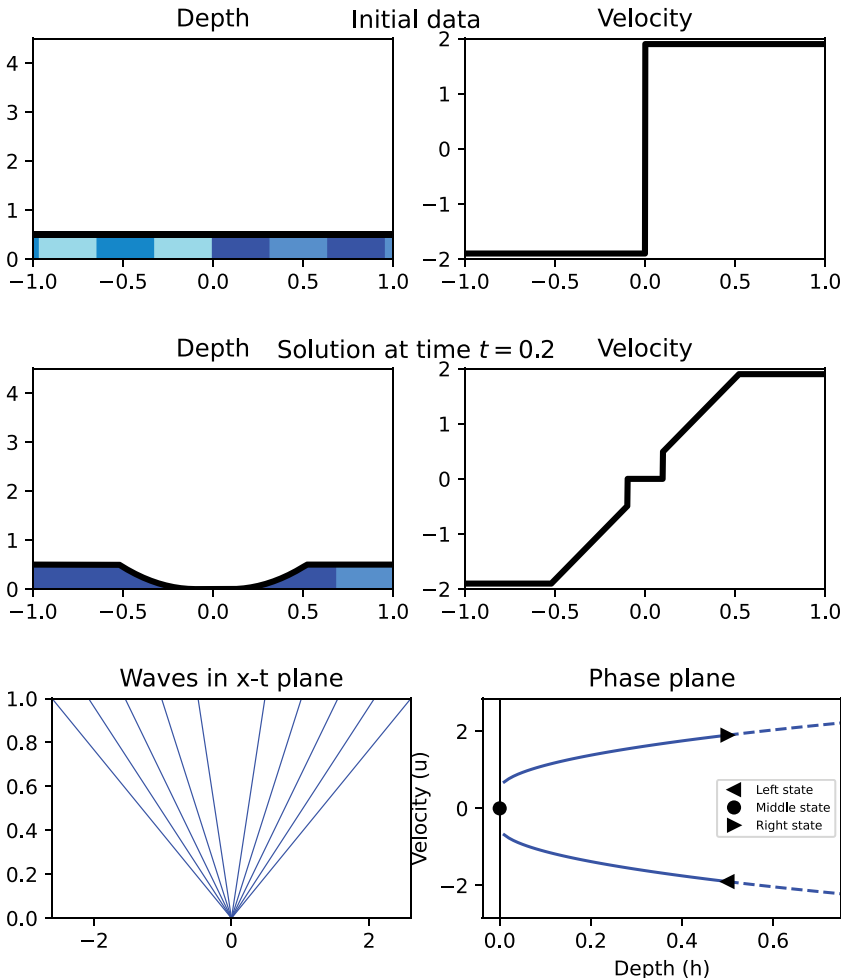
rarefactions still have the structure derived above; to complete the solution we only need to know the range of centered characteristics included in each rarefaction. This is most conveniently determined by considering the h - u plane. The 1-integral curve passing through q_ℓ is given by $u = u_\ell + 2(\sqrt{gh_\ell} - \sqrt{gh_r})$; it reaches a dry state $h = 0$ at $u = u_\ell + 2\sqrt{gh_\ell}$. Similarly, the 2-integral curve passing through q_r reaches a dry state $h = 0$ at $u = u_r - 2\sqrt{gh_r}$. Thus the dry-state solution is given by

$$q(x, t) = \begin{cases} q_\ell & \text{if } x/t \leq u_\ell - \sqrt{gh_\ell}, \\ \tilde{q}_\ell(x/t) & \text{if } u_\ell - \sqrt{gh_\ell} \leq x/t \leq u_\ell + 2\sqrt{gh_\ell}, \\ 0 & \text{if } u_\ell + 2\sqrt{gh_\ell} \leq x/t \leq u_r - 2\sqrt{gh_r}, \\ \tilde{q}_r(x/t) & \text{if } u_r - 2\sqrt{gh_r} \leq x/t \leq u_r + \sqrt{gh_r}, \\ q_r & \text{if } x/t \geq u_r + \sqrt{gh_r}, \end{cases} \quad (7.29)$$

where $\tilde{q}_\ell(x/t)$ is given by (7.26)–(7.27) and $\tilde{q}_r(x/t)$ is given by a similar expression.

Here's an example of a Riemann problem whose solution involves a dry middle state:

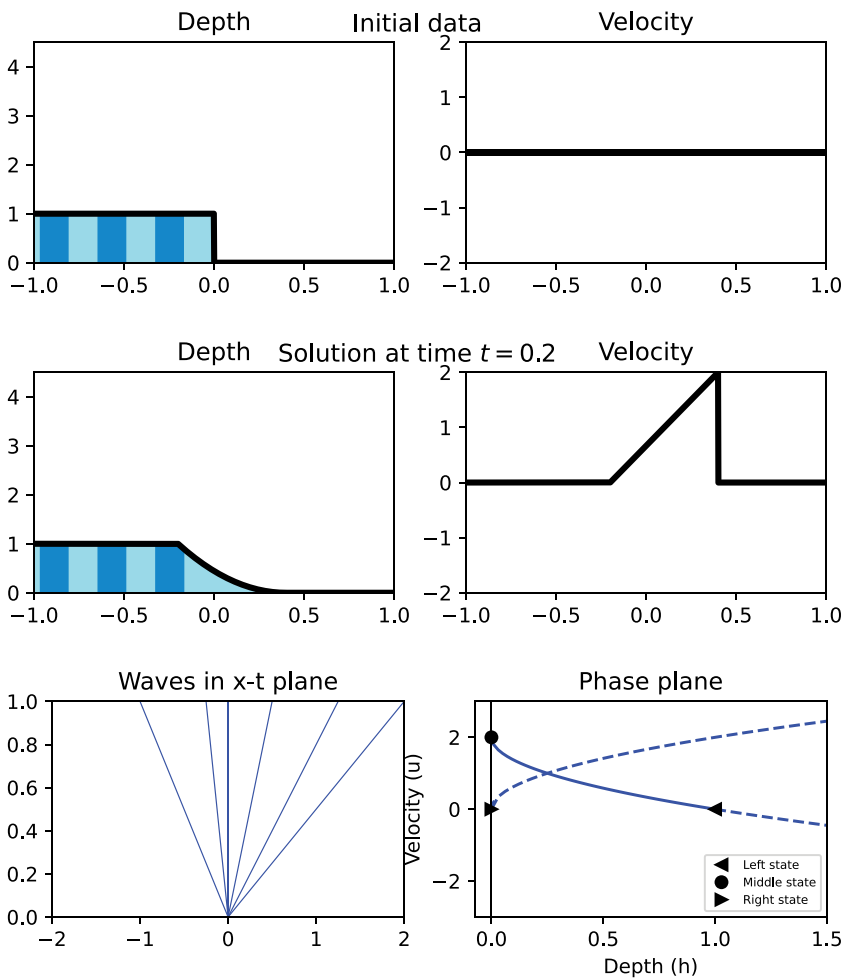
In [15]: `shallow_water.plot_riemann_SW(h_l=0.5, h_r=0.5, u_l=-1.9, u_r=1.9)`



7.6.2 ▪ Dry initial states

What if the left or right state in the Riemann problem is dry? In this case, the solution consists only of a rarefaction wave connecting the nondry state to $h = 0$.

```
In [16]: shallow_water.plot_riemann_SW(h_l=1, h_r=0, u_l=0, u_r=0,
                                         particle_paths=False)
```



7.7 ▪ Interactive phase plane

The live notebook, or this webpage,⁴⁰ provides an interactive view that allows you to see how the solution changes as you move the left and right states around in the phase plane. Be sure to click and drag the dark circles indicating the values of q_ℓ and q_r in the top left plot. The time can also be adjusted by dragging the circle in the top right plot.

The Hugoniot loci are plotted in red and the integral curves in blue. The portion of each curve that satisfies the entropy condition is plotted as a solid line, while

⁴⁰http://www.clawpack.org/riemann_book/phase_plane/shallow_water_small.html

the entropy-violating parts are plotted as dashed lines. The curves are plotted for both the left and right states. Thus the function ϕ_ℓ just described consists of the solid curve (blue and red parts) passing through the left state q_ℓ ; the function ϕ_r is the solid curve (blue and red parts) passing through q_r . The intersection of these curves is labeled q_m .

7.8 ■ Interactive solution

Finally, the notebook contains code that generates an interactive plot showing the solution to the Riemann problem for arbitrary left and right states, including solutions with dry states.

Chapter 8

Shallow water equations with a tracer

We can augment the shallow water equations discussed in Chapter 7 with a quantity $\phi(x, t)$ that measures the concentration of a tracer that is advected with the fluid motion (but has no influence on the fluid dynamics). If ϕ is measured in units of mass per unit volume (which is really per unit area of the vertical cross section in this one-dimensional example), then the mass per unit length along the x axis is given by $h(x, t)\phi(x, t)$. The quantity ϕ satisfies the variable-coefficient advection equation in advective (nonconservative) form:

$$\phi_t(x, t) + u(x, t)\phi_x(x, t) = 0.$$

This equation is also called the *color equation*, since we can think of ϕ as measuring the concentration of a dye that changes the color of the water. We will use this interpretation in the plots below. By setting the initial conditions $\phi(x, 0)$ to be piecewise constant with different values corresponding to different colors, we can visualize the motion of the fluid better. We will use a dark shade of blue for the water that is initially to the right of a dam at $x = 0$ and a lighter shade of blue for the water that is initially to the left.

The quantity $h\phi$ satisfies the conservative form of the advection equation,

$$(h\phi)_t + (uh\phi)_x = 0.$$

This can be derived by combining the color equation with the conservation of mass equation $h_t + (hu)_x = 0$. Since $h\phi$ measures the concentration of dye per unit length in x , and we assume molecules of dye are not created or destroyed, it makes sense that this is the conserved quantity.

The full system of equations in conservation form,

$$q_t(x, t) + f(q(x, t))_x = 0, \tag{8.1}$$

thus takes the form

$$\begin{aligned} h_t + (hu)_x &= 0, \\ (hu)_t + \left(hu^2 + \frac{1}{2}gh^2\right)_x &= 0, \\ (h\phi)_t + (uh\phi)_x &= 0. \end{aligned}$$

The Riemann solution for this system has three waves. Two of these waves, with speeds $u \pm c$ (where $c = \sqrt{gh}$), are just the same waves we studied in the shallow water system. We refer to these as the 1-wave and the 2-wave, to maintain consistency with Chapter 7. The additional wave appearing in this system (the *tracer wave*) moves at the fluid velocity u and carries only a jump in the tracer concentration ϕ . It is our first example of what is known as a contact discontinuity or *linearly degenerate* wave. Because the fluid velocity is constant across this wave, tracer characteristics travel parallel to the wave on either side. Thus this family does not admit shock or rarefaction waves. We will see another example of linear degeneracy in Chapter 9 and give a more general definition at that point.

The 1- and 2-waves can of course each be either a shock wave or a rarefaction wave depending on the initial data, as explained in Chapter 7, since the tracer has no effect on the structure of these waves. Below we consider a dam break problem, in which case there is one of each.

If you wish to examine the Python code for this chapter, see

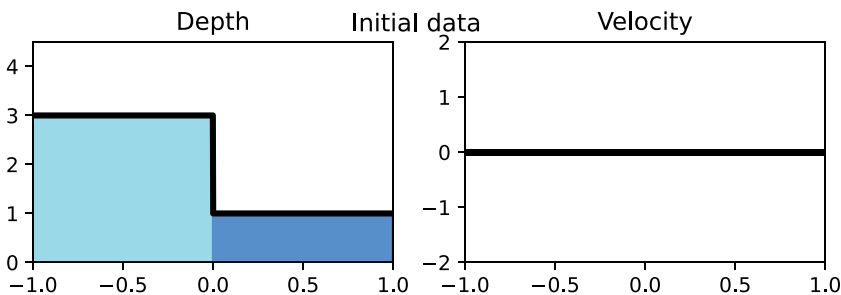
- `exact_solvers/shallow_water.py` ... on GitHub,⁴¹
- `exact_solvers/shallow_demos.py` ... on GitHub.⁴²

8.1 ■ Dam break problem

Here we plot the solution to the dam break Riemann problem for the shallow water equations with the addition of a tracer. The tracer is now also taken to be piecewise constant, with light blue representing the value ϕ_ℓ taken in the water the left of the dam and dark blue representing the value ϕ_r in the water on the right side of the dam.

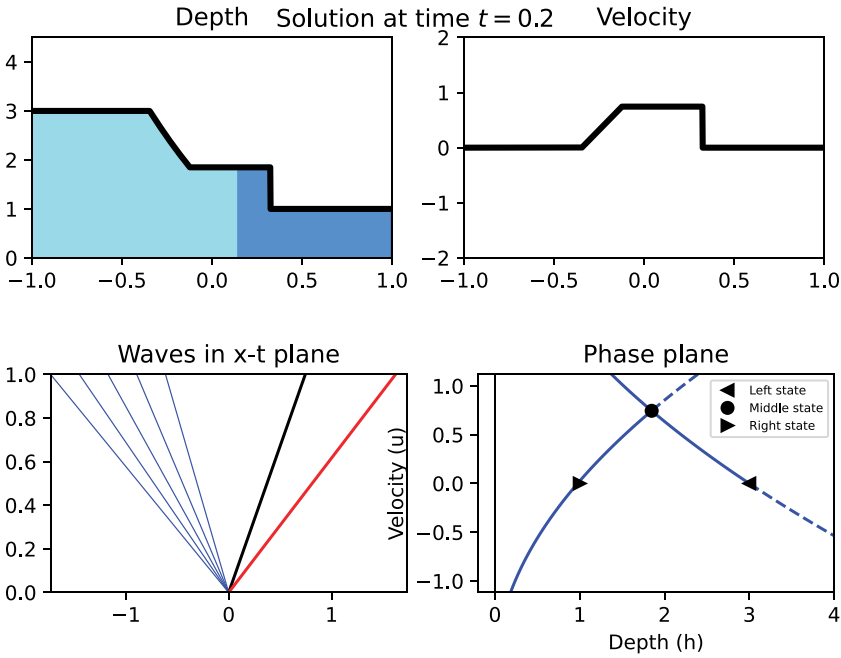
The structure of the depth and velocity is exactly the same as seen and discussed in detail in Chapter 7, and the value of ϕ is constant across the 1-rarefaction and 2-shock waves. The discontinuity in ϕ shows the propagation of the contact wave in the Riemann solution, the contact discontinuity across which ϕ is discontinuous while h and u are continuous.

In [3]: `shallow_water.plot_riemann_SW(h_l=3, h_r=1, u_l=0., u_r=0., tracer=True)`



⁴¹https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/shallow_water.py

⁴²https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/shallow_demos.py



In the live notebook you can select which set of characteristics to plot. Note that the characteristics corresponding to the tracer are equivalent to the particle paths of the water that were shown in similar examples in Chapter 7. This is because the characteristic speed is equal to u , the fluid velocity.

8.2 ▪ Hyperbolic structure

The canonical form (8.1) arises by defining

$$q = \begin{pmatrix} h \\ hu \\ h\phi \end{pmatrix}, \quad f = \begin{pmatrix} hu \\ hu^2 + \frac{1}{2}gh^2 \\ uh\phi \end{pmatrix}. \quad (8.2)$$

In terms of the conserved quantities, the flux is

$$f(q) = \begin{pmatrix} q_2 \\ q_2^2/q_1 + \frac{1}{2}gq_1^2 \\ q_3q_2/q_1 \end{pmatrix}. \quad (8.3)$$

Thus the flux Jacobian is

$$f'(q) = \begin{pmatrix} 0 & 1 & 0 \\ -(q_2/q_1)^2 + gq_1 & 2q_2/q_1 & 0 \\ -q_3q_2/q_1^2 & q_3/q_1 & q_2/q_1 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 \\ -u^2 + gh & 2u & 0 \\ -u\phi & \phi & u \end{pmatrix}. \quad (8.4)$$

Its eigenvalues are

$$\lambda_1 = u - \sqrt{gh}, \quad \lambda_{\text{tracer}} = u, \quad \lambda_2 = u + \sqrt{gh},$$

with corresponding eigenvectors

$$r_1 = \begin{bmatrix} 1 \\ u - \sqrt{gh} \\ \phi \end{bmatrix}, \quad r_{\text{tracer}} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad r_2 = \begin{bmatrix} 1 \\ u + \sqrt{gh} \\ \phi \end{bmatrix}.$$

We see immediately that the tracer wave carries only a jump in the tracer itself. It may seem puzzling that the third entry of r_1 and r_3 is nonzero. Notice that each of these waves carries a jump in h ; since ϕ is constant across each of these waves, there must be a corresponding jump in $q_3 = \phi h$.

8.3 ■ Linear degeneracy

The speed of the tracer wave depends only on u , and u is constant across the tracer wave. This means that tracer characteristics can neither converge nor diverge due to a jump in ϕ , but rather run parallel to it. Characteristic fields with this property are referred to as being **linearly degenerate** since the Riemann solution to the linear advection equation has this same property, that the characteristic speed is independent of changes in the variable that jumps across the wave.

If we compute the gradient of the eigenvalue λ_{tracer} by differentiating $u = hu/h$ with respect to each of the components of $q = (h, hu, h\phi)$, we obtain

$$\nabla \lambda_{\text{tracer}} = \begin{bmatrix} -hu/h^2 \\ 1/h \\ 0 \end{bmatrix}. \quad (8.5)$$

This vector is orthogonal to the corresponding eigenvector, i.e.,

$$\nabla \lambda_{\text{tracer}} \cdot r_{\text{tracer}} \equiv 0, \quad (8.6)$$

for all values of q . This is the general mathematical condition for a linearly degenerate field, since it shows that the eigenvalue is unchanged as we move along integral curves of the corresponding eigenvalue.

We will see another example of a linearly degenerate characteristic family in the next chapter, Chapter 9, where we will see that the Riemann solution to the Euler equations has a very similar structure to that of the shallow water equations with a tracer. In general it consists of two nonlinear acoustic waves that can be either shock or rarefaction waves, depending on the initial data, and an intermediate contact discontinuity that propagates with the intermediate fluid velocity, and across which the fluid velocity is constant.

Chapter 9

The Euler equations of gas dynamics

In this notebook, we discuss the equations and the structure of the exact solution to the Riemann problem. In Chapters 13 and 14, we will investigate approximate Riemann solvers.

9.1 ■ Fluid dynamics

In this chapter we study the system of hyperbolic PDEs that governs the motions of a compressible gas in the absence of viscosity. These consist of conservation laws for *mass*, *momentum*, and *energy*. Together, they are referred to as the *compressible Euler equations*, or simply the Euler equations. Our discussion here is fairly brief; for much more detail see [9] or [17].

9.1.1 ■ Mass conservation

We will use $\rho(x, t)$ to denote the fluid density and $u(x, t)$ for its velocity. Then the equation for conservation of mass is just the familiar *continuity equation*:

$$\rho_t + (\rho u)_x = 0.$$

9.1.2 ■ Momentum conservation

We discussed the conservation of momentum in a fluid in Chapter 3. For convenience, we review the ideas here. The momentum density is given by the product of mass density and velocity, ρu . The momentum flux has two components. First, the momentum is transported in the same way that the density is; this flux is given by the momentum density times the velocity: ρu^2 .

To understand the second term in the momentum flux, we must realize that a fluid is made up of many tiny molecules. The density and velocity we are modeling are average values over some small region of space. The individual molecules in that region are not all moving with the exact velocity u ; that's just their average. Each molecule also has some additional random velocity component. These random velocities are what accounts for the *pressure* of the fluid, which we'll denote

by p . These velocity components also lead to a net flux of momentum. Thus the momentum conservation equation is

$$(\rho u)_t + (\rho u^2 + p)_x = 0.$$

This is very similar to the conservation of momentum equation in the shallow water equations, as discussed in Chapter 7, in which case hu is the momentum density and $\frac{1}{2}gh^2$ is the hydrostatic pressure. For gas dynamics, a different expression must be used to compute the pressure p from the conserved quantities. This relation is called the *equation of state* of the gas, as further discussed below.

9.1.3 ■ Energy conservation

The energy has two components: internal energy density ρe and kinetic energy density $\rho u^2/2$:

$$E = \rho e + \frac{1}{2}\rho u^2.$$

Like the momentum flux, the energy flux involves both bulk transport (Eu) and transport due to pressure (pu):

$$E_t + (u(E + p))_x = 0.$$

9.1.4 ■ Equation of state

You may have noticed that we have four unknowns (density, momentum, energy, and pressure) but only three conservation laws. We need one more relation to close the system. That relation, known as the equation of state, expresses how the pressure is related to the other quantities. We'll focus on the case of a polytropic ideal gas, for which

$$p = \rho e(\gamma - 1).$$

Here γ is the ratio of specific heats, which for air is approximately 1.4.

9.2 ■ Hyperbolic structure of the one-dimensional Euler equations

We can write the three conservation laws as a single system $q_t + f(q)_x = 0$ by defining

$$q = \begin{pmatrix} \rho \\ \rho u \\ E \end{pmatrix}, \quad f(q) = \begin{pmatrix} \rho u \\ \rho u^2 + p \\ u(E + p) \end{pmatrix}. \quad (9.1)$$

These make up the one-dimensional Euler system. As usual, one can define the 3×3 Jacobian matrix by differentiating this flux function with respect to the three components of q .

In our discussion of the structure of these equations, it is convenient to work with the primitive variables (ρ, u, p) rather than the conserved variables. The quasilinear form is particularly simple in the primitive variables:

$$\begin{bmatrix} \rho \\ u \\ p \end{bmatrix}_t + \begin{bmatrix} u & \rho & 0 \\ 0 & u & 1/\rho \\ 0 & \gamma\rho & u \end{bmatrix} \begin{bmatrix} \rho \\ u \\ p \end{bmatrix}_x = 0. \quad (9.2)$$

9.2.1 ■ Characteristic velocities

The eigenvalues of the flux Jacobian $f'(q)$ for the one-dimensional Euler equations are

$$\lambda_1 = u - c, \quad \lambda_2 = u, \quad \lambda_3 = u + c. \quad (9.3)$$

Here c is the sound speed:

$$c = \sqrt{\frac{\gamma p}{\rho}}.$$

These are also the eigenvalues of the coefficient matrix appearing in (9.2) and show that acoustic waves propagate at speeds $\pm c$ relative to the fluid velocity u . There is also a characteristic speed $\lambda_2 = u$ corresponding to the transport of entropy at the fluid velocity, as further discussed below.

The eigenvectors of the coefficient matrix appearing in (9.2) are

$$r_1 = \begin{bmatrix} -\rho/c \\ 1 \\ -\rho c \end{bmatrix}, \quad r_2 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad r_3 = \begin{bmatrix} \rho/c \\ 1 \\ \rho c \end{bmatrix}. \quad (9.4)$$

These vectors show the relation between jumps in the primitive variables across waves in each family. The eigenvectors of the flux Jacobian $f'(q)$ arising from the conservative form (9.1) would be different and would give the relation between jumps in the conserved variables across each wave.

Notice that the second characteristic speed, λ_2 , depends only on u and that u does not change as we move in the direction of r_2 . In other words, the 2-characteristic velocity is constant on 2-integral curves. This is similar to the wave that carries changes in the tracer that we considered in Chapter 8. We say this characteristic field is *linearly degenerate*; it admits neither shocks nor rarefactions. In a simple 2-wave, all characteristics are parallel. A jump in this family carries a change only in the density and is referred to as a *contact discontinuity*.

Mathematically, the p th field is linearly degenerate if

$$\nabla \lambda_p(q) \cdot r_p(q) = 0, \quad (9.5)$$

since in this case the eigenvalue $\lambda_p(q)$ does not vary in the direction of the eigenvector $r_p(q)$ and hence is constant along integral curves of this family. (Recall that $r_p(q)$ is the tangent vector at each point on the integral curve.)

The other two fields have characteristic velocities that *do* vary along the corresponding integral curves. Moreover, they vary monotonically as we move along

an integral curve, always increasing as we move in one direction, and decreasing in the other. Mathematically, this means that

$$\nabla \lambda_p(q) \cdot r_p(q) \neq 0 \quad (9.6)$$

as q varies, so that the directional derivative of $\lambda_p(q)$ cannot change sign as we move along the curve. This is analogous to the flux for a scalar problem being convex and means that the 1-wave and the 3-wave in any Riemann solution to the Euler equations will be a single shock or rarefaction wave, not the sort of compound waves we observed in Chapter 6 in the nonconvex scalar case. Any characteristic field satisfying (9.6) is said to be *genuinely nonlinear*.

9.2.2 ■ Entropy

Another important quantity in gas dynamics is the *specific entropy*:

$$s = c_v \log(p/\rho^\gamma) + C,$$

where c_v and C are constants. From the expression (9.4) for the eigenvector r_2 , we see that the pressure and velocity are constant across a 2-wave. A simple 2-wave is also called an *entropy wave* because a variation in density while the pressure remains constant requires a variation in the entropy of the gas as well. On the other hand, a simple acoustic wave (a continuously varying pure 1-wave or 3-wave) has constant entropy throughout the wave; the specific entropy is a Riemann invariant for these families.

A shock wave (either a 1-wave or a 3-wave) satisfies the Rankine–Hugoniot condition and exhibits a jump in entropy. To be physically correct, the entropy of the gas must *increase* as gas molecules pass through the shock, leading to the *entropy condition* for selecting shock waves. We have already seen this term used in the context of scalar nonlinear equations for shallow water flow, even though the entropy condition in those cases did not involve the physical entropy.

9.2.3 ■ Riemann invariants

Since the Euler equations have three components, we expect each integral curve (a one-dimensional set in three-dimensional space) to be defined by two Riemann invariants. These are

$$1: s, u + \frac{2c}{\gamma - 1}; \quad (9.7)$$

$$2: u, p; \quad (9.8)$$

$$3: s, u - \frac{2c}{\gamma - 1}. \quad (9.9)$$

9.2.4 ■ Integral curves

The level sets of these Riemann invariants are two-dimensional surfaces; the intersection of two appropriate level sets defines an integral curve.

The 2-integral curves, of course, are simply lines of constant pressure and velocity (with varying density). Since the field is linearly degenerate, these coincide

with the Hugoniot loci. We can determine the form of the 1- and 3-integral curves using the Riemann invariants above. For a curve passing through (ρ_0, u_0, p_0) , we find

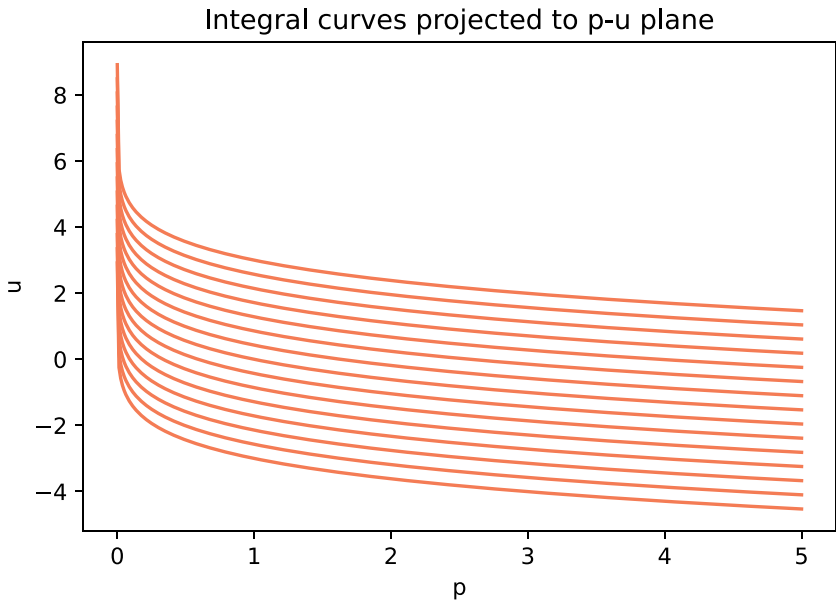
$$\rho(p) = (p/p_0)^{1/\gamma} \rho_0, \tag{9.10}$$

$$u(p) = u_0 \pm \frac{2c_0}{\gamma - 1} \left(1 - (p/p_0)^{(\gamma-1)/(2\gamma)} \right). \tag{9.11}$$

Here the plus sign is for 1-waves and the minus sign is for 3-waves. Below we plot the projection of some integral curves on the p - u plane. If you wish to examine the Python code for this chapter, see

- `exact_solvers/euler.py` ... on GitHub,⁴³
- `exact_solvers/euler_demos.py` ... on GitHub.⁴⁴

```
In [3]: interact(euler.plot_integral_curves,
                gamma=widgets.FloatSlider(min=1.1,max=3,value=1.4),
                rho_0=widgets.FloatSlider(min=0.1,max=3.,value=1.,
                description=r'$\rho_0$'));
```



9.3 ■ Rankine–Hugoniot jump conditions

The Hugoniot loci for 1- and 3-shocks are

$$\rho(p) = \left(\frac{1 + \beta p/p_0}{p/p_\ell + \beta} \right), \tag{9.12}$$

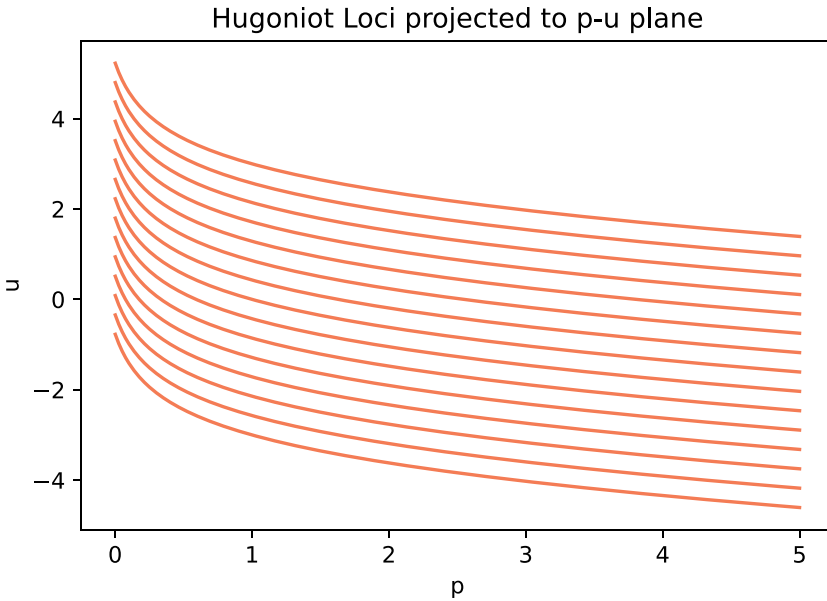
⁴³https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/euler.py
⁴⁴https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/euler_demos.py

$$u(p) = u_0 \pm \frac{2c_0}{\sqrt{2\gamma(\gamma-1)}} \left(\frac{1 - p/p_0}{\sqrt{1 + \beta p/p_0}} \right), \quad (9.13)$$

where $\beta = (\gamma + 1)/(\gamma - 1)$. Here the plus sign is for 1-shocks and the minus sign is for 3-shocks.

Below we plot the projection of some Hugoniot loci on the p - u plane.

```
In [4]: interact(euler.plot_hugoniot_loci,
                gamma=widgets.FloatSlider(min=1.1,max=3,value=1.4),
                rho_0=widgets.FloatSlider(min=0.1,max=3.,value=1.,
                description=r'$\rho_0$'));
```



9.3.1 ■ Entropy condition

As mentioned above, a shock wave is physically relevant only if the entropy of the gas increases as the gas particles move through the shock. A discontinuity satisfying the Rankine–Hugoniot jump conditions that violates this entropy condition (an *entropy-violating shock*) is not physically correct and should be replaced by a rarefaction wave in the Riemann solution.

This physical entropy condition is equivalent to the mathematical condition that for a 1-shock to be physically relevant, the 1-characteristics must impinge on the shock (the Lax entropy condition). If the entropy condition is violated, the 1-characteristics will spread out, allowing the insertion of an expansion fan (rarefaction wave).

9.4 ■ Exact solution of the Riemann problem

The general Riemann solution is found following the steps listed below. This is essentially the same procedure used to determine the correct solution to the Riemann problem for the shallow water equations in Chapter 7, where more details are given.

The Euler equations are a system of three equations, and the general Riemann solution consists of three waves, so we must determine two intermediate states rather than the one intermediate state in the shallow water equations. However, it is nearly as simple because we know that the pressure and velocity are constant across the 2-wave, so there is a single intermediate pressure p_m and velocity u_m in both intermediate states, and it is only the density that takes different values ρ_{m1} and ρ_{m2} . Moreover, any jump in density is allowed across the 2-wave, and we have expressions given above for how $u(p)$ varies along any integral curve or Hugoniot locus, expressions that do not explicitly involve ρ . So we can determine the intermediate p_m by finding the intersection point of two relevant curves in step 3 of this general algorithm:

1. Define a piecewise function giving the middle-state velocity u_m that can be connected to the left state by an entropy-satisfying shock or rarefaction as a function of the middle-state pressure p_m .
2. Define a piecewise function giving the middle-state velocity u_m that can be connected to the right state by an entropy-satisfying shock or rarefaction as a function of the middle-state pressure p_m .
3. Use a nonlinear rootfinder to find the intersection of the two functions defined above.
4. Use the Riemann invariants to find the intermediate-state densities and the solution structure inside any rarefaction waves.

Step 4 above requires finding the structure of rarefaction waves. This can be done using the fact that the Riemann invariants are constant through the rarefaction wave. See Chapter 14 of [9] for more details.

9.5 ■ Examples of Riemann solutions

Here we present some representative examples of Riemann problems and solutions. The examples chosen are closely related to the examples used in Chapter 7, so you might want to refer back to that chapter and compare the results.

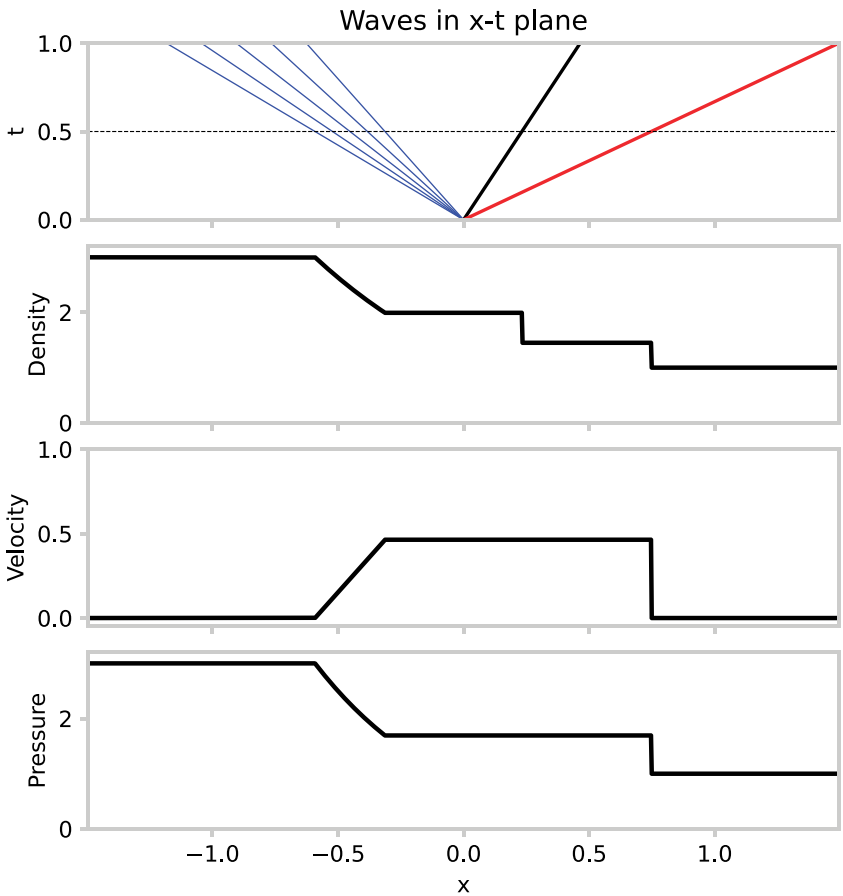
9.5.1 ■ Problem 1: Sod shock tube

First we consider the classic shock tube problem. The initial condition consists of high density and pressure on the left, low density and pressure on the right, and zero velocity on both sides. The solution is composed of a shock propagating to the right (3-shock) while a left-going rarefaction forms (1-rarefaction). In between these two waves, there is a jump in the density, which is the contact discontinuity (2-wave) in the linearly degenerate characteristic field.

Note that this set of initial conditions is analogous to the dam break problem for shallow water equations, and the resulting structure of the solution is very similar to that obtained when those equations are solved with the addition of a scalar tracer. However, in the Euler equations the entropy jump across a 2-wave does affect the fluid dynamics on either side, so this is not a passive tracer and solving the Riemann problem is slightly more complex.

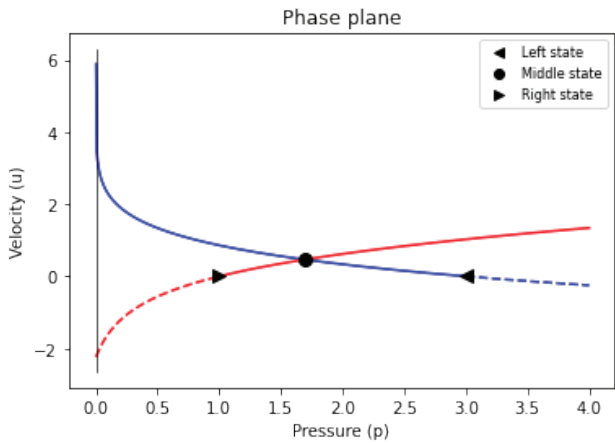
```
In [5]: left_state = State(Density = 3.,
                          Velocity = 0.,
                          Pressure = 3.)
       right_state = State(Density = 1.,
                          Velocity = 0.,
                          Pressure = 1.)

       euler.riemann_solution(left_state, right_state)
```



Here is a plot of the solution in the phase plane, showing the integral curve connecting the left and middle states and the Hugoniot locus connecting the middle and right states.

In [6]: `euler.phase_plane_plot(left_state, right_state)`

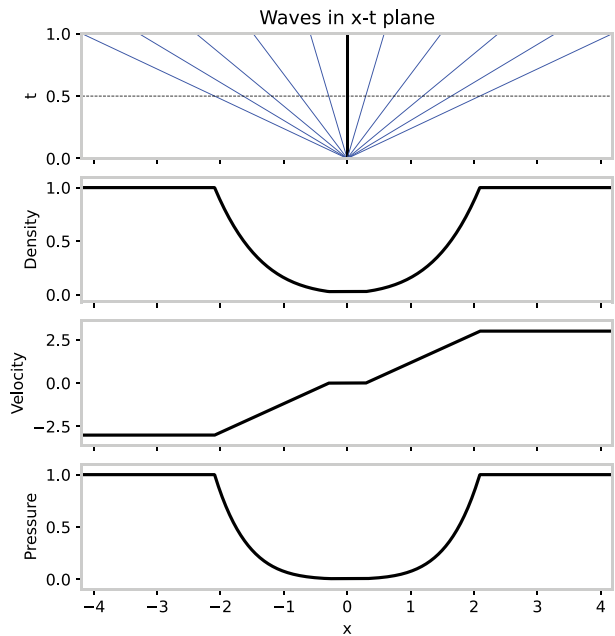


9.5.2 ▪ Problem 2: Symmetric expansion

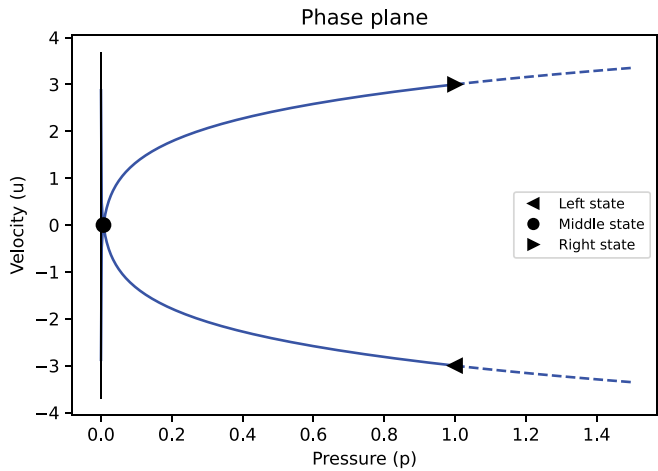
Next we consider the case of equal densities and pressures and equal and opposite velocities, with the initial states moving away from each other. The result is two rarefaction waves (the contact has zero strength).

```
In [7]: left_state = State(Density = 1.,
                           Velocity = -3.,
                           Pressure = 1.)
right_state = State(Density = 1.,
                   Velocity = 3.,
                   Pressure = 1.)

euler.riemann_solution(left_state,right_state);
```



In [8]: `euler.phase_plane_plot(left_state, right_state)`

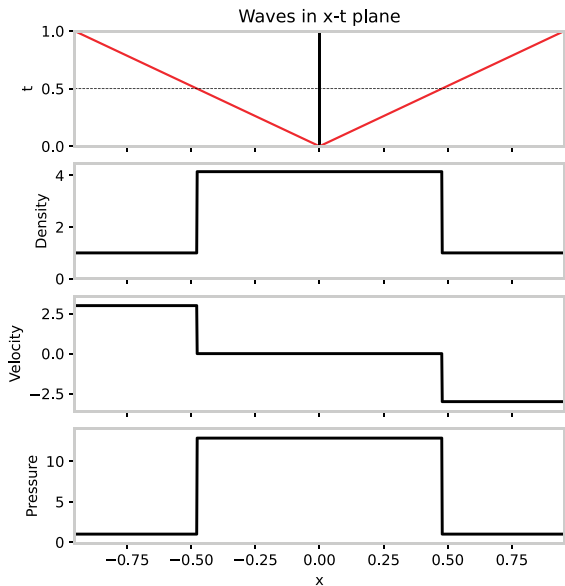


9.5.3 ■ Problem 3: Colliding flows

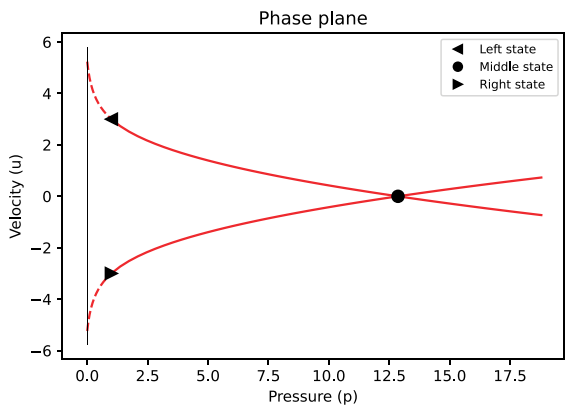
Next, consider the case in which the left and right states are moving toward each other. This leads to a pair of shocks, with a high-density, high-pressure state in between.

```
In [9]: left_state = State(Density = 1.,
                           Velocity = 3.,
                           Pressure = 1.)
right_state = State(Density = 1.,
                   Velocity = -3.,
                   Pressure = 1.)

euler.riemann_solution(left_state, right_state)
```



```
In [10]: euler.phase_plane_plot(left_state, right_state)
```

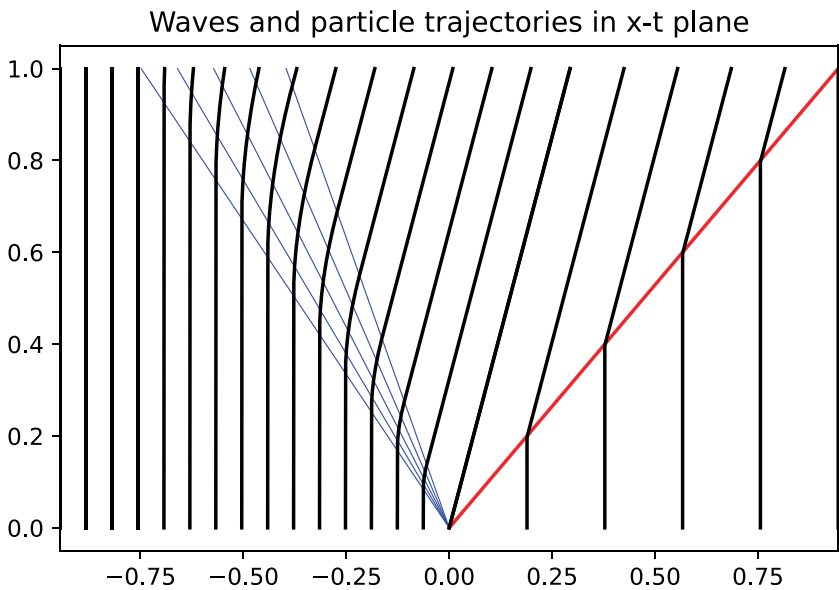


9.6 ▪ Plot particle trajectories

In the next plot of the Riemann solution in the x - t plane, we also plot the trajectories of a set of particles initially distributed along the x axis at $t = 0$, with the spacing inversely proportional to the density. The evolution of the distance between particles gives an indication of how the density changes.

```
In [11]: left_state = State(Density = 3.,
                             Velocity = 0.,
                             Pressure = 3.)
        right_state = State(Density = 1.,
                             Velocity = 0.,
                             Pressure = 1.)

        euler.plot_riemann_trajectories(left_state, right_state)
```



Since the distance between particles in the above plot is inversely proportional to the density, we see that the density around a particle increases as it goes through the shock wave but decreases through the rarefaction wave and that in general there is a jump in density across the contact discontinuity, which lies along the particle trajectory emanating from $x = 0$ at $t = 0$.

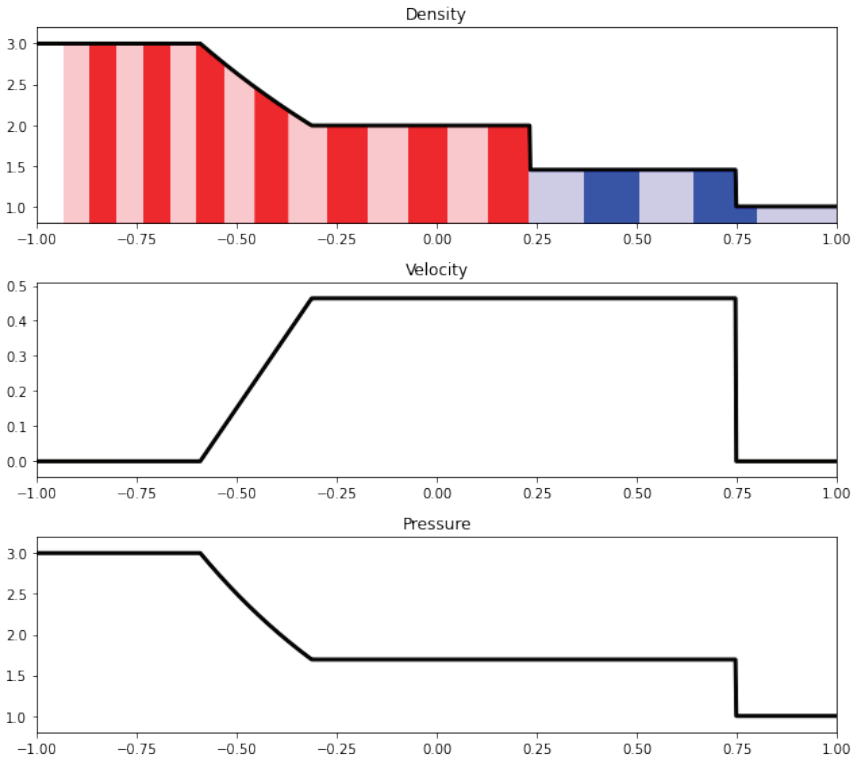
9.7 ■ Riemann solution with a colored tracer

Next we plot the Riemann solution with the density plot also showing an advected color to help visualize the flow better. The fluid initially to the left of $x = 0$ is colored red and that initially to the right of $x = 0$ is colored blue, with stripes of different shades of these colors to help visualize the motion of the fluid.

Let's plot the Sod shock tube data with this colored tracer:

```
In [12]: def plot_with_stripes_t_slider(t):
          euler_demos.plot_with_stripes(rho_l=3.,u_l=0.,p_l=3.,
                                         rho_r=1.,u_r=0.,p_r=1.,
                                         gamma=gamma,t=t)

          interact(plot_with_stripes_t_slider,
                   t=widgets.FloatSlider(min=0.,max=1.,step=0.1,value=0.5));
```



Note the following in the figure above:

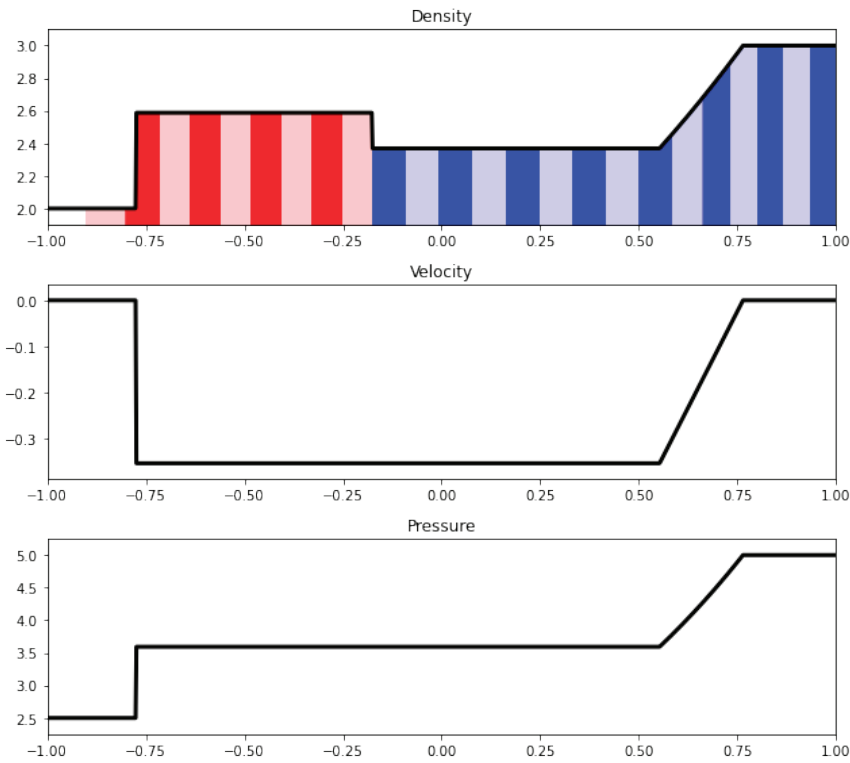
- The edges of each stripe are being advected with the fluid velocity, so you can visualize how the fluid is moving.

- The width of each stripe is initially inversely proportional to the density of the fluid, so that the total mass of gas within each stripe is the same.
- The total mass within each stripe remains constant as the flow evolves, and the width of each stripe remains inversely proportional to the local density.
- The interface between the red and the blue gas moves with the contact discontinuity. The velocity and pressure are constant, but the density can vary across this wave.

9.8 ▀ Interactive Riemann solver

The initial configuration specified below gives a rather different looking solution than when using initial conditions of Sod, but with the same mathematical structure. *In the live notebook, you can easily adjust the initial data and immediately see the resulting solution.*

```
In [13]: euler_demos.euler_demo1(rho_l=2.,u_l=0.,p_l=2.5,
                                   rho_r=3.,u_r=0.,p_r=5., gamma=gamma)
```



9.9 ▀ Riemann problems with vacuum

A vacuum state (with zero pressure and density) in the Euler equations is similar to a dry state (with depth $h = 0$) in the shallow water equations. It can arise in the solution of the Riemann problem in two ways:

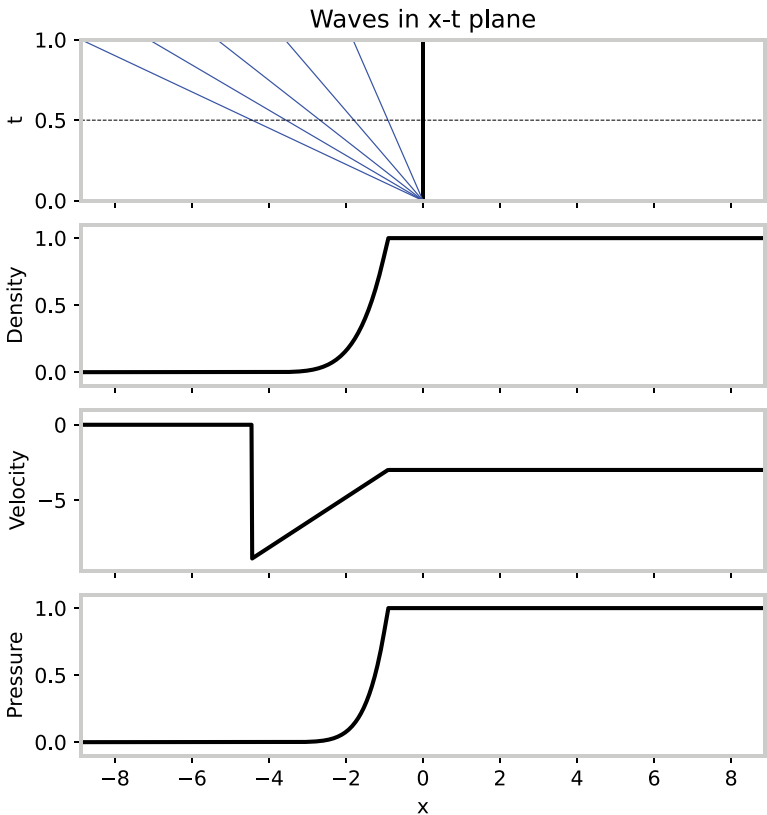
1. An initial left or right vacuum state: in this case the Riemann solution consists of a single rarefaction, connecting the nonvacuum state to the vacuum state.
2. A problem where the left and right states are not in vacuum but the middle states are in vacuum: since this means the middle pressure is smaller than that to the left or right, this can occur only if the 1- and 3-waves are both rarefactions. These rarefactions are precisely those required to connect the left and right states to the middle vacuum state.

9.9.1 ■ Initial vacuum state

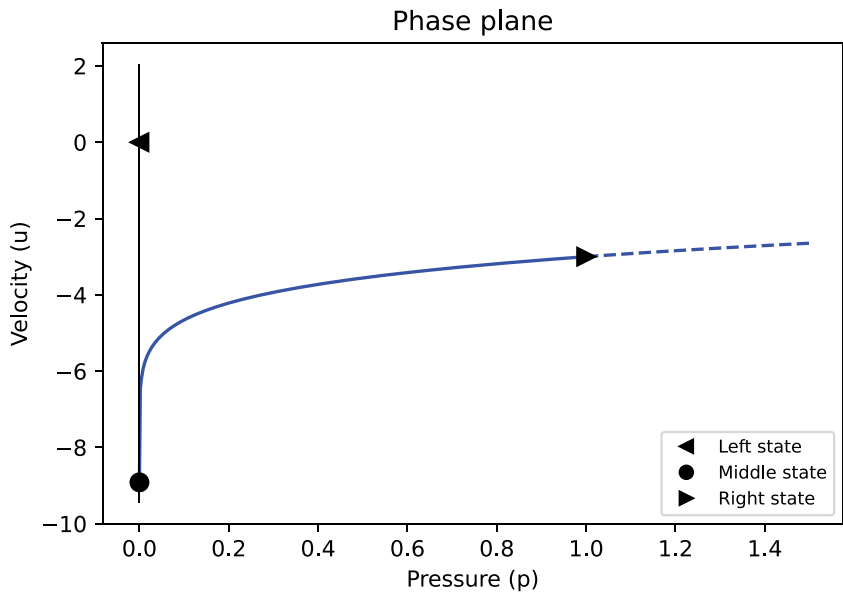
Next we start with the density and pressure set to 0 in the left state. The velocity plot looks a bit strange, but note that the velocity is undefined in vacuum. The solution structure consists of a rarefaction wave, similar to what is observed in the analogous case of a dam break problem with dry land on one side (depth = 0), as discussed in Chapter 7.

```
In [14]: left_state = State(Density = 0.,
                             Velocity = 0.,
                             Pressure = 0.)
        right_state = State(Density = 1.,
                             Velocity = -3.,
                             Pressure = 1.)

        euler.riemann_solution(left_state, right_state)
```



```
In [15]: euler.phase_plane_plot(left_state, right_state)
```



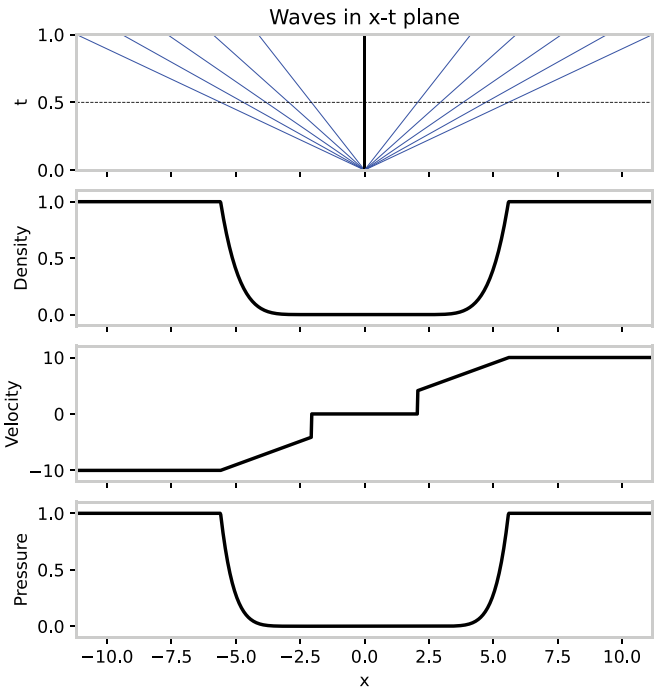
The phase plane plot may look odd, but recall that in the vacuum state velocity is undefined, and since $p_\ell = p_m = 0$, the left and middle states are actually the same.

9.9.2 ▪ Middle vacuum state

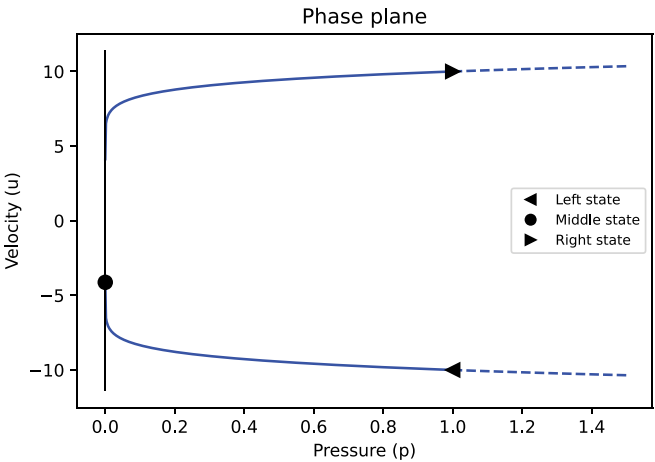
Finally, we consider an example where there is sufficiently strong outflow ($u_\ell < 0$ and $u_r > 0$) for a vacuum state to form, analogous to the dry state that appears in the similar example in Chapter 7.

```
In [16]: left_state = State(Density = 1.,
                             Velocity = -10.,
                             Pressure = 1.)
right_state = State(Density = 1.,
                    Velocity = 10.,
                    Pressure = 1.)

euler.riemann_solution(left_state, right_state)
```



```
In [17]: euler.phase_plane_plot(left_state, right_state)
```



Again the phase plane plot may look odd, but since the velocity is undefined in the vacuum state, the middle state with $p_m = 0$ actually connects to both integral curves, which correspond to the two outgoing rarefaction waves.

Part II

Approximate Riemann solvers

Chapter 10

Introduction to approximate Riemann solvers

In Part II of this book we present a number of *approximate Riemann solvers*. We have already seen that for many important hyperbolic systems it is possible to work out the exact Riemann solution for arbitrary left and right states. However, for complicated nonlinear systems, such as the Euler equations (see Chapter 9), this exact solution can only be determined by solving a nonlinear system of algebraic equations for the intermediate states and the waves that connect them. This can be done to arbitrary precision, but only at some computational expense. The cost of exactly solving a single Riemann problem may seem insignificant, but it can become prohibitively expensive when the Riemann solver is used as a building block in a finite volume method. In this case a Riemann problem must be solved at every cell edge at every time step.

For example, if we consider a very coarse grid in one space dimension with only 100 cells and take 100 time steps, then 10,000 Riemann problems must be solved. In solving practical problems in two or three space dimensions it is not unusual to require the solution of billions or trillions of Riemann problems. In this context it can be very important to develop efficient approximate Riemann solvers that quickly produce a sufficiently good approximation to the true Riemann solution.

The following points have helped to guide the development of approximate Riemann solvers:

- If the solution is smooth over much of the domain, then the jump in states between neighboring cells will be very small (on the order of Δx , the cell size) for most of the Riemann problems encountered in the numerical solution. Even if the hyperbolic system being studied is nonlinear, for such data the equations can be approximated by a linearization and we have seen that linear Riemann problems can be solved more easily than nonlinear ones. Rather than solving a nonlinear system of equations by some iterative method, one need only solve a linear system (provided the eigenvalues and eigenvectors of the Jacobian matrix are known analytically, as they often are for practical problems). In many cases the solution of this linear system can also be worked out analytically and is easy to implement, so numerical linear algebra is not required.

- In spite of smoothness over much of the domain, in interesting problems there are often isolated discontinuities such as shock waves that are important to model accurately. So some Riemann problems arising in a finite volume method may have large jumps between the left and right states. Hence a robust approximate Riemann solver must also handle these cases without introducing too much error.
- But even in the case of large jumps in the data, it may not be necessary or worthwhile to solve the Riemann problem exactly. The information produced by the Riemann solver goes into a numerical method that updates the approximate solution in each grid cell, and the exact structure of the Riemann solution is lost in the process.

Each chapter in this part of the book illustrates some common approximate Riemann solvers in the context of one of the nonlinear systems studied in Part I. We focus on two popular approaches to devising approximate Riemann solvers, though these are certainly not the only approaches: linearized solvers and two-wave solvers.

10.1 ■ Finite volume methods

We give a short review of Riemann-based finite volume methods to illustrate what is typically needed from a Riemann solver in order to implement such methods. In one space dimension, a finite volume approximation to the solution $q(x, t_n)$ at the n th time step consists of discrete values Q_j^n , each of which can be viewed as approximating the cell average of the solution over a grid cell $x_{j-1/2} < x < x_{j+1/2}$ for some discrete grid. The cell length $\Delta x_j = x_{j+1/2} - x_{j-1/2}$ is often uniform, but this is not required. Many methods for hyperbolic conservation laws are written in *conservation form*, in which the numerical solution is advanced from time t_n to $t_{n+1} = t_n + \Delta t_n$ by the explicit formula

$$Q_j^{n+1} = Q_j^n - \frac{\Delta t_n}{\Delta x_j} (F_{j+1/2}^n - F_{j-1/2}^n) \quad (10.1)$$

for some definition of the *numerical flux* $F_{j-1/2}^n$, typically based on Q_{j-1}^n and Q_j^n (and possibly other nearby cell values at time t_n).

Dividing (10.1) by Δt_n and rearranging, this form can be viewed as a discretization of $q_t + f(q)_x = 0$, provided the numerical flux is *consistent* with the true flux $f(q)$ in a suitable manner. In particular, if the Q_i used in defining $F_{j-1/2}^n$ are all equal to the same value $\bar{q}$, then $F_{j-1/2}^n$ should reduce to $f(\bar{q})$.

A big advantage of using conservation form is that the numerical method is conservative. The sum $\sum \Delta x_j Q_j^n$ approximates the integral $\int q(x, t_n) dx$. Multiplying (10.1) by Δx_j and summing shows that at time t_{n+1} this sum only changes due to fluxes at the boundaries of the region in question (due to cancellation of the flux differences when summing), a property shared with the true solution. For problems with shock waves, using methods in conservation form is particularly important since nonconservative formulations can lead to methods that converge to discontinuous solutions that look fine but are not correct; e.g., the shock wave might propagate at entirely the wrong speed.

10.1.1 ■ Godunov's method

Trying to compute numerical approximations to hyperbolic problems with strong shock waves is challenging because of the discontinuities in the solution—classical interpretations of $(F_{j+1/2}^n - F_{j-1/2}^n)/\Delta x_j \approx \partial f/\partial x$ break down, oscillations near discontinuities often appear, and methods can easily go catastrophically unstable.

The landmark paper of Godunov [3] was the first to suggest using the solution to Riemann problems in defining the numerical flux: $F_{j-1/2}^n$ is obtained by evaluating $f(Q_{j-1/2}^*)$, where $Q_{j-1/2}^*$ is the Riemann solution evaluated along the ray $x/t = 0$ after the standard Riemann problem is solved between states $q_\ell = Q_{j-1}^n$ and $q_r = Q_j^n$ (with the discontinuity placed at $x = t = 0$ as usual in the definition of the Riemann problem). If the numerical solution is now defined as a piecewise-constant function with value Q_j^n in the j th cell at time t_n , then the exact solution takes this value along the cell interface $x_{j-1/2}$ for sufficiently small later times $t > t_n$ (until waves from other cell interfaces begin to interact).

The classic Godunov method was developed for gas dynamics, and the exact Riemann solution was used, but since only one value is used from this solution and the rest of the structure is thrown away, it is natural to use some *approximate Riemann solver* that more cheaply estimates $Q_{j-1/2}^*$. The approximations discussed in the next few chapters are often suitable.

Godunov's method turns out to be very robust—because the shock structure of the solution is used in defining the interface flux, the method generally remains stable provided that the *Courant–Friedrichs–Lewy condition* is satisfied, which restricts the allowable time step relative to the cell sizes and wave speeds by requiring that no wave can pass through more than one grid cell in a single time step. This is clearly a necessary condition for convergence of the method, based on domain of dependence arguments, and for Godunov's method (with the exact solver) this is generally sufficient as well, as verified in countless simulations (though seemingly impossible to prove in complete generality for nonlinear systems of equations). When the exact solution is replaced by an approximate solution, the method may not work as well, and so some care has to be used in defining a suitable approximation.

10.1.2 ■ High-resolution methods

In spite of its robustness, Godunov's method is seldom used as just described because the built-in *numerical viscosity* that gives it robustness also leads to very smeared out solutions, particularly around discontinuities, unless a very fine computational grid is used. For the advection equation, Godunov's method reduces to the standard *first-order upwind* method, and in general it is only first-order accurate even on smooth solutions.

A wide variety of higher-order Godunov-type (i.e., Riemann solver-based) methods have been developed. One approach first reconstructs better approximations to the solution at each time from the states Q_j^n , e.g., a piecewise polynomial that is linear or quadratic in each grid cell rather than constant, and then uses the states from these polynomials evaluated at the cell interfaces to define the Riemann problems. Another approach, used in the “wave propagation methods”

developed in [9], for example, is to take the waves that come out of the Riemann solution based on the original data to also define second-order correction terms. In either case some *limiters* must generally be applied to avoid nonphysical oscillations in solutions, particularly when the true solution has discontinuities. There is a vast literature on such methods; see, e.g., many of the books cited in the preface. For our present purposes the main point is that an approximate Riemann solver is a necessary ingredient in many methods that are commonly used to obtain high-resolution approximations to hyperbolic PDEs.

10.2 ■ Notation and structure of approximate solutions

We consider a single Riemann problem with left state q_ℓ and right state q_r . These states might be Q_{j-1}'' and Q_j'' for a typical interface when using Godunov's method, or other states defined from them, e.g., after doing a polynomial reconstruction. At any rate, from now on we will not discuss the numerical methods or the grid in its totality but simply focus on how to define an approximate Riemann solution based on an arbitrary pair of states. The resulting *interface solution* and *interface flux* will be denoted simply by Q^* and F^* , respectively, as approximations to the Riemann solution and flux along $x/t = 0$ in the similarity solution.

The Riemann solution resolves the jump $\Delta q = (q_r - q_\ell)$ into a set of propagating waves. In both of the approaches described below, the approximate Riemann solution consists entirely of traveling discontinuities; i.e., there are no rarefaction waves in the approximate solution, although there may be a discontinuity that approximates such a wave. One should rightly worry about whether the approximate solution generated with such a method will satisfy the required entropy condition and end up with rarefaction waves where needed rather than entropy-violating shocks, and we address this to some extent in the examples in the following chapters. It is important to remember that we are discussing the approximate solver that will be used *at every grid interface* in every time step, and the numerical viscosity inherent in the numerical method can lead to rarefactions in the overall numerical approximation even if each approximate Riemann solution lacks rarefactions. Nonetheless, some care is needed, particularly in the case of *transonic* rarefactions, as we will see.

Following [9], we refer to these traveling discontinuities as *waves* and denote them by $\mathcal{W}_p \in \mathbb{R}^m$, where the index p denotes the characteristic family and typically ranges from 1 to m for a system of m equations, although in an approximate solver the number of waves may be smaller (or possibly larger). At any rate, they always have the property that

$$q_r - q_\ell = \sum_p \mathcal{W}_p. \quad (10.2)$$

For each wave, the approximate solver must also give a wave speed $s_p \in \mathbb{R}$. For a linear system such as acoustics, the true solution has this form, with the s_p being eigenvalues of the coefficient matrix and each wave a corresponding eigenvector, as described in Chapter 3. One class of approximate Riemann solvers discussed below is based on approximating a nonlinear problem by a linearization locally at each interface.

Once a set of waves and speeds has been defined, we can define an interface flux as follows: the waves for which $s_p < 0$ are traveling to the left, while those with $s_p > 0$ are traveling to the right, and so as an interface flux we could use $f(Q^*)$, where Q^* is defined by either

$$Q^* = q_\ell + \sum_{p:s_p < 0} \mathcal{W}_p$$

or

$$Q^* = q_r - \sum_{p:s_p > 0} \mathcal{W}_p.$$

These two expressions give the same value for Q^* unless there is a wave with $s_p = 0$, in which case they could be different if the corresponding $\mathcal{W}_p$ is nonzero. However, if we are using the *exact* Riemann solution (e.g., for a linear system or a nonlinear problem in which the solution consists only of shock waves), then a stationary discontinuity with $s_p = 0$ must have no jump in flux across it (by the Rankine–Hugoniot condition), so even if the two values of Q^* differ, the flux $f(Q^*)$ is uniquely defined. For an approximate Riemann solution, this might not be true.

Another way to define the interface flux F^* is as

$$F^* = f(q_\ell) + \sum_{p:s_p \leq 0} s_p \mathcal{W}_p = f(q_r) - \sum_{p:s_p \geq 0} s_p \mathcal{W}_p, \quad (10.3)$$

suggested by the fact that this is the correct flux along $x/t = 0$ for a linear system $f(q) = Aq$ or for a nonlinear system with only shocks in the solution; in these cases the Rankine–Hugoniot condition implies that $s_p \mathcal{W}_p$ is equal to the jump in flux across each wave. Note that in this expression the terms in the sum for $s_p = 0$ drop out, so the two expressions always agree.

The wave propagation algorithms described in [9] and implemented in Clawpack use a form of Godunov’s method based on the sums appearing in (10.3), called *fluctuations*, to update the neighboring cell averages, rather than the flux difference form. In these methods the waves and speeds are further used (after applying a limiter to the waves) to obtain the high-resolution corrections. An advantage of working with fluctuations, waves, and speeds rather than interface fluxes is that these quantities often make sense also for *nonconservative hyperbolic systems*, such as the variable-coefficient linear problem $q_t + A(x)q_x = 0$, for which there is no “flux function.” A Riemann problem is defined by prescribing matrices A_ℓ and A_r along with the initial data q_ℓ and q_r , e.g., by using the material properties in the grid cells to the left and right of the interface for acoustics through a heterogeneous material. The waves are then naturally defined using the eigenvectors of A_ℓ corresponding to negative eigenvalues for the left-going waves and using eigenvectors of A_r corresponding to positive eigenvalues for the right-going waves. See [9] for more details.

Updating cell averages by fluctuations rather than flux differencing will give a conservative method (when applied to a hyperbolic problem in conservation form) only if the waves and speeds in the approximate solver satisfy

$$\sum_{p=1}^m s_p \mathcal{W}_p = f(q_r) - f(q_\ell). \quad (10.4)$$

This is a natural condition to require of our approximate Riemann solvers in general, even though the flux-differencing form (10.1) always leads to a conservative method, since this is satisfied by the exact solution in cases where it consists only of discontinuities and each wave satisfies the Rankine–Hugoniot condition. When (10.4) is satisfied we say the approximate solver is conservative.

10.3 ■ Linearized Riemann solvers

Consider a nonlinear system $q_t + f(q)_x = 0$. If q_ℓ and q_r are close to each other, as is often the case over smooth regions of a more general solution, then the nonlinear system can be approximated by a linear problem of the form $q_t + \hat{A}q_x = 0$. The coefficient matrix $\hat{A}$ should be some approximation to $f'(q_\ell) \approx f'(q_r)$ in the case where $\|q_\ell - q_r\|$ is small. The idea of a general linearized Riemann solver is to define a matrix $\hat{A}(q_\ell, q_r)$ that has this property but also makes sense as an approximation in the case when $\|q_\ell - q_r\|$ is not small. For many nonlinear systems there is a *Roe linearization*, a particular function that works very well based on ideas introduced originally by Roe [15]. For systems such as the shallow water equations or the Euler equations, there are closed-form expressions for the eigenvalues and eigenvectors of $\hat{A}$ and the solution of the linearized Riemann problem, leading to efficient solvers. These will be presented in the next few chapters.

10.4 ■ Two-wave solvers

Since the Riemann solution impacts the overall numerical solution based only on how it modifies the two neighboring solution values, it seems reasonable to consider approximations in which only a single wave propagates in each direction. The solution will have a single intermediate state q_m such that $\mathcal{W}_1 = q_m - q_\ell$ and $\mathcal{W}_2 = q_r - q_m$. There are apparently $m + 2$ values to be determined: the middle state $q_m \in \mathbb{R}^m$ and the speeds s_1, s_2 . In order for the approximate solver to be conservative, it must satisfy (10.4), and hence we have the m conditions

$$f(q_r) - f(q_\ell) = s_1 \mathcal{W}_1 + s_2 \mathcal{W}_2. \quad (10.5)$$

This can be solved for the middle state to find

$$q_m = \frac{f(q_r) - f(q_\ell) - s_2 q_r + s_1 q_\ell}{s_1 - s_2}. \quad (10.6)$$

It remains only to specify the wave speeds, and it is in this specification that the various two-wave solvers differ. In the following sections we briefly discuss the choice of wave speed for a scalar problem; the choice for systems will be elaborated on in subsequent chapters.

Typically $s_1 < 0 < s_2$, and so the intermediate state we need is $Q^* = q_m$ and $F^* = f(Q^*)$. However, in some cases both s_1 and s_2 could have the same sign, in which case F^* is either $f(q_\ell)$ or $f(q_r)$.

This class of solvers is also an ingredient in the so-called *central schemes* [5]. Due to the extreme simplicity of two-wave solvers, the resulting central schemes are often even referred to as being *Riemann-solver-free*.

10.4.1 ■ Lax–Friedrichs and local Lax–Friedrichs

The simplest such solver is the *Lax–Friedrichs (LF) method*, in which it is assumed that both waves have the same speed, in opposite directions:

$$-s_1 = s_2 = a,$$

where $a \geq 0$. Then (10.6) becomes

$$q_m = -\frac{f(q_r) - f(q_\ell)}{2a} + \frac{q_r + q_\ell}{2}.$$

In the original LF method, the wave speed a is taken to be the same in every Riemann problem over the entire grid; in the *local Lax–Friedrichs (LLF) method*, a different speed a may be chosen for each Riemann problem.

For stability reasons, the wave speed should be chosen at least as large as the fastest wave speed appearing in the true Riemann solution. However, choosing a wave speed that is too large leads to excess diffusion. For the LLF method (originally due to Rusanov) [9], the wave speed is chosen as

$$a(q_r, q_\ell) = \max(|f'(q)|),$$

where the maximum is taken over all values of q between q_r and q_ℓ . This ensures stability but may still introduce substantial damping of slower waves.

10.4.2 ■ Harten–Lax–van Leer

A less dissipative solver can be obtained by allowing the left- and right-going waves to have different speeds. This approach was developed in [4]. The solution is then determined by (10.6). In the original Harten–Lax–van Leer (HLL) solver, it was suggested to again use speeds that bound the possible speeds occurring in the true solution. For a scalar problem, this translates to

$$s_1 = \min(f'(q)),$$

$$s_2 = \max(f'(q)),$$

where again the minimum and maximum are taken over all values between q_r and q_ℓ . Many refinements of this choice have been proposed in the context of systems of equations, some of which will be discussed in later chapters.

Chapter 11

An approximate solver for Burgers' equation

As a first example, we return to the inviscid Burgers' equation that we studied in Chapter 4:

$$q_t + \left(\frac{1}{2} q^2 \right)_x = 0. \quad (11.1)$$

Although it is easy to solve the Riemann problem for (11.1) exactly, it is nevertheless interesting to consider approximate solvers because a numerical scheme does not make use of the full Riemann solution. Furthermore, Burgers' equation provides a simple setting in which some of the intricacies of more complicated approximate solvers can be introduced. Recall that we are interested in **approximate solutions that consist entirely of traveling discontinuities**.

Recall from Chapter 10 that we use Q^* to denote the state along $x/t = 0$ in the approximate Riemann solution, and F^* for the corresponding interface flux that might be used in a finite volume method.

11.1 ■ Shock wave solutions

Recall that the exact Riemann solution for (11.1) consists of a single shock or rarefaction wave. We have a shock if $q_\ell > q_r$, and we have a rarefaction if $q_\ell < q_r$. In the case of a shock wave, we can simply use the exact solution as our approximation. We have a single wave of strength $\mathcal{W} = q_r - q_\ell$ traveling at speed $s = (q_r + q_\ell)/2$ (since there is only one wave, we drop the subscript on $\mathcal{W}_\infty$ and s_1).

In this case, the numerical flux is $F^* = f(q_\ell)$ if $s \geq 0$ and $F^* = f(q_r)$ if $s \leq 0$. In the special case $s = 0$ we have a stationary shock, and it must be that $f(q_\ell) = f(q_r)$.

11.2 ■ Rarefaction wave solutions

As discussed in Chapter 10, for numerical purposes it is convenient to approximate a rarefaction wave by a traveling discontinuity. For Burgers' equation this may seem unnecessary, but for more complicated solvers for systems of equations it will be essential.

We will approximate the effect of the rarefaction wave by a fictitious shock

$$\mathcal{W} = q_r - q_\ell,$$

whose speed is given by the Rankine–Hugoniot jump condition:

$$s = \frac{f(q_r) - f(q_\ell)}{q_r - q_\ell} = \frac{q_r + q_\ell}{2}.$$

Recall that this is indeed a weak solution of the Riemann problem. This fictitious shock is not entropy satisfying, but as long as it's traveling entirely to the left or entirely to the right, the effect on the numerical solution will be the same as if we used an (entropy-satisfying) rarefaction wave. This must be true since both solutions are conservative, so averaging the wave over the single cell it affects must give the same value. The numerical flux is again $F^* = f(q_\ell)$ if $s \geq 0$ and $F^* = f(q_r)$ if $s \leq 0$.

Because this is a scalar equation with convex flux, both the Roe and the HLL approach will simplify to what we have already described. But we briefly discuss them here to further illustrate the main ideas.

11.3 ■ A Roe solver

Now we consider a linearized solver in which we replace our nonlinear hyperbolic system with a linearization about some intermediate state $\hat{q}$. For Burgers' equation, the quasilinear form is $q_t + qq_x = 0$ and the linearization gives

$$q_t + \hat{q}q_x = 0.$$

This is simply the advection equation with velocity $\hat{q}$. The solution of the Riemann problem for this equation consists of a wave $\mathcal{W} = q_r - q_\ell$ traveling at speed $s = \hat{q}$. It remains only to determine the state $\hat{q}$, which is also the wave speed.

The defining feature of a Roe linearization is that it should give the exact solution in case the states (q_ℓ, q_r) lie on a single Hugoniot locus, i.e., when the solution is a single shock. We can achieve this by choosing

$$\hat{q} = \frac{q_r + q_\ell}{2},$$

so that the Rankine–Hugoniot condition is satisfied. This is equivalent to using the approximate solver already described in the sections above.

11.3.1 ■ Examples

Below we show solutions for three examples: the first involves a shock, the second a (nontransonic) rarefaction, and the third a transonic rarefaction. In the first row we plot the exact solution in terms of the waves in the x - t plane; in the second row we plot numerical solutions obtained with PyClaw by using a simple first-order method combined with the Riemann solver.

To examine the Python code for this chapter, see

- `exact_solvers/burgers.py` ... on GitHub.⁴⁵

```
In [3]: def setup(q_l, q_r, N=500, efix=False):

    from clawpack import pyclaw
    from clawpack import riemann

    rs = riemann.burgers_1D_py.burgers_1D
    solver = pyclaw.ClawSolver1D(rs)
    solver.order = 1
    solver.kernel_language = 'Python'

    solver.bc_lower[0]=pyclaw.BC.extrap
    solver.bc_upper[0]=pyclaw.BC.extrap

    x = pyclaw.Dimension(-1.0,1.0,N,name='x')
    domain = pyclaw.Domain([x])
    state = pyclaw.State(domain,1)

    state.problem_data['efix'] = efix

    xc = state.grid.p_centers[0]

    state.q[0 ,:] = (xc<=0)*q_l + (xc>0)*q_r

    claw = pyclaw.Controller()
    claw.tfinal = 0.5
    claw.solution = pyclaw.Solution(state,domain)
    claw.solver = solver
    claw.keep_copy = True
    claw.verbosity=0

    return claw
```

```
In [4]: N = 200 # number of grid cells to use

shock = setup(2.,1.,N=N)
shock.run()
shocksol = burgers.exact_riemann_solution(2.,1.)

raref = setup(1.,2.,N=N)
raref.run()
rarefsol = burgers.exact_riemann_solution(1.,2.)

transonic = setup(-1.,2.,N=N)
transonic.run()
transonicsol = burgers.exact_riemann_solution(-1.,2.)

def plot_frame(i):
```

⁴⁵https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/burgers.py

```

fig, axes = plt.subplots(2,3,figsize=(8,4))
for ax in axes[0]:
    ax.set_xlim((-1,1)); ax.set_ylim((-1.1,2.1))

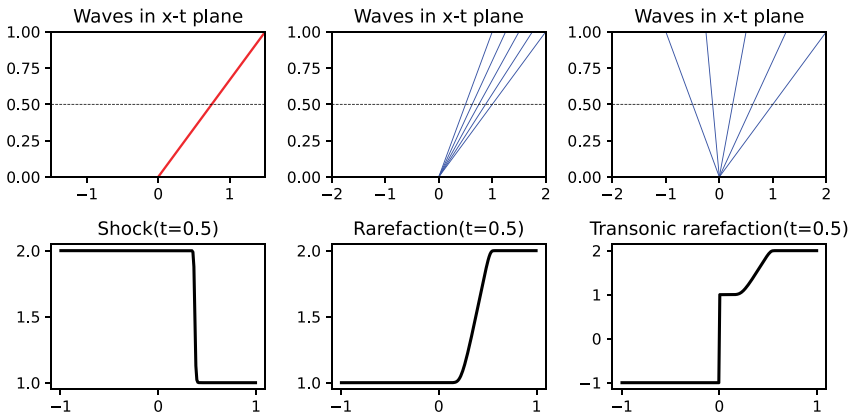
sxc = shock.frames[0].grid.x.centers
rxs = raref.frames[0].grid.x.centers
txc = transonic.frames[0].grid.x.centers

t = i/10.
axes[1][0].plot(sxc, shock.frames[i].q[0,:], '-k', lw=2)
axes[1][0].set_title('Shock(t=' + str(t) + ')')
axes[1][1].plot(rxs, raref.frames[i].q[0,:], '-k', lw=2)
axes[1][1].set_title('Rarefaction(t=' + str(t) + ')')
axes[1][2].plot(txc, transonic.frames[i].q[0,:], '-k', lw=2)
axes[1][2].set_title('Transonic rarefaction(t=' + str(t) + ')')

riemann_tools.plot_waves(*shocksol, ax=axes[0][0], t=t)
riemann_tools.plot_waves(*rarefsol, ax=axes[0][1], t=t)
riemann_tools.plot_waves(*transonicsol, ax=axes[0][2], t=t)
plt.tight_layout()
plt.show()

interact(plot_frame, i=IntSlider(min=0, max=10, value=5,
    description='Frame'));

```



The solutions obtained for the shock wave and for the first rarefaction wave are good approximations of the true solution. In the case of the transonic rarefaction, however, we see that part of the rarefaction has been replaced by an entropy-violating shock. Refining the grid in this numerical solution would not help; the method converges to a weak solution that does not satisfy the entropy condition. *In the live notebook you can try adjusting N in the cell above to investigate this.*

At the end of this chapter we will show how to apply an *entropy fix* so that the solver gives a good approximation also in the transonic case.

11.4 ■ Two-wave solvers

For Burgers' equation, the exact Riemann solution consists of a single wave. It is thus natural to modify the HLL approach by assuming that one of the waves vanishes and denoting the speed of the other wave simply by s . Then the conservation condition discussed in Section 10.4 reduces to

$$f(q_r) - f(q_\ell) = s(q_r - q_\ell),$$

which is just the Rankine–Hugoniot condition and again leads to the speed discussed above. Since the solution involves only one wave, that wave must carry the entire jump $q_r - q_\ell$, so this solver is entirely equivalent to that already described.

It is also possible to follow the full HLL approach, taking

$$\begin{aligned}s_1 &= \min f'(q) = \min(q_\ell, q_r), \\ s_2 &= \max f'(q) = \max(q_\ell, q_r).\end{aligned}$$

Since $f(q) = q^2/2$, it is easy to verify that regardless of the values of q_ℓ and q_r , this leads to

$$q_m = \frac{q_r + q_\ell}{2},$$

so that each of the two waves carries half of the jump.

11.5 ■ Transonic rarefactions

In the approaches above, the solution was approximated by a single wave traveling either to the left or to the right. For this scalar problem, this “approximation” is, in fact, an exact weak solution of the Riemann problem. As discussed already, we do not typically need to worry about the fact that it may be entropy violating, since the effect on the numerical solution (after averaging) is identical to that of the entropy-satisfying solution.

However, if $q_\ell < 0 < q_r$, then the true solution is a *transonic rarefaction*, in which part of the wave travels to the left and part travels to the right. In this case, the true Riemann solution would lead to changes to both the left and the right adjacent cells, whereas an approximate solution with a single wave will only modify one or the other. This leads to an incorrect numerical solution (on the macroscopic level), as seen in the PyClaw example above. It is therefore necessary to modify the Riemann solver, imposing what is known as an *entropy fix* in the transonic case.

Specifically, we use a solution consisting of two waves, each of which captures the net effect of the corresponding rarefaction wave that appears in the exact solution:

$$\mathcal{W}_1 = q_m - q_\ell, \quad s_1 = \frac{q_\ell + q_m}{2}, \quad (11.2)$$

$$\mathcal{W}_2 = q_r - q_m, \quad s_2 = \frac{q_m + q_r}{2}. \quad (11.3)$$

For Burgers' equation, the value $q_s = 0$ is the *sonic point* satisfying $f'(q_s) = 0$. A transonic rarefaction wave takes the value $Q^* = q_s$ along $x/t = 0$, so it makes sense to choose $q_m = 0$. The formulas above then simplify to

$$\mathcal{W}_1 = -q_\ell, \quad s_1 = q_\ell/2 < 0, \quad (11.4)$$

$$\mathcal{W}_2 = q_r, \quad s_2 = q_r/2 > 0. \quad (11.5)$$

Note that this can also be viewed as an HLL solver, although not with the usual choice of wave speeds. Choosing the wave speeds $s_1 = q_\ell/2$ and $s_2 = q_r/2$ and then solving for q_m by requiring conservation gives $q_m = 0$.

We now repeat the PyClaw simulation performed above, but with the entropy fix applied.

```
In [5]: N = 200 # number of grid cells to use
shock_efix = setup(2.,1.,N=N,efix=True)
shock_efix.run()
shocksol = burgers.exact_riemann_solution(2.,1.)

raref_efix = setup(1.,2.,N=N,efix=True)
raref_efix.run()
rarefsol = burgers.exact_riemann_solution(1.,2.)

transonic_efix = setup(-1.,2.,N=N,efix=True)
transonic_efix.run()
transonicsol = burgers.exact_riemann_solution(-1.,2.)

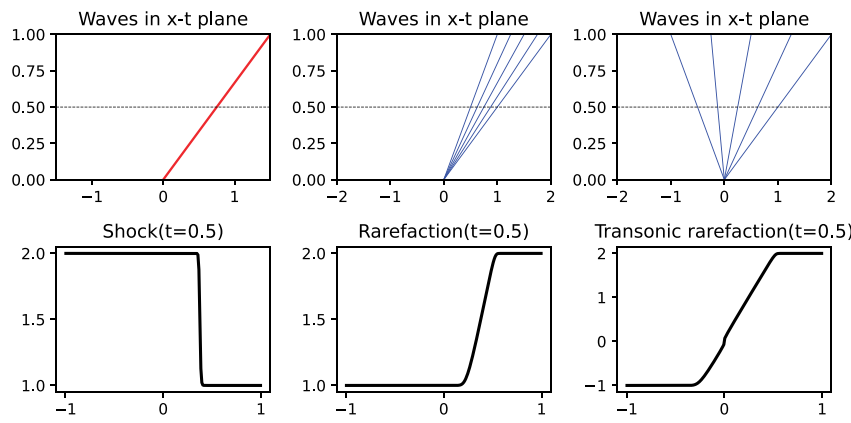
def plot_frame(i):
    fig, axes = plt.subplots(2,3,figsize=(8,4))
    for ax in axes[0]:
        ax.set_xlim((-1,1)); ax.set_ylim((-1.1,2.1))

    sxc = shock_efix.frames[0].grid.x.centers
    rxc = raref_efix.frames[0].grid.x.centers
    txc = transonic_efix.frames[0].grid.x.centers

    t = i/10.
    axes[1][0].plot(sxc, shock_efix.frames[i].q[0,:], '-k', lw=2)
    axes[1][0].set_title('Shock(t=' + str(t) + ')')
    axes[1][1].plot(rxc, raref_efix.frames[i].q[0,:], '-k', lw=2)
    axes[1][1].set_title('Rarefaction(t=' + str(t) + ')')
    axes[1][2].plot(txc, transonic_efix.frames[i].q[0,:], '-k', lw=2)
    axes[1][2].set_title('Transonic rarefaction(t=' + str(t) + ')')

    riemann_tools.plot_waves(*shocksol, ax=axes[0][0], t=t)
    riemann_tools.plot_waves(*rarefsol, ax=axes[0][1], t=t)
    riemann_tools.plot_waves(*transonicsol, ax=axes[0][2], t=t)
    plt.tight_layout()
    plt.show()
```

```
interact(plot_frame, i=IntSlider(min=0, max=10, value=5,
description='Frame'));
```



The entropy fix has no effect on the first two solutions, since it is applied only in the case of a transonic rarefaction. The third solution is greatly improved over the previous result. There is still a small *entropy glitch* visible at the origin, but the numerical solution will converge to the correct weak solution as the grid is refined. In the live notebook you can try adjusting N in the cell above to investigate this.

Chapter 12

Approximate solvers for the shallow water equations

In this chapter we discuss approximate Riemann solvers for the shallow water equations:

$$h_t + (hu)_x = 0, \quad (12.1)$$

$$(hu)_t + \left(hu^2 + \frac{1}{2}gh^2 \right)_x = 0. \quad (12.2)$$

To examine the Python code for this chapter, and for the exact Riemann solution, see

- `exact_solvers/shallow_water.py` ... on GitHub.⁴⁶

We investigated the exact solution of the Riemann problem for this system in Chapter 7. Recall that the system has two genuinely nonlinear characteristic fields, so the solution consists of two waves, each of which may be a shock or a rarefaction.

Recall that the flux Jacobian is

$$f'(q) = \begin{pmatrix} 0 & 1 \\ -(q_2/q_1)^2 + gq_1 & 2q_2/q_1 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -u^2 + gh & 2u \end{pmatrix}. \quad (12.3)$$

Its eigenvalues are

$$\lambda_1 = u - \sqrt{gh}, \quad \lambda_2 = u + \sqrt{gh}, \quad (12.4)$$

with corresponding eigenvectors

$$r_1 = \begin{bmatrix} 1 \\ u - \sqrt{gh} \end{bmatrix}, \quad r_2 = \begin{bmatrix} 1 \\ u + \sqrt{gh} \end{bmatrix}. \quad (12.5)$$

⁴⁶https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/shallow_water.py

12.1 ■ Roe solver

To define a linearized Riemann solver, we consider the quasilinear form $q_t + f'(q)q_x = 0$ and replace the Jacobian $f'(q)$ by some matrix $\hat{A}(q_\ell, q_r)$ depending on the left and right states. This matrix should satisfy three properties:

1. Consistency: $\hat{A}(q_\ell, q_r) \rightarrow f'(q)$ as $q_\ell, q_r \rightarrow q$.
2. Hyperbolicity: $\hat{A}$ must be diagonalizable with real eigenvalues, so that we can define the waves and speeds needed in the approximate solution.
3. Conservation: $\hat{A}(q_\ell, q_r)(q_r - q_\ell) = f(q_r) - f(q_\ell)$.

The first two properties are essential for any linearized approximate Riemann solver. The third property is the defining property of a Roe solver. It ensures that if q_r and q_ℓ can be connected by a single shock (i.e., if they lie on a common Hugoniot locus), then the approximate solver will in fact yield the exact solution—a single discontinuity propagating at the speed given by the Rankine–Hugoniot jump condition. This property is quite useful, since at most points (in space and time) where a shock appears, it is an isolated shock corresponding to one characteristic family. Only at isolated points in space and time do multiple shocks interact.

Here we derive the Roe solver for the shallow water equations in a simple and straightforward way by applying the conservation requirement. Roe's original paper [15] (which dealt with the Euler equations rather than the shallow water equations) gave a different and possibly more general derivation that is also reviewed in Chapter 15 of [9].

One natural approach is to look for a matrix $\hat{A}$ that is the flux Jacobian evaluated at some average state:

$$\hat{A}(q_\ell, q_r) = f'(\hat{q}).$$

This guarantees property 2 and will ensure property 1 as long as $\hat{q}$ is chosen as some kind of average of q_ℓ and q_r . For the shallow water system, this means that the linearized equations take the form

$$\begin{aligned} h_t + (hu)_x &= 0, \\ (hu)_t + (g\hat{h} - \hat{u}^2)h_x + 2\hat{u}(hu)_x &= 0. \end{aligned} \quad (12.6)$$

Observe that (12.6) is linear *not in the physical variables h, u but in the conserved variables h, hu* . In the context of a Riemann solver, $\hat{q}$ will be some kind of average of the initial states q_ℓ, q_r .

The characteristic speeds of the linear hyperbolic system (12.6) are just the characteristic speeds of the original shallow water system, evaluated at the average state $\hat{q}$:

$$s_1 = \hat{u} - \sqrt{g\hat{h}}, \quad s_2 = \hat{u} + \sqrt{g\hat{h}}, \quad (12.7)$$

with corresponding eigenvectors

$$\hat{r}_1 = r_1(\hat{q}) = \begin{bmatrix} 1 \\ \hat{u} - \sqrt{g\hat{h}} \end{bmatrix}, \quad \hat{r}_2 = r_2(\hat{q}) = \begin{bmatrix} 1 \\ \hat{u} + \sqrt{g\hat{h}} \end{bmatrix}. \quad (12.8)$$

The linearized solver simply decomposes the jump $q_r - q_\ell$ in terms of the eigenvectors, defining the waves

$$q_r - q_\ell = \alpha_1 \hat{r}_1 + \alpha_2 \hat{r}_2 = \mathcal{W}_1 + \mathcal{W}_2, \quad (12.9)$$

and uses the speeds given by (12.7). We refer to α_1, α_2 as the wave strengths; they can be found by solving the system above, which amounts to

$$\alpha = \hat{R}^{-1}(q_r - q_\ell), \quad (12.10)$$

where $\hat{R} = (\hat{r}_1 | \hat{r}_2)$ is the matrix of right eigenvectors, and thus $\hat{R}^{-1}$ is the matrix whose rows are the left eigenvectors. This leads to the expressions

$$\alpha_1 = \frac{(\hat{u} + \hat{c})\delta_1 - \delta_2}{2\hat{c}}, \quad (12.11)$$

$$\alpha_2 = \frac{-(\hat{u} - \hat{c})\delta_1 + \delta_2}{2\hat{c}}, \quad (12.12)$$

where $\hat{c} = \sqrt{g\hat{h}}$ and $\delta = q_r - q_\ell$.

12.1.1 ■ Derivation of Roe averages

It remains only to specify the average state $(\hat{h}, \hat{u})$. We do so by solving the conservation condition

$$f'(\hat{q})(q_r - q_\ell) = f(q_r) - f(q_\ell). \quad (12.13)$$

For convenience we recall here the relevant quantities

$$f'(\hat{q}) = \begin{pmatrix} 0 & 1 \\ -\hat{u}^2 + g\hat{h} & 2\hat{u} \end{pmatrix}, \quad (12.14)$$

$$q_r - q_\ell = \begin{pmatrix} h_r - h_\ell \\ h_r u_r - h_\ell u_\ell \end{pmatrix}, \quad f(q_r) - f(q_\ell) = \begin{pmatrix} h_r u_r - h_\ell u_\ell \\ h_r u_r^2 - h_\ell u_\ell^2 + \frac{g}{2}(h_r^2 - h_\ell^2) \end{pmatrix}.$$

The first equation of (12.14) is an identity, satisfied no matter how $\hat{u}$ and $\hat{h}$ are chosen. We therefore turn to the second equation:

$$(-\hat{u}^2 + g\hat{h})(h_r - h_\ell) + 2\hat{u}(h_r u_r - h_\ell u_\ell) = h_r u_r^2 - h_\ell u_\ell^2 + \frac{g}{2}(h_r^2 - h_\ell^2). \quad (12.15)$$

Since (12.15) must hold for any value of g , we can separately equate the terms involving g and those not involving g . For the terms with g in them, we get

$$\hat{h}(h_r - h_\ell) = \frac{1}{2}(h_r^2 - h_\ell^2), \quad (12.16)$$

which gives the arithmetic average

$$\hat{h} = \frac{h_r + h_\ell}{2}. \quad (12.17)$$

Equating terms that do not involve g , we get a quadratic equation for $\hat{u}$:

$$(h_r - h_\ell)\hat{u}^2 - 2(h_ru_r - h_\ell u_\ell)\hat{u} + h_ru_r^2 - h_\ell u_\ell^2 = 0.$$

The two roots of this equation are

$$\hat{u}_\pm = \frac{h_ru_r - h_\ell u_\ell \mp \sqrt{h_r h_\ell}(u_\ell - u_r)}{h_r - h_\ell}.$$

Simplifying these expressions further, we find

$$\hat{u}_\pm = \frac{\sqrt{h_r}u_r \pm \sqrt{h_\ell}u_\ell}{\sqrt{h_r} \pm \sqrt{h_\ell}}, \quad (12.18)$$

where the same sign (plus or minus) must be taken in both numerator and denominator. Either of these expressions can be used for $\hat{u}$, as long as $h_r \neq h_\ell$. The value $\hat{u}_+$ is preferable for two reasons: first, it is well defined even when $h_r = h_\ell$; second, it has the appearance of a weighted average of u , and its value always lies between u_ℓ and u_r .

The values given by (12.17) and (12.18) (with the plus sign) are known as the *Roe-averaged* depth and velocity. This completes the definition of the approximate Roe solver. An implementation appears below.

```
In [3]: def shallow_water_roe(q_l, q_r, grav=1.):
        """
        Approximate Roe solver for the shallow water equations.
        """

        h_l = q_l[0]
        hu_l = q_l[1]
        u_l = hu_l/h_l
        h_r = q_r[0]
        hu_r = q_r[1]
        u_r = hu_r/h_r

        delta = q_r - q_l

        # Roe averages
        h_hat = (h_r + h_l)/2.
        u_hat = (np.sqrt(h_r)*u_r + np.sqrt(h_l)*u_l) \
                / (np.sqrt(h_r) + np.sqrt(h_l))
        c_hat = np.sqrt(grav*h_hat)

        s1 = u_hat - c_hat
        s2 = u_hat + c_hat

        alpha1 = ((u_hat+c_hat)*delta[0] - delta[1])/(2*c_hat)
        alpha2 = (- (u_hat-c_hat)*delta[0] + delta[1])/(2*c_hat)

        h_m = q_l[0] + alpha1
        hu_m = q_l[1] + alpha1 * (u_hat - c_hat)
        q_m = np.array([h_m, hu_m])

        states = np.column_stack([q_l, q_m, q_r])
        speeds = [s1, s2]
```

```

wave_types = ['contact', 'contact']

def reval(xi):
    h_out = (xi<s1)*h_l + (s1<=xi)*(xi<=s2)*h_m + (s2<xi)*h_r
    hu_out = (xi<s1)*hu_l + (s1<=xi)*(xi<=s2)*hu_m + (s2<xi)*hu_r
    return h_out, hu_out

return states, speeds, reval, wave_types

```

Here is an example showing the solution from the Roe solver, compared with the exact Riemann solution.

In [4]: # Dam break example

```

h_l = 4; h_r = 1
u_l = 0; u_r = 0

```

```

def make_plot_functions(h_l, u_l, h_r, u_r):
    hu_l = h_l*u_l; hu_r = h_r*u_r
    q_l = np.array([h_l, hu_l]); q_r = np.array([h_r, hu_r])
    ex_states, ex_speeds, ex_reval, ex_wave_types = \
        sw.exact_riemann_solution(q_l, q_r)
    states, speeds, reval, wave_types = shallow_water_roe(q_l, q_r)

    pr = rt.make_plot_function([ex_states, states],
                              [ex_speeds, speeds],
                              [ex_reval, reval],
                              [ex_wave_types, wave_types],
                              variable_names=sw.conservated_variables)

    pp = sw.phase_plane_plot(q_l, q_r, g=1., y_axis='u',
                             approx_states=states)

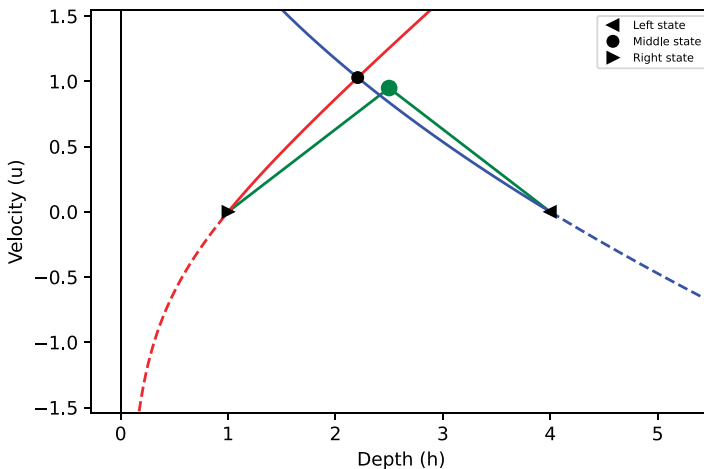
    return pp, pr

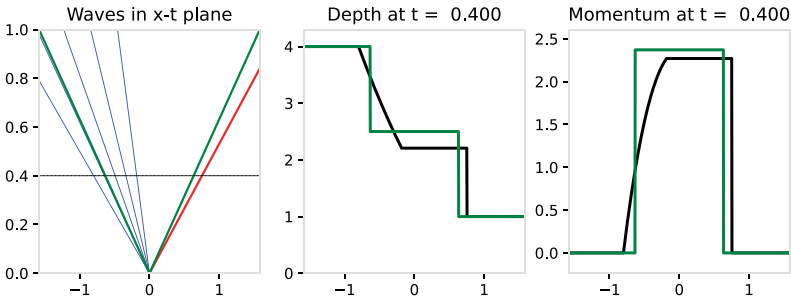
```

```

phase_plane, comparison = make_plot_functions(h_l, u_l, h_r, u_r)
interact(comparison, t=widgets.FloatSlider(min=0, max=.9, step=.1, value=0.4));

```

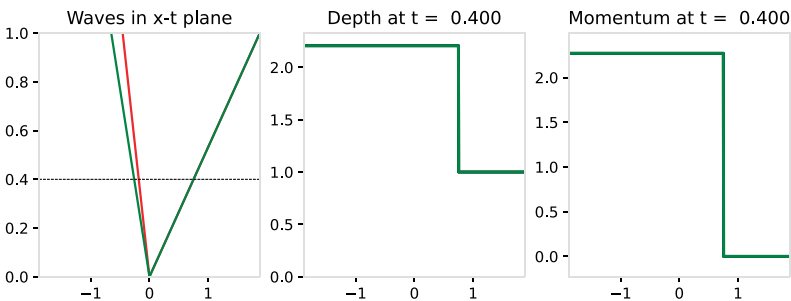
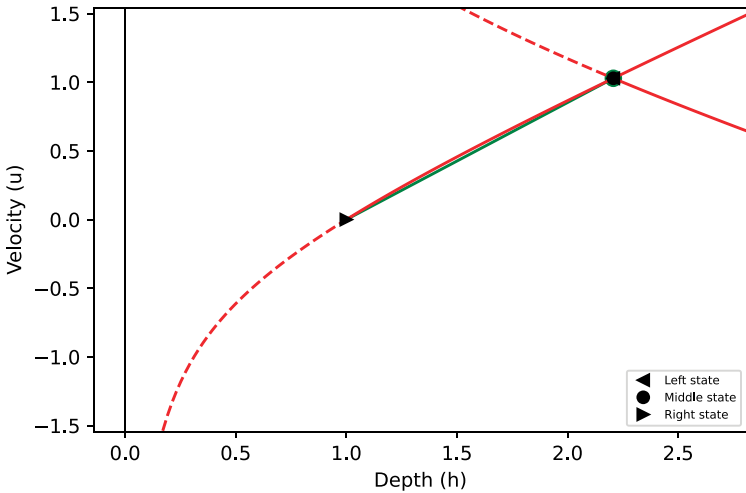




Here and throughout this section, we plot the approximate solution in green. As usual, Hugoniot loci and shock paths are plotted in red, while integral curves and rarefaction fans are plotted in blue.

Notice that the rarefaction wave is approximated by a discontinuity. The next example illustrates the exactness property for isolated shocks. Here we have chosen q_ℓ and q_r such that they are related by a single 2-shock (up to rounding errors).

```
In [5]: h_l = 2.20698770767; h_r = 1.  
        u_l = 2.27057814896/h_l; hu_r = 0.  
  
phase_plane, comparison = make_plot_functions(h_l, u_l, h_r, u_r)  
interact(comparison, t=widgets.FloatSlider(min=0,max=.9,step=.1,value=0.4));
```



Because the exact and approximate solutions are the same, the exact solution is not visible in these plots. The only exception is the 1-wave in the first plot; the speed of the 1-wave appears different for the Roe solution and the exact solution, but this is unimportant since the 1-wave has zero strength.

12.1.2 ■ Entropy fix

As discussed previously, using an entropy-violating discontinuity in place of a rarefaction can be problematic in the case of a transonic rarefaction. For the Roe solver, we can again apply an entropy fix, splitting the single discontinuity into two waves, to avoid this problem. Several different entropy fixes have been proposed; here we focus on one that is quite general and effective in practice.

Suppose that $\lambda_1(q_\ell) < 0$ while $\lambda_1(q_m) > 0$. In this case, it appears that the 1-wave should be a transonic rarefaction. We take the 1-wave $\mathcal{W}_1$ of the Roe solver, with speed s_1 , and split it into two waves, $\mathcal{W}_{1\ell} = \alpha_{1\ell} r_1$ and $\mathcal{W}_{1r} = \alpha_{1r} r_1$, with speeds $\lambda_1(q_\ell), \lambda_1(q_m)$. Two requirements determine the strength of these waves. First, the total jump across them should be equal to the jump across the original wave:

$$\mathcal{W}_{1\ell} + \mathcal{W}_{1r} = \mathcal{W}_1.$$

Second, we must maintain conservation:

$$\lambda_1(q_\ell)\mathcal{W}_{1\ell} + \lambda_1(q_m)\mathcal{W}_{1r} = s_1\mathcal{W}_1.$$

This leads to

$$\begin{aligned}\mathcal{W}_{1\ell} &= \frac{\lambda_1(q_m) - s_1}{\lambda_1(q_m) - \lambda_1(q_\ell)} \mathcal{W}_1, \\ \mathcal{W}_{1r} &= \frac{s_1 - \lambda_1(q_\ell)}{\lambda_1(q_m) - \lambda_1(q_\ell)} \mathcal{W}_1.\end{aligned}$$

We can check in a similar way for a transonic 2-rarefaction and modify the 2-wave accordingly. Note that only one of the two characteristic fields can be transonic in the Riemann solution for the shallow water equations.

This entropy fix is implemented in the solver below.

```
In [6]: def shallow_water_roe_with_efix(q_l, q_r, grav=1., efix=True):
        """
        Approximate Roe solver for the shallow water equations,
        with entropy fix.
        """

        h_l = q_l[0]
        hu_l = q_l[1]
        u_l = hu_l/h_l
        h_r = q_r[0]
        hu_r = q_r[1]
        u_r = hu_r/h_r

        delta = q_r - q_l
```

```

# Roe averages
h_hat = (h_r + h_l)/2.
u_hat = (np.sqrt(h_r)*u_r + np.sqrt(h_l)*u_l) / \
        (np.sqrt(h_r) + np.sqrt(h_l))
c_hat = np.sqrt(grav*h_hat)

s1 = u_hat - c_hat
s2 = u_hat + c_hat

alpha1 = ( (u_hat+c_hat)*delta[0] - delta[1])/(2*c_hat)
#alpha2 = (-(u_hat-c_hat)*delta[0] + delta[1])/(2*c_hat)

h_m = q_l[0] + alpha1
hu_m = q_l[1] + alpha1 * (u_hat - c_hat)
u_m = hu_m/h_m
q_m = np.array([h_m, hu_m])

transonic = None
if efix:
    # Check if either wave appears transonic
    lambda_1_l = u_l - np.sqrt(grav*h_l)
    lambda_1_m = u_m - np.sqrt(grav*h_m)
    lambda_2_m = u_m + np.sqrt(grav*h_m)
    lambda_2_r = u_r + np.sqrt(grav*h_r)

    if lambda_1_l < 0 < lambda_1_m: # 1-wave appears transonic
        transonic = 1
    elif lambda_2_m < 0 < lambda_2_r: # 2-wave appears transonic
        transonic = 2

if transonic == 1:
    beta = (lambda_1_m - s1)/(lambda_1_m - lambda_1_l)
    alpha1_l = beta*alpha1
    #alpha_1_r = (1-beta)*alpha1
    h_m_new = q_l[0] + alpha1_l
    hu_m_new = q_l[1] + alpha1_l * (u_hat - c_hat)
    q_m_new = np.array([h_m_new, hu_m_new])
    states = np.column_stack([q_l, q_m_new, q_m, q_r])
    speeds = [lambda_1_l, lambda_1_m, s2]
    wave_types = ['contact']*3

def reval(xi):
    h_out = (xi<speeds[0])*states[0,0] + \
            (speeds[0]<=xi)*(xi<speeds[1])*states[0,1] + \
            (speeds[1]<=xi)*(xi<speeds[2])*states[0,2] + \
            (speeds[2]<=xi)*states[0,3]
    hu_out = (xi<speeds[0])*states[1,0] + \
            (speeds[0]<=xi)*(xi<speeds[1])*states[1,1] + \
            (speeds[1]<=xi)*(xi<speeds[2])*states[1,2] + \
            (speeds[2]<=xi)*states[1,3]
    return h_out, hu_out

```

```

elif transonic == 2:
    beta = (lambda_2_r - s2)/(lambda_2_r - lambda_2_m)
    #alpha2_l = beta*alpha2
    alpha2_r = (1-beta)*alpha2
    h_m_new = q_r[0] - alpha2_r
    hu_m_new = q_r[1] - alpha2_r*(u_hat+c_hat)
    q_m_new = np.array([h_m_new, hu_m_new])
    states = np.column_stack([q_l, q_m, q_m_new, q_r])
    speeds = [s1, lambda_2_, lambda_2_r]
    wave_types = ['contact']*3

def reval(xi):
    h_out = (xi<speeds[0])*states[0,0] + \
            (speeds[0]<=xi)*(xi<speeds[1])*states[0,1] + \
            (speeds[1]<=xi)*(xi<speeds[2])*states[0,2] + \
            (speeds[2]<=xi)*states[0,3]
    hu_out = (xi<speeds[0])*states[1,0] + \
            (speeds[0]<=xi)*(xi<speeds[1])*states[1,1] + \
            (speeds[1]<=xi)*(xi<speeds[2])*states[1,2] + \
            (speeds[2]<=xi)*states[1,3]
    return h_out, hu_out

else:
    states = np.column_stack([q_l, q_m, q_r])
    speeds = [s1, s2]
    wave_types = ['contact', 'contact']

def reval(xi):
    h = (xi<s1)*h_l + (s1<=xi)*(xi<=s2)*h_m + (s2<xi)*h_r
    hu = (xi<s1)*hu_l + (s1<=xi)*(xi<=s2)*hu_m + (s2<xi)*hu_r
    return h, hu

return states, speeds, reval, wave_types

```

An implementation of this solver for use in Clawpack can be found here.⁴⁷

12.1.3 ■ Roe solver examples

Here are some examples of problems with transonic rarefactions, showing the behavior of the solver. The first example involves two rarefaction waves; the left-going one is transonic.

```
In [7]: h_l = 1; h_r = 1
        u_l = 0.5; u_r = 2
```

```

def compare_efix(h_l, u_l, h_r, u_r):
    hu_l = h_l*u_l; hu_r = h_r*u_r
    q_l = np.array([h_l, hu_l]); q_r = np.array([h_r, hu_r])
    ex_states, ex_speeds, ex_reval, ex_wave_types = \
        sw.exact_riemann_solution(q_l, q_r)

```

⁴⁷https://github.com/clawpack/riemann/blob/FA16/src/rp1_shallow_roe_with_efix.f90

```

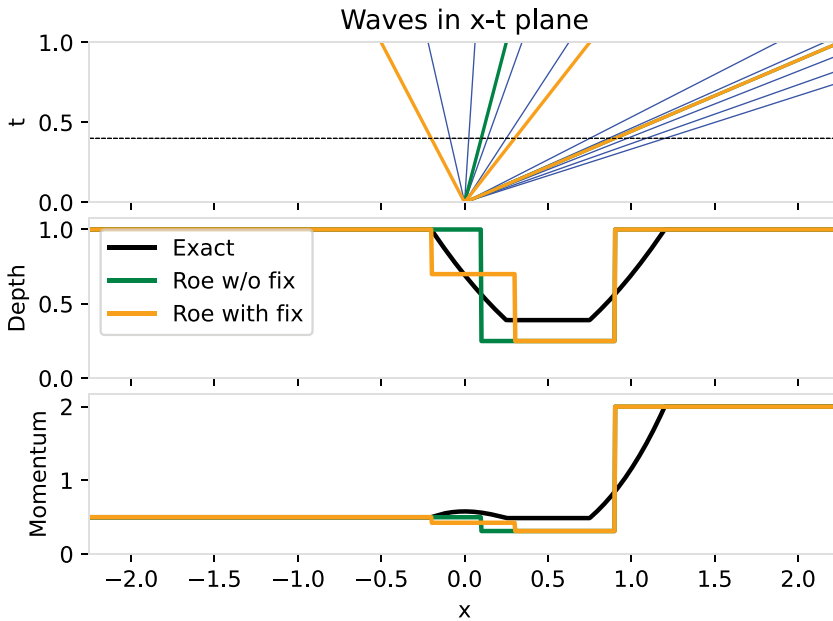
states, speeds, reval, wave_types = \
    shallow_water_roe_with_efix(q_l, q_r)
no_efix_states, no_efix_speeds, no_efix_reval,
    no_efix_wave_types = shallow_water_roe(q_l, q_r)

plot_function = \
    rt.make_plot_function([ex_states, no_efix_states, states],
        [ex_speeds, no_efix_speeds, speeds],
        [ex_reval, no_efix_reval, reval],
        [ex_wave_types, no_efix_wave_types,
         wave_types],
        ['Exact', 'Roe w/o fix', 'Roe with fix'],
        layout='vertical',
        variable_names=sw.conserved_variables)

interact(plot_function,
    t=widgets.FloatSlider(min=0,max=.9,step=.1,value=0.4));
return q_l, q_r, states, no_efix_states

q_l, q_r, states, no_efix_states = compare_efix(h_l,u_l,h_r,u_r)

```

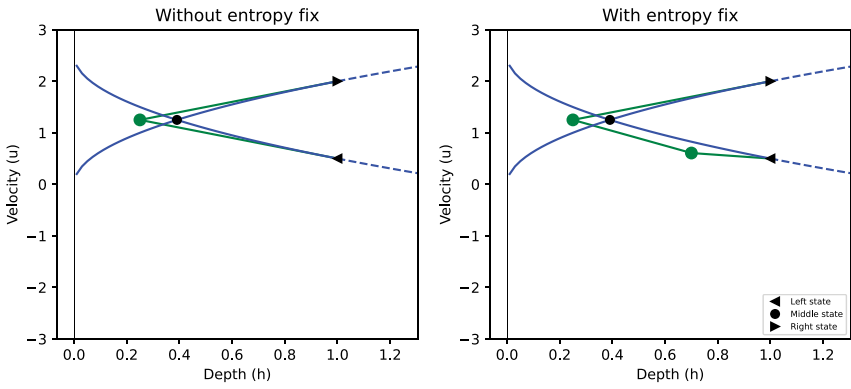


```

In [8]: fig, ax = plt.subplots(1,2,figsize=(10,4))
        sw.phase_plane_plot(q_l,q_r,g=1.,y_axis='u',
            approx_states=no_efix_states,ax=ax[0])
        sw.phase_plane_plot(q_l,q_r,g=1.,y_axis='u',
            approx_states=states,ax=ax[1],color='orange')

```

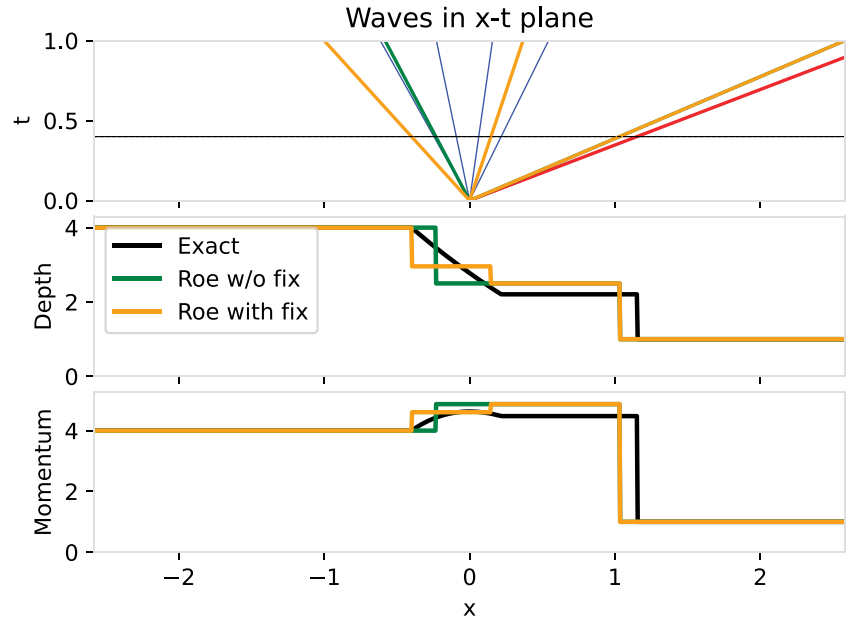
```
ax[0].set_title('Without entropy fix')
ax[1].set_title('With entropy fix');
```



The next example is a dam break problem; the rarefaction wave is transonic because initially the water is all moving to the right.

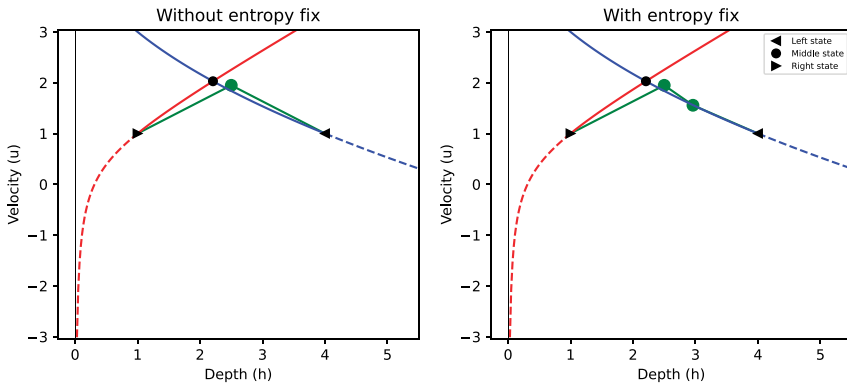
```
In [9]: h_l = 4; h_r = 1
        u_l = 1; u_r = 1

        q_l, q_r, states, no_efix_states = compare_efix(h_l,u_l,h_r,u_r)
```



```
In [10]: fig, ax = plt.subplots(1,2,figsize=(10,4))
        sw.phase_plane_plot(q_l,q_r,g=1.,y_axis='u',
                           approx_states=no_efix_states,ax=ax[0])
        sw.phase_plane_plot(q_l,q_r,g=1.,y_axis='u',
                           approx_states=states,ax=ax[1],color='orange')
```

```
ax[0].set_title('Without entropy fix')
ax[1].set_title('With entropy fix');
```



12.1.4 • Positivity

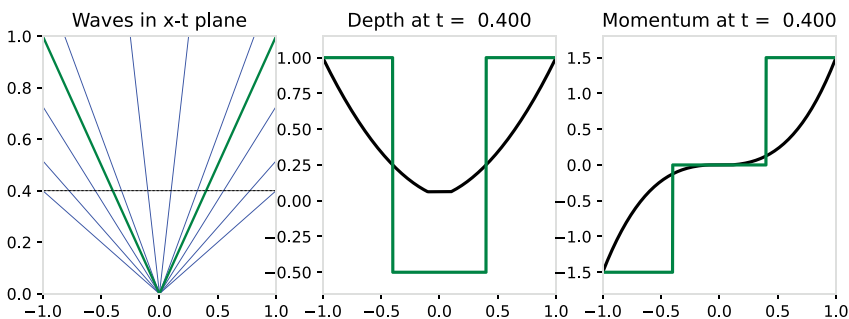
In the shallow water system, it is both physically and mathematically essential that the depth remain nonnegative. Unfortunately, as the next example shows, the Roe solver does not always guarantee this.

```
In [11]: h_l = 1; h_r = 1
         u_l = -1.5; u_r = 1.5

         hu_l = h_l*u_l; hu_r = h_r*u_r
         q_l = np.array([h_l, hu_l]); q_r = np.array([h_r, hu_r])
         ex_states, ex_speeds, ex_reval, ex_wave_types = \
             sw.exact_riemann_solution(q_l, q_r)
         states, speeds, reval, wave_types = \
             shallow_water_roe_with_efix(q_l, q_r)

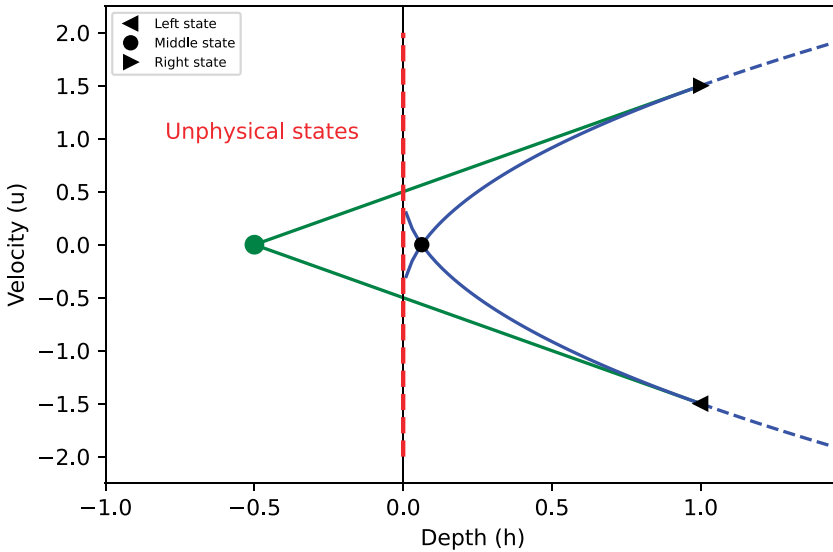
         plot_function = \
             rt.make_plot_function([ex_states, states],
                                   [ex_speeds, speeds],
                                   [ex_reval, reval],
                                   [ex_wave_types,
                                    wave_types],
                                   variable_names=sw.conserved_variables)

         interact(plot_function,
                   t=widgets.FloatSlider(min=0, max=.9, step=.1, value=0.4));
```



```
In [12]: sw.phase_plane_plot(q_l,q_r,g=1.,y_axis='u',
                             approx_states=states, hmin = -1)
plt.plot([0,0], [-2,2], 'r--')
plt.text(-0.8,1.5,'Unphysical states',color='red')
```

Out[12]:



In fact, this is a feature of all linearized Riemann solvers: for some Riemann problems, they will produce negative intermediate states. The HLL solver described next does guarantee a positive solution for all initial data.

12.2 ■ HLL solver

12.2.1 ■ Derivation and implementation

Recall that an HLL approximate solver is based on a fictitious solution consisting of two waves (discontinuities) with a single intermediate state q_m between them. The waves are thus

$$\mathcal{W}_1 = q_m - q_\ell, \quad (12.19)$$

$$\mathcal{W}_2 = q_r - q_m. \quad (12.20)$$

The middle state is determined by the requirement of conservation:

$$s_1 \mathcal{W}_1 + s_2 \mathcal{W}_2 = f(q_r) - f(q_\ell), \quad (12.21)$$

which can be solved for q_m to find

$$q_m = \frac{f(q_r) - f(q_\ell) - s_2 q_r + s_1 q_\ell}{s_1 - s_2}. \quad (12.22)$$

The key remaining ingredient is the choice of the wave speeds s_1, s_2 . These should approximate the wave speeds appearing in the true Riemann solution. In general, it is desirable for the purpose of stability that the speeds used be approximations of the fastest (left- and right-going) waves in the Riemann solution. Indeed, in the original paper of Harten, Lax, and van Leer [4] it was proposed to use lower and upper bounds on the true wave speeds. However, overestimating the wave speeds leads to increased diffusion, and various refinements to the choice of wave speeds were subsequently proposed. Here we choose the wave speeds based on a suggestion of Einfeldt (originally in the context of the Euler equations); the resulting solver is often referred to as an HLLE solver.

For the shallow water system, the 1-wave speed is always less than or equal to the 2-wave speed. If the 1-wave is a rarefaction, then the minimum wave velocity is given simply by $\lambda_1(q_\ell)$, the characteristic speed at the left edge of the 1-rarefaction fan. If the left-going wave speed is a shock, then the minimum wave velocity is the shock speed. We cannot determine the shock speed exactly without solving the Riemann problem—just what we hope to avoid. Instead, we can use an approximate shock speed based on the Roe average:

$$\hat{s}_1 = \hat{u} - \hat{c}.$$

Since we don't know whether the 1-wave should be a shock or a rarefaction, we use a lower bound on these two speeds by taking their minimum:

$$s_1 = \min(\lambda_1(q_\ell), \hat{u} - \hat{c}).$$

Similarly, the second wave speed is given by

$$s_2 = \max(\lambda_2(q_r), \hat{u} + \hat{c}).$$

This completes the definition of the solver. Notice that if the states q_ℓ, q_r are connected by a single 2-shock, then the value of s_2 will be the exact shock speed (since the Roe speed is exact in this case, and the shock speed will be faster than the characteristic speed ahead of it). Furthermore, by substituting the Rankine–Hugoniot condition into (12.22), we see that the middle state in this case will be $q_m = q_\ell$. Thus the HLLE solver gives the exact solution for this case. Similarly, it yields the exact solution when the initial states are connected by just a 1-shock. It shares this nice property with the Roe solver (as long as the wave speeds are chosen as just described).

```
In [13]: def shallow_water_hll(q_l, q_r, grav=1.):
        """
        HLLE approximate solver for the shallow water equations.
        """
        h_l = q_l[0]
        hu_l = q_l[1]
        u_l = hu_l/h_l
        h_r = q_r[0]
        hu_r = q_r[1]
        u_r = hu_r/h_r

        # Roe averages
        h_hat = (h_r + h_l)/2.
```

```

u_hat = (np.sqrt(h_r)*u_r + np.sqrt(h_l)*u_l) / \
        (np.sqrt(h_r) + np.sqrt(h_l))
c_hat = np.sqrt(grav*h_hat)

lambda_1_l = u_l - np.sqrt(grav*h_l)
lambda_2_r = u_r + np.sqrt(grav*h_r)

s1 = min(lambda_1_l, u_hat - c_hat)
s2 = max(lambda_2_r, u_hat + c_hat)

h_m = (hu_r - hu_l - s2*h_r + s1*h_l)/(s1-s2)
hu_m = (hu_r*u_r - hu_l*u_l + 0.5*grav*(h_r**2 - h_l**2) \
        - s2*hu_r + s1*hu_l)/(s1-s2)
q_m = np.array([h_m, hu_m])

states = np.column_stack([q_l,q_m,q_r])
speeds = [s1, s2]
wave_types = ['contact', 'contact']

def reval(xi):
    h_out = (xi<s1)*h_l + (s1<=xi)*(xi<=s2)*h_m + (s2<xi)*h_r
    hu_out = (xi<s1)*hu_l + (s1<=xi)*(xi<=s2)*hu_m + (s2<xi)*hu_r
    return h_out, hu_out

return states, speeds, reval, wave_types

```

An implementation of this solver for use in Clawpack can be found here.⁴⁸

12.2.2 ■ Examples using the HLL solver

```

In [14]: def compare_solutions(q_l, q_r, solvers):
    outputs = []
    states = {}

    for solver in solvers:
        if solver.lower() == 'exact':
            outputs.append(sw.exact_riemann_solution(q_l,q_r))
        if solver.lower() == 'hll':
            outputs.append(shallow_water_hll(q_l, q_r))
            states['hll'] = outputs[-1][0]
        if solver.lower() == 'roe':
            outputs.append(shallow_water_roe_with_efix(q_l, q_r))
            states['roe'] = outputs[-1][0]
        if solver.lower() == 'roe no efix':
            outputs.append(shallow_water_roe(q_l, q_r))
            states['roe no efix'] = outputs[-1][0]

    plot_function = \
        rt.make_plot_function([val[0] for val in outputs],
                               [val[1] for val in outputs],
                               [val[2] for val in outputs],

```

⁴⁸https://github.com/clawpack/riemann/blob/FA16/src/rp1_shallow_hlle.f90

```
[val[3] for val in outputs],
solvers, layout='horizontal',
variable_names=sw.conservated_variables)
```

```
interact(plot_function,
         t=widgets.FloatSlider(min=0,max=0.9,step=0.1,value=0.4));
```

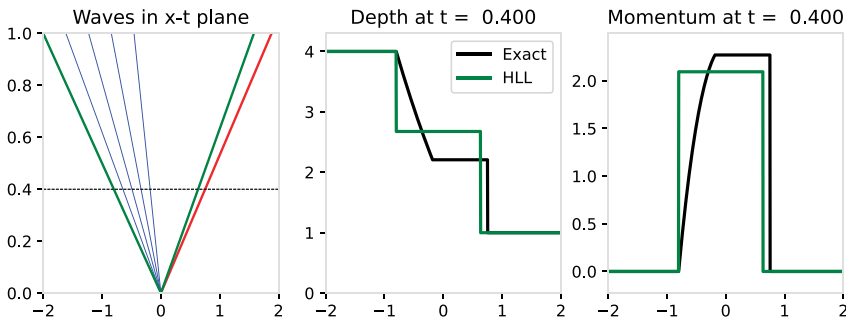
```
return states
```

Our first example is a standard dam break problem.

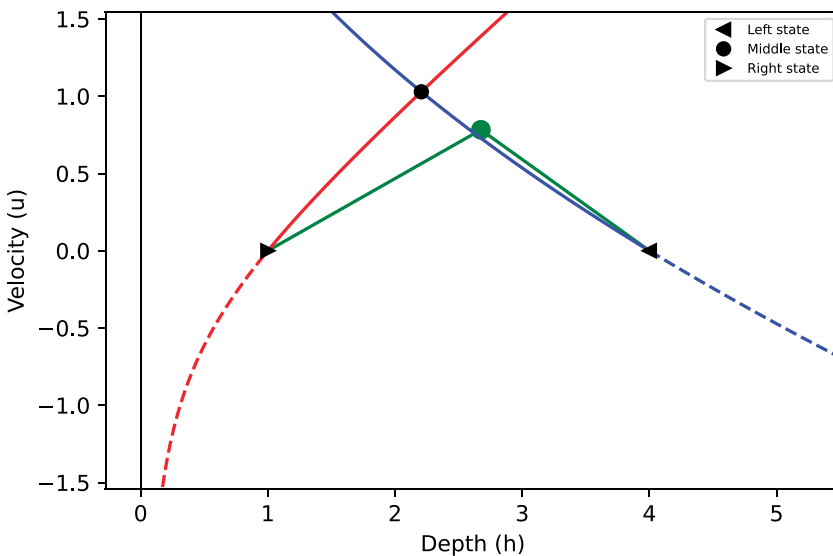
```
In [15]: # Dam break example
         h_l = 4; h_r = 1
         u_l = 0; u_r = 0

         hu_l = h_l*u_l; hu_r = h_r*u_r
         q_l = np.array([h_l, hu_l]); q_r = np.array([h_r, hu_r])

         states = compare_solutions(q_l, q_r, ['Exact', 'HLL'])
```



```
In [16]: sw.phase_plane_plot(q_l,q_r,g=1.,y_axis='u',
                             approx_states=states['hll'])
```

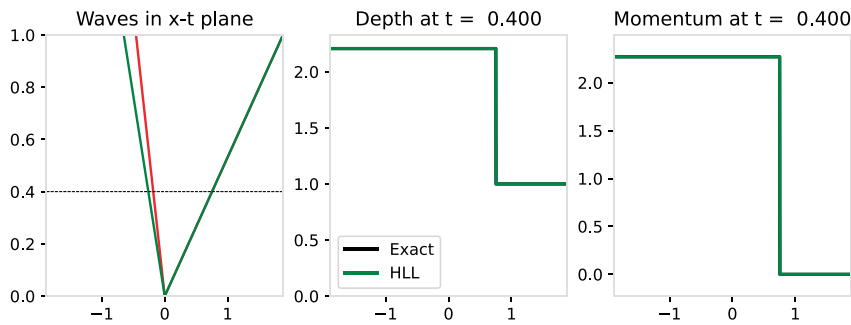


In the next example, the left and right states are constructed to be connected exactly by a single right-going shock. As we have discussed, the solver captures this solution exactly.

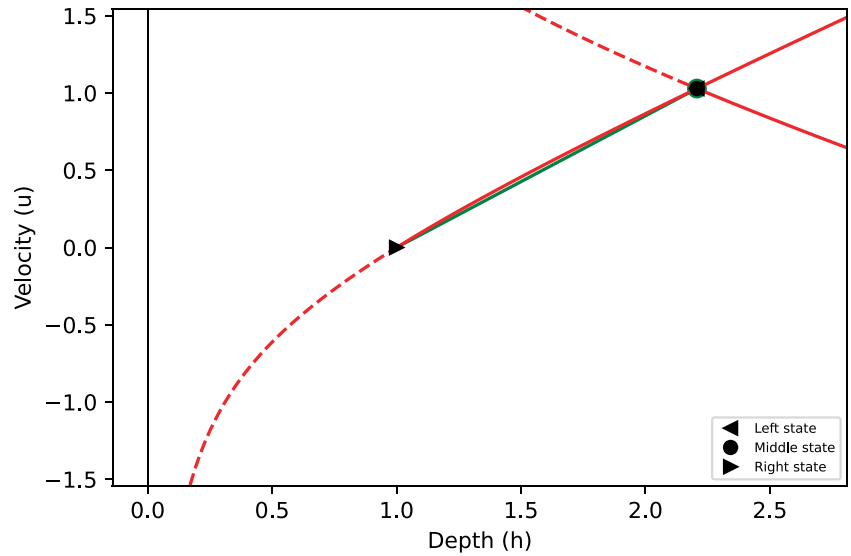
```
In [17]: # Isolated 2-shock example
        h_l = 2.20698770767; h_r = 1.
        u_l = 2.27057814896/h_l; hu_r = 0.

        hu_l = h_l*u_l; hu_r = h_r*u_r
        q_l = np.array([h_l, hu_l]); q_r = np.array([h_r, hu_r])

        states = compare_solutions(q_l, q_r, ['Exact', 'HLL'])
```



```
In [18]: sw.phase_plane_plot(q_l,q_r,g=1.,y_axis='u',
                             approx_states=states['hll'])
```



The final example includes a transonic rarefaction.

In [19]: *# Transonic rarefaction example a*

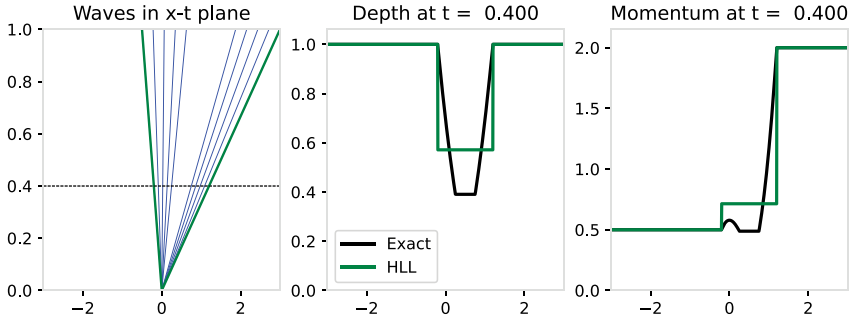
```
h_l = 1; h_r = 1
```

```
u_l = 0.5; u_r = 2
```

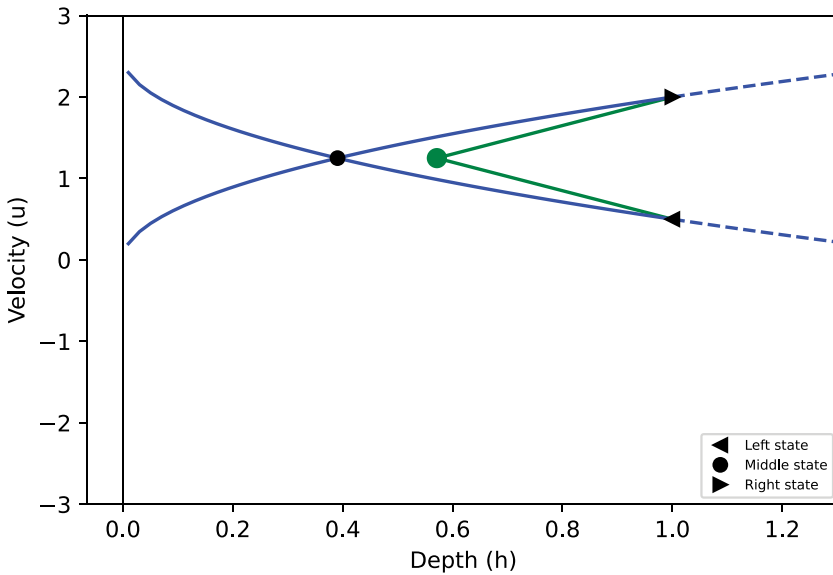
```
hu_l = h_l*u_l; hu_r = h_r*u_r
```

```
q_l = np.array([h_l, hu_l]); q_r = np.array([h_r, hu_r])
```

```
states = compare_solutions(q_l, q_r, ['Exact', 'HLL'])
```



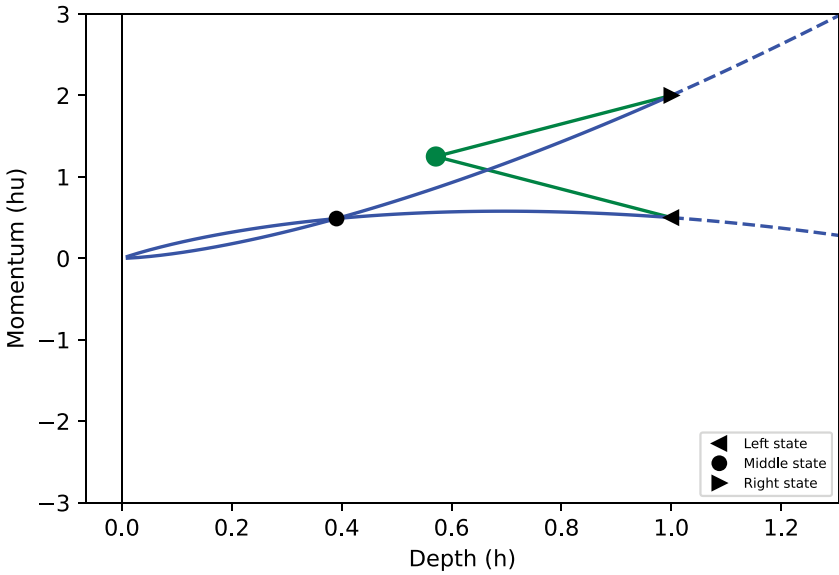
In [20]: `sw.phase_plane_plot(q_l,q_r,g=1.,y_axis='u',
approx_states=states['hll'])`



Notice that, by design, the HLL wave always lies exactly on the leading edge of a rarefaction wave when compared to the true solution.

The momentum solution above has the interesting feature that the solution in the 1-rarefaction is not monotone but appears as a “bump.” This can be explained by plotting the solution in the h - hu phase plane, where we see that the integral curve has a local maximum between q_ℓ and q_m .

```
In [21]: sw.phase_plane_plot(q_l,q_r,g=1.,y_axis='hu',
                             approx_states=states['hll'])
```



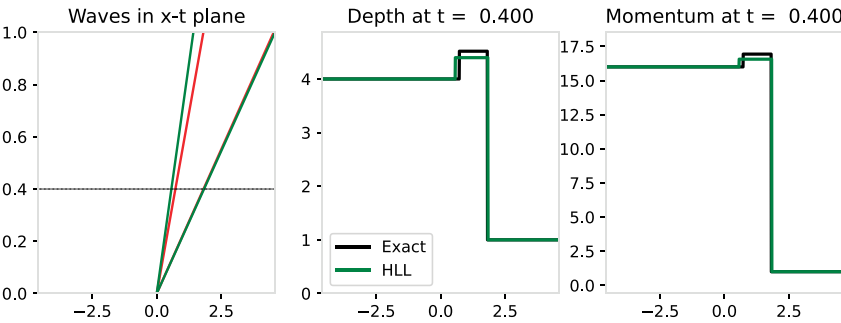
12.2.3 ■ Entropy and positivity

It can be shown that the HLL solver always preserves positivity and does not require an entropy fix. Both of these properties result in part from the fact that the HLL solver is more diffusive than the Roe solver.

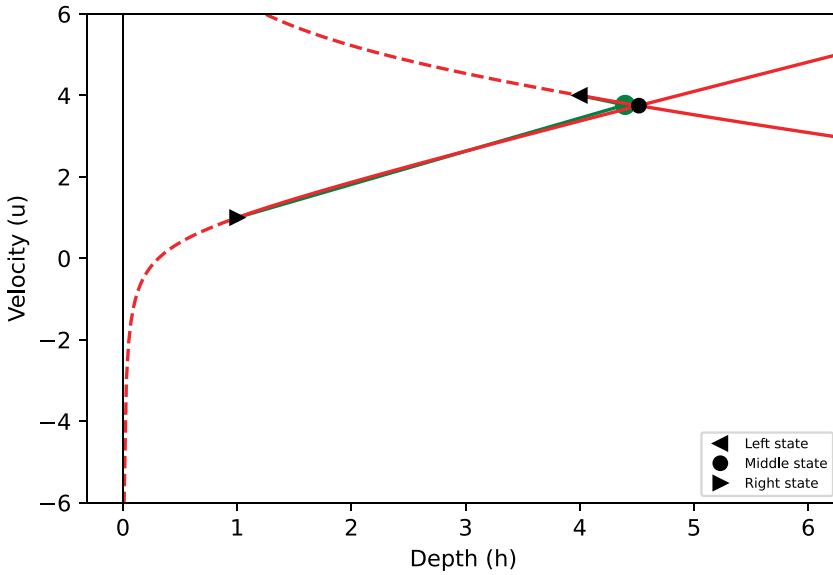
```
In [22]: # Transonic rarefaction example b
h_l = 4; h_r = 1
u_l = 4; u_r = 1

hu_l = h_l*u_l; hu_r = h_r*u_r
q_l = np.array([h_l, hu_l]); q_r = np.array([h_r, hu_r])

states = compare_solutions(q_l, q_r, ['Exact', 'HLL'])
```



```
In [23]: sw.phase_plane_plot(q_l,q_r,g=1.,y_axis='u',
                             approx_states=states['hll'])
```

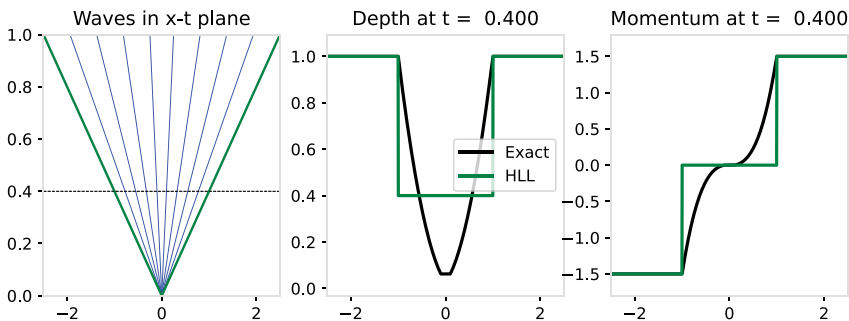


The final example below illustrates that the HLL solver maintains positivity when the same initial data is used as in the example with the Roe solver above, where a negative intermediate depth was obtained.

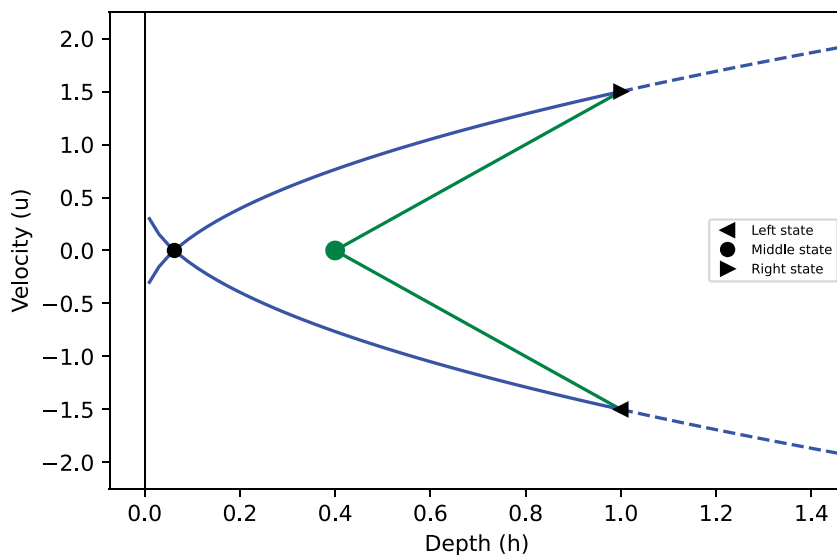
```
In [24]: h_l = 1; h_r = 1
         u_l = -1.5; u_r = 1.5

         hu_l = h_l*u_l; hu_r = h_r*u_r
         q_l = np.array([h_l, hu_l]); q_r = np.array([h_r, hu_r])

         states = compare_solutions(q_l, q_r, ['Exact', 'HLL'])
```



```
In [25]: sw.phase_plane_plot(q_l,q_r,g=1.,y_axis='u',
                             approx_states=states['hll'])
```



The above discussion concerns the nature of a single approximate Riemann solution. In a numerical method, the approximate solver is used repeatedly, so it is interesting to compare the performance of the HLL and Roe Riemann solvers when they are used within a full discretization to solve the shallow water equations. We will do so in Chapter 14.

Chapter 13

Approximate solvers for the Euler equations of gas dynamics

In this chapter we discuss approximate solvers for the one-dimensional Euler equations

$$\rho_t + (\rho u)_x = 0, \quad (13.1)$$

$$(\rho u)_t + (\rho u^2 + p)_x = 0, \quad (13.2)$$

$$E_t + ((E + p)u)_x = 0. \quad (13.3)$$

As in Chapter 9, we focus on the case of an ideal gas, for which the total energy is given by

$$E = \frac{p}{\gamma - 1} + \frac{1}{2}\rho u^2. \quad (13.4)$$

To examine the Python code for this chapter, and for the exact Riemann solution, see

- `exact_solvers/euler.py` ... on GitHub.⁴⁹

13.1 ■ Roe solver

We first derive a Roe solver for the Euler equations, following the same approach as in Chapter 12. Namely, we assume that $\hat{A} = f'(\hat{q})$ for some average state $\hat{q}$ and impose the condition of conservation:

$$f'(\hat{q})(q_r - q_\ell) = f(q_r) - f(q_\ell). \quad (13.5)$$

We will need the following quantities:

$$q = \begin{pmatrix} \rho \\ \rho u \\ E \end{pmatrix}, \quad f(q) = \begin{pmatrix} \rho u \\ \rho u^2 + p \\ Hu\rho \end{pmatrix}, \quad (13.6)$$

⁴⁹https://github.com/clawpack/riemann_book/blob/FA16/exact_solvers/euler.py

$$f'(\hat{q}) = \begin{pmatrix} 0 & 1 & 0 \\ \frac{\gamma-3}{2}\hat{u}^2 & (3-\gamma)\hat{u} & \gamma-1 \\ \frac{\gamma-1}{2}\hat{u}^3 - \hat{u}\hat{H} & \hat{H} - (\gamma-1)\hat{u}^2 & \gamma\hat{u} \end{pmatrix}. \quad (13.7)$$

Here $H = \frac{E+p}{\rho}$ is the enthalpy. We have rewritten most expressions involving E in terms of H because it simplifies the derivation that follows. We now solve (13.5) to find $\hat{u}$ and $\hat{H}$. It turns out that, for the case of a polytropic ideal gas, the average density $\hat{\rho}$ plays no role in the Roe solver.

The first equation of the system (13.5) is an identity, satisfied independent of our choice of $\hat{q}$. The second equation is (using (13.4))

$$\begin{aligned} & \frac{\gamma-3}{2}\hat{u}^2(\rho_r - \rho_\ell) + (3-\gamma)\hat{u}(\rho_r u_r - \rho_\ell u_\ell) \\ & + (\gamma-1) \left(\frac{p_r - p_\ell}{\gamma-1} + \frac{1}{2}(\rho_r u_r^2 - \rho_\ell u_\ell^2) \right) = \rho_r u_r^2 - \rho_\ell u_\ell^2 + p_r - p_\ell, \end{aligned} \quad (13.8)$$

which simplifies to a quadratic equation for $\hat{u}$:

$$(\rho_r - \rho_\ell)\hat{u}^2 - 2(\rho_r u_r - \rho_\ell u_\ell)\hat{u} + (\rho_r u_r^2 - \rho_\ell u_\ell^2) = 0, \quad (13.9)$$

with roots

$$\hat{u}_\pm = \frac{\rho_r u_r - \rho_\ell u_\ell \mp \sqrt{\rho_r \rho_\ell}(u_\ell - u_r)}{\rho_r - \rho_\ell} = \frac{\sqrt{\rho_r} u_r \pm \sqrt{\rho_\ell} u_\ell}{\sqrt{\rho_r} \pm \sqrt{\rho_\ell}}. \quad (13.10)$$

Notice that this is identical to the Roe average of the velocity for the shallow water equations if we replace the density ρ with depth h . As before, we choose the root u_+ since it is well defined for all values of ρ_r, ρ_ℓ .

Next we find $\hat{H}$ by solving the last equation of the system (13.5), which reads

$$\begin{aligned} & \left(\frac{\gamma-1}{2}\hat{u}^3 - \hat{u}\hat{H} \right) (\rho_r - \rho_\ell) \\ & + \left(\hat{H} - (\gamma-1)\hat{u}^2 \right) (\rho_r u_r - \rho_\ell u_\ell) + \gamma\hat{u}(E_r - E_\ell) = H_r u_r \rho_r - H_\ell u_\ell \rho_\ell. \end{aligned} \quad (13.11)$$

We can simplify this using the equality $\gamma E = \rho H + \frac{\gamma-1}{2}\rho u^2$ and solve for $\hat{H}$ to find

$$\begin{aligned} \hat{H}_\pm &= \frac{\rho_r H_r(u_r - \hat{u}_\pm) - \rho_\ell H_\ell(u_\ell - \hat{u}_\pm)}{\rho_r u_r - \rho_\ell u_\ell - \hat{u}_\pm(\rho_r - \rho_\ell)} \\ &= \frac{\rho_r H_r(u_r - \hat{u}_\pm) - \rho_\ell H_\ell(u_\ell - \hat{u}_\pm)}{\pm \sqrt{\rho_r \rho_\ell}(u_r - u_\ell)} \end{aligned}$$

$$\begin{aligned}
&= \frac{\rho_r H_r - \rho_\ell H_\ell \mp \sqrt{\rho_r \rho_\ell} (H_r - H_\ell)}{\rho_r - \rho_\ell} \\
&= \frac{\sqrt{\rho_r} H_r \pm \sqrt{\rho_\ell} H_\ell}{\sqrt{\rho_r} \pm \sqrt{\rho_\ell}}.
\end{aligned}$$

Once more, we take the plus sign in the final expression for $\hat{H}$, giving the Roe averages

$$\hat{u} = \frac{\sqrt{\rho_r} u_r + \sqrt{\rho_\ell} u_\ell}{\sqrt{\rho_r} + \sqrt{\rho_\ell}}, \quad \hat{H} = \frac{\sqrt{\rho_r} H_r + \sqrt{\rho_\ell} H_\ell}{\sqrt{\rho_r} + \sqrt{\rho_\ell}}.$$

To implement the Roe solver, we also need the eigenvalues and eigenvectors of the averaged flux Jacobian $f'(\hat{q})$. These are just the eigenvalues of the true Jacobian, evaluated at the averaged state:

$$\lambda_1 = \hat{u} - \hat{c}, \quad \lambda_2 = \hat{u}, \quad \lambda_3 = \hat{u} + \hat{c}, \quad (13.12)$$

$$r_1 = \begin{bmatrix} 1 \\ \hat{u} - \hat{c} \\ \hat{H} - \hat{u}\hat{c} \end{bmatrix}, \quad r_2 = \begin{bmatrix} 1 \\ \hat{u} \\ \frac{1}{2}\hat{u}^2 \end{bmatrix}, \quad r_3 = \begin{bmatrix} 1 \\ \hat{u} + \hat{c} \\ \hat{H} + \hat{u}\hat{c} \end{bmatrix}. \quad (13.13)$$

Here $\hat{c} = \sqrt{(\gamma - 1)(\hat{H} - \hat{u}^2/2)}$.

Solving the system of equations

$$q_r - q_\ell = \sum_{p=1}^3 \mathcal{W}_p = \sum_{p=1}^3 \alpha_p r_p \quad (13.14)$$

for the wave strengths gives

$$\alpha_2 = \delta_1 + (\gamma - 1) \frac{\hat{u}\delta_2 - \delta_3}{\hat{c}^2}, \quad (13.15)$$

$$\alpha_3 = \frac{\delta_2 + (\hat{c} - \hat{u})\delta_1 - \hat{c}\alpha_2}{2\hat{c}}, \quad (13.16)$$

$$\alpha_1 = \delta_1 - \alpha_2 - \alpha_3, \quad (13.17)$$

where $\delta = q_r - q_\ell$. We now have everything we need to implement the Roe solver.

```
In [3]: def roe_averages(q_l, q_r, gamma=1.4):
    rho_sqrt_l = np.sqrt(q_l[0])
    rho_sqrt_r = np.sqrt(q_r[0])
    p_l = (gamma-1.)*(q_l[2]-0.5*(q_l[1]**2)/q_l[0])
    p_r = (gamma-1.)*(q_r[2]-0.5*(q_r[1]**2)/q_r[0])
    denom = rho_sqrt_l + rho_sqrt_r
    u_hat = (q_l[1]/rho_sqrt_l + q_r[1]/rho_sqrt_r)/denom
```

```

H_hat = ((q_l[2]+p_l)/rho_sqrt_l + (q_r[2]+p_r)/rho_sqrt_r)/denom
c_hat = np.sqrt((gamma-1)*(H_hat-0.5*u_hat**2))

return u_hat, c_hat, H_hat

def Euler_roe(q_l, q_r, gamma=1.4):
    """
    Approximate Roe solver for the Euler equations.
    """

    rho_l = q_l[0]
    rho_u_l = q_l[1]
    u_l = rho_u_l/rho_l
    rho_r = q_r[0]
    rho_u_r = q_r[1]
    u_r = rho_u_r/rho_r

    u_hat, c_hat, H_hat = roe_averages(q_l, q_r, gamma)

    dq = q_r - q_l

    s1 = u_hat - c_hat
    s2 = u_hat
    s3 = u_hat + c_hat

    alpha2 = (gamma-1.)/c_hat**2 * ((H_hat-u_hat**2)*dq[0]+u_hat*dq[1]-dq[2])
    alpha3 = (dq[1] + (c_hat - u_hat)*dq[0] - c_hat*alpha2) / (2.*c_hat)
    alpha1 = dq[0] - alpha2 - alpha3

    r1 = np.array([1., u_hat-c_hat, H_hat - u_hat*c_hat])
    r2 = np.array([1., u_hat, 0.5*u_hat**2])
    q_l_star = q_l + alpha1*r1
    q_r_star = q_l_star + alpha2*r2

    states = np.column_stack([q_l,q_l_star,q_r_star,q_r])
    speeds = [s1, s2, s3]
    wave_types = ['contact', 'contact', 'contact']

    def reval(xi):
        rho = (xi<s1)*states[0,0] + (s1<=xi)*(xi<s2)*states[0,1] + \
            (s2<=xi)*(xi<s3)*states[0,2] + (s3<=xi)*states[0,3]
        mom = (xi<s1)*states[1,0] + (s1<=xi)*(xi<s2)*states[1,1] + \
            (s2<=xi)*(xi<s3)*states[1,2] + (s3<=xi)*states[1,3]
        E = (xi<s1)*states[2,0] + (s1<=xi)*(xi<s2)*states[2,1] + \
            (s2<=xi)*(xi<s3)*states[2,2] + (s3<=xi)*states[2,3]
        return rho, mom, E

    return states, speeds, reval, wave_types

```

An implementation of this solver for use in Clawpack can be found here.⁵⁰ Recall that an exact Riemann solver for the Euler equations appears in `exact_solvers/euler.py`.

13.1.1 ■ Examples

Let's compare the Roe approximation to the exact solution. As a first example, we use the Sod shock tube.

```
In [4]: def compare_solutions(left_state, right_state, solvers=['Exact', 'HLL']):
    q_l = np.array(euler.primitive_to_conservative(*left_state))
    q_r = np.array(euler.primitive_to_conservative(*right))

    outputs = []
    states = {}

    for solver in solvers:
        if solver.lower() == 'exact':
            outputs.append(euler.exact_riemann_solution(q_l, q_r))
        if solver.lower() == 'hll':
            outputs.append(Euler_hll(q_l, q_r))
            states['hll'] = outputs[-1][0]
        if solver.lower() == 'roe':
            outputs.append(Euler_roe(q_l, q_r))
            states['roe'] = outputs[-1][0]

    plot_function = \
        rt.make_plot_function([val[0] for val in outputs],
                              [val[1] for val in outputs],
                              [val[2] for val in outputs],
                              [val[3] for val in outputs],
                              solvers, layout='vertical',
                              variable_names=euler.primitive_variables,
                              derived_variables=euler.cons_to_prim,
                              vertical_spacing=0.15,
                              show_time_legend=True)

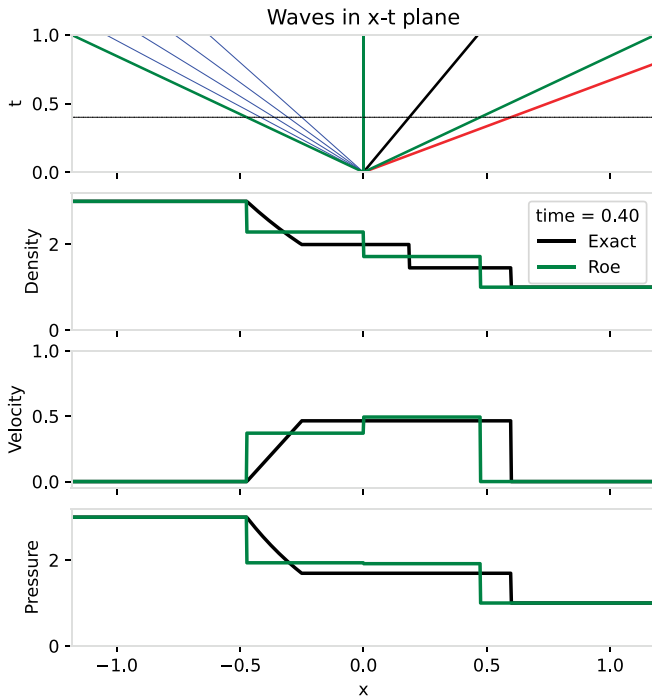
    interact(plot_function,
             t=widgets.FloatSlider(min=0, max=0.9, step=0.1, value=0.4));

    return states

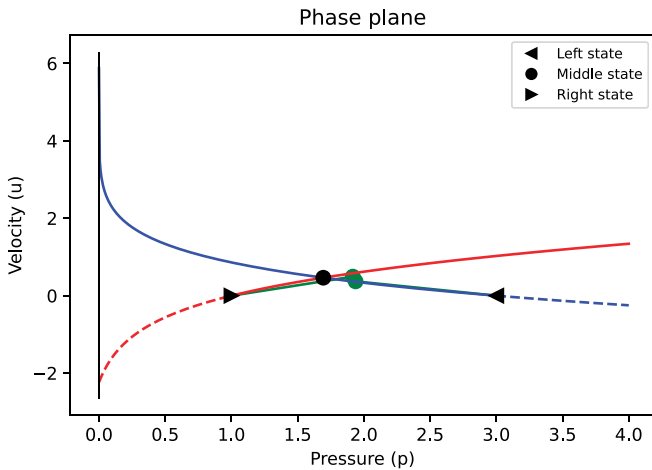
In [5]: left = State(Density = 3.,
                     Velocity = 0.,
                     Pressure = 3.)
    right = State(Density = 1.,
                  Velocity = 0.,
                  Pressure = 1.)
```

⁵⁰https://github.com/clawpack/riemann/blob/FA16/src/rpl_euler_with_efix.f90

```
states = compare_solutions(left, right, solvers=['Exact','Roe'])
```



```
In [6]: euler.phase_plane_plot(left, right, approx_states=states['roe'])
```



Recall that in the true solution the middle wave is a contact discontinuity and carries only a jump in the density. For that reason the three-dimensional phase space plot is generally shown projected onto the pressure-velocity plane as shown above: The two intermediate states in the true solution have the same pressure and velocity, and so are denoted by a single middle state in the phase plane plot.

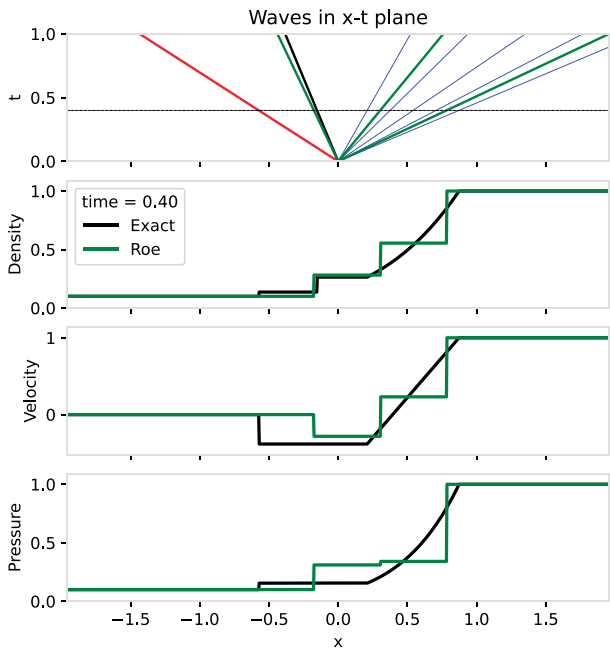
The Roe solver, on the other hand, generates a middle wave that carries a jump in all three variables, two green dots appear in the plot above for the two middle states (though the pressure jump is quite small in this example). For a Riemann

problem like this one with zero initial velocity on both sides, the Roe average velocity must also be zero, so the middle wave is stationary; this is of course not typically true in the exact solution, even when $u_\ell = u_r = 0$.

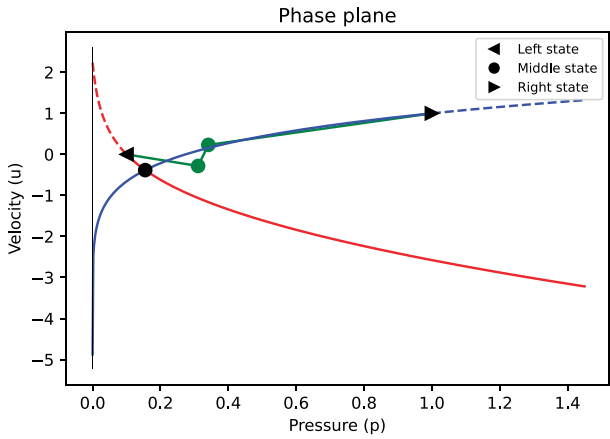
Here is a second example. Experiment with the initial states to explore how the Roe solution compares to the exact solution.

```
In [7]: left = State(Density = 0.1,
                    Velocity = 0.,
                    Pressure = 0.1)
right = State(Density = 1.,
              Velocity = 1.,
              Pressure = 1.)

states = compare_solutions(left, right, solvers=['Exact', 'Roe'])
```



```
In [8]: euler.phase_plane_plot(left, right, approx_states=states['roe'])
```



13.1.2 ■ Single-shock solution

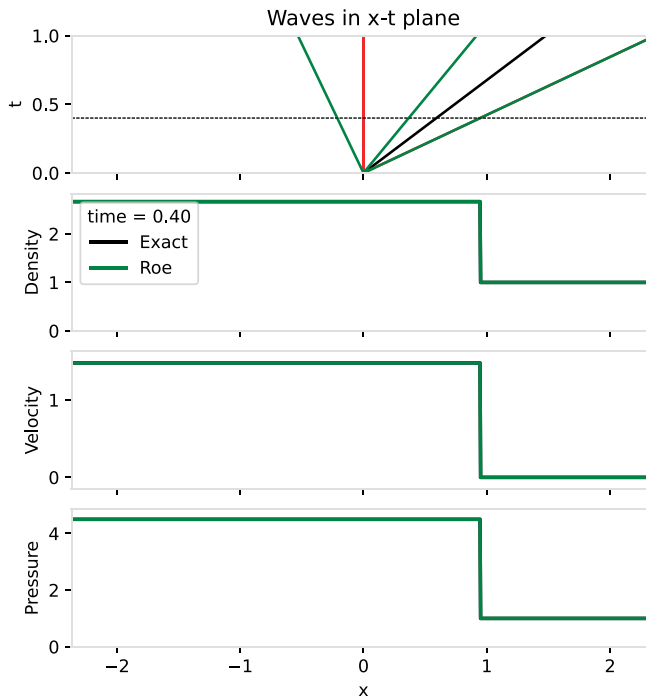
Next we demonstrate the exactness property of the Roe solver by applying it to a case where the left and right states are connected by a single shock wave.

```
In [9]: M = 2. # Mach number of the shock wave
gamma = 1.4
mu = 2*(M**2-1)/(M*(gamma+1.))
right = State(Density = 1.,
              Velocity = 0.,
              Pressure = 1.)
c_r = np.sqrt(gamma*right.Pressure/right.Density)

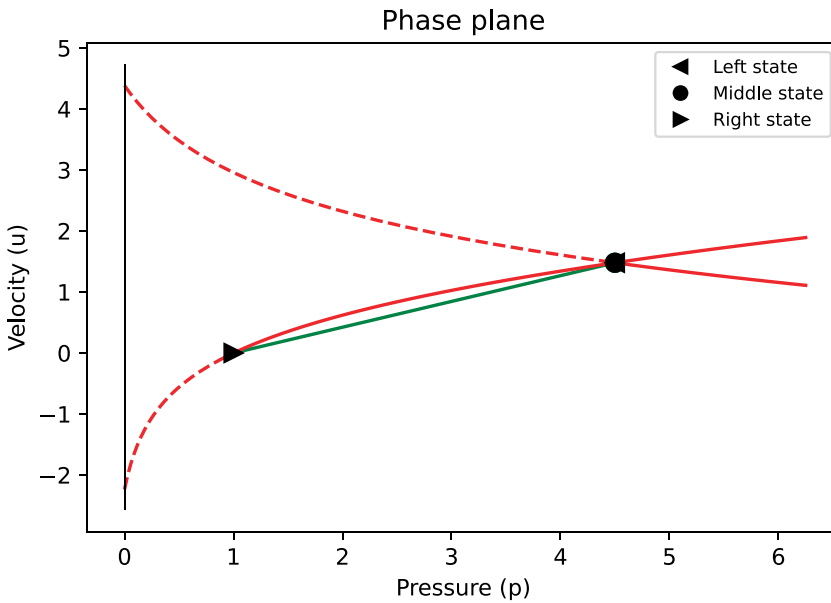
rho_l = right.Density * M/(M-mu)
p_l = right.Pressure * ((2*M**2-1)*gamma+1)/(gamma+1)
u_l = mu*c_r

left = State(Density = rho_l,
             Velocity = u_l,
             Pressure = p_l)

states = compare_solutions(left, right, solvers=['Exact', 'Roe'])
```



In [10]: `euler.phase_plane_plot(left, right, approx_states=states['roe'])`



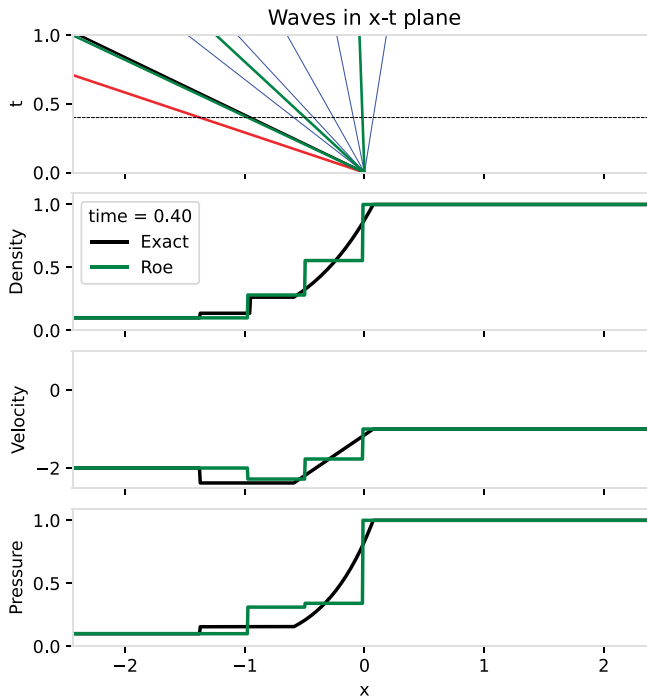
It is evident that the solution consists of a single right-going shock. The exact solution cannot be seen because it coincides exactly with the Roe solution. The path of the shock in the first plot also cannot be seen since it is plotted under the path of the rightmost Roe solution wave. The two solutions differ only in the wave speeds predicted for the other two waves, but since these waves have zero strength this makes no difference.

13.1.3 ■ Transonic rarefactions and an entropy fix

Here is an example of a Riemann problem whose solution includes a transonic 2-rarefaction:

```
In [11]: left = State(Density = 0.1,
                      Velocity = -2.,
                      Pressure = 0.1)
          right = State(Density = 1.,
                      Velocity = -1.,
                      Pressure = 1.)
```

```
states = compare_solutions(left, right, solvers=['Exact', 'Roe'])
```



Notice that in the exact solution, the right edge of the rarefaction travels to the right. In the Roe solution, all waves travel to the left. As in the case of the shallow water equations, here too this behavior can lead to unphysical solutions when this approximate solver is used in a numerical discretization. To correct this, we can split the single wave into two when a transonic rarefaction is present, similar to what is done in the shallow water equations. We do not go into details here.

13.2 ■ HLL Solver

Recall that an HLL solver uses only two waves with a constant state between them. The Euler equations are our first example for which the number of waves in the true solution is larger than the number of waves in the approximate solution. As one might expect, this leads to noticeable inaccuracy in solutions produced by the solver.

Again following Einfeldt, the left-going wave speed is chosen to be the minimum of the Roe speed for the 1-wave and the characteristic speed λ^1 in the left state q_ℓ . The right-going wave speed is chosen to be the maximum of the Roe speed for the 3-wave and the characteristic speed λ^3 in the right state q_r . Effectively, this means that

$$s_1 = \min(u_\ell - c_\ell, \hat{u} - \hat{c}), \quad (13.18)$$

$$s_2 = \max(u_r + c_r, \hat{u} + \hat{c}). \quad (13.19)$$

Recall that once we have chosen these two wave speeds, conservation dictates the value of the intermediate state:

$$q_m = \frac{f(q_r) - f(q_\ell) - s_2 q_r + s_1 q_\ell}{s_1 - s_2}. \quad (13.20)$$

```
In [12]: def Euler_hlle(q_l, q_r, gamma=1.4):
    """HLLE approximate solver for the Euler equations."""

    rho_l = q_l[0]
    rhou_l = q_l[1]
    u_l = rhou_l/rho_l
    rho_r = q_r[0]
    rhou_r = q_r[1]
    u_r = rhou_r/rho_r
    E_r = q_r[2]
    E_l = q_l[2]

    u_hat, c_hat, H_hat = roe_averages(q_l, q_r, gamma)
    p_r = (gamma-1.) * (E_r - rho_r*u_r**2/2.)
    p_l = (gamma-1.) * (E_l - rho_l*u_l**2/2.)
    H_r = (E_r+p_r) / rho_r
    H_l = (E_l+p_l) / rho_l
    c_r = np.sqrt((gamma-1.)*(H_r-u_r**2/2.))
    c_l = np.sqrt((gamma-1.)*(H_l-u_l**2/2.))

    s1 = min(u_l-c_l,u_hat-c_hat)
    s2 = max(u_r+c_r,u_hat+c_hat)

    rho_m = (rhou_r - rhou_l - s2*rho_r + s1*rho_l)/(s1-s2)
    rhou_m = (rho_r*u_r**2 - rho_l*u_l**2 \
              + p_r - p_l - s2*rhou_r + s1*rhou_l)/(s1-s2)
    E_m = ( u_r*(E_r+p_r) - u_l*(E_l+p_l) - s2*E_r + s1*E_l)/(s1-s2)
    q_m = np.array([rho_m, rhou_m, E_m])

    states = np.column_stack([q_l,q_m,q_r])
    speeds = [s1, s2]
    wave_types = ['contact','contact']

    def reval(xi):
        rho = (xi<s1)*rho_l + (s1<=xi)*(xi<=s2)*rho_m + (s2<xi)*rho_r
        mom = (xi<s1)*rhou_l + (s1<=xi)*(xi<=s2)*rhou_m + (s2<xi)*rhou_r
        E = (xi<s1)*E_l + (s1<=xi)*(xi<=s2)*E_m + (s2<xi)*E_r
        return rho, mom, E

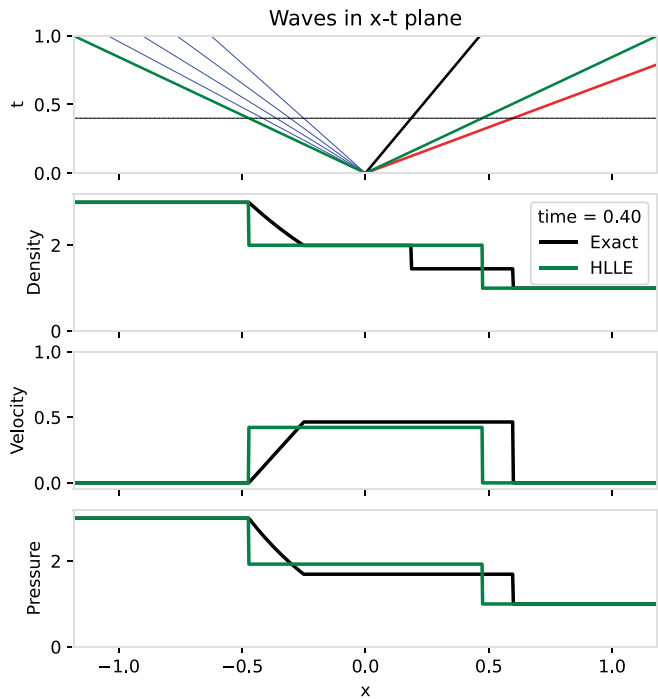
    return states, speeds, reval, wave_types
```

13.2.1 ■ Examples

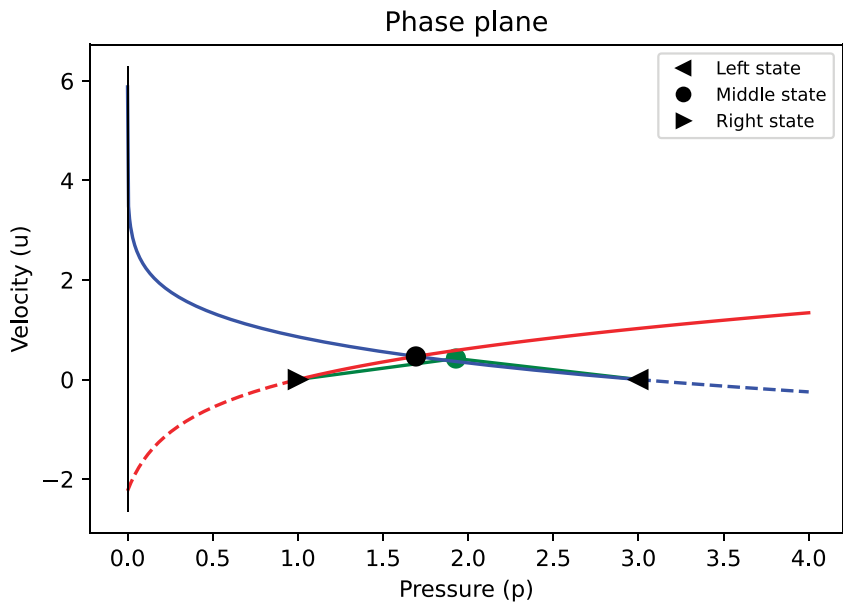
```
In [13]: left = State(Density = 3.,
                      Velocity = 0.,
                      Pressure = 3.)
```

```
right = State(Density = 1.,
              Velocity = 0.,
              Pressure = 1.)

states = compare_solutions(left, right, solvers=['Exact', 'HLL'])
```



```
In [14]: euler.phase_plane_plot(left, right, approx_states=states['hll'])
```

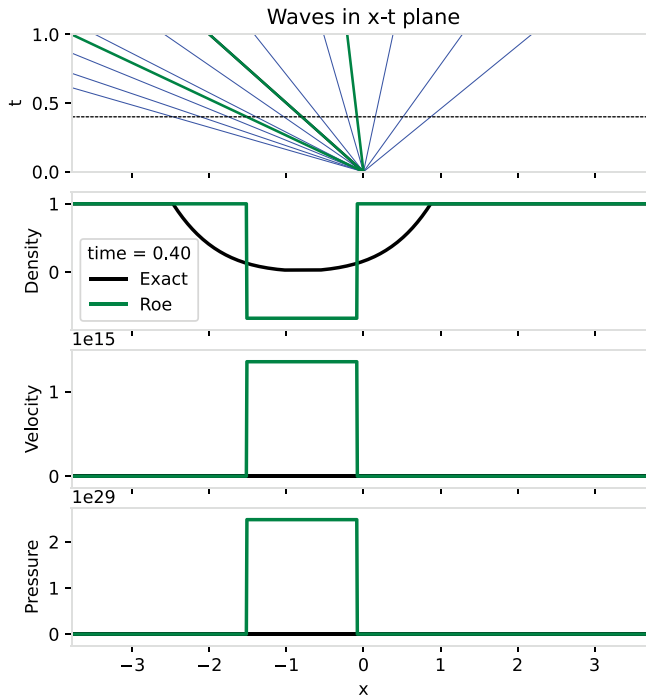


13.2.2 ■ Preservation of positivity

Just as we saw in the case of the shallow water equations, the Roe solver (or any linearized solver) for the Euler equations fails to preserve positivity of the pressure and/or density in some situations. Here is one example:

```
In [15]: left = State(Density = 1.,
                     Velocity = -5.,
                     Pressure = 1.)
        right = State(Density = 1.,
                     Velocity = 1.,
                     Pressure = 1.)

        states = compare_solutions(left, right, solvers=['Exact', 'Roe'])
```



As we can see, in this example each Roe solver wave moves much more slowly than the leading edge of the corresponding true rarefaction. In order to maintain conservation, this implies that the middle Roe state must have lower density than the true middle state. This leads to a negative density. Note that the velocity and pressure take huge values in the intermediate state.

The HLLE solver, on the other hand, guarantees positivity of the density and pressure. Since the HLLE wave speed in the case of a rarefaction is always the speed of the leading edge of the true rarefaction, and since the HLLE solution is conservative, the density in a rarefaction will always be at least as great as that of the true solution. This can be seen clearly in the example below.

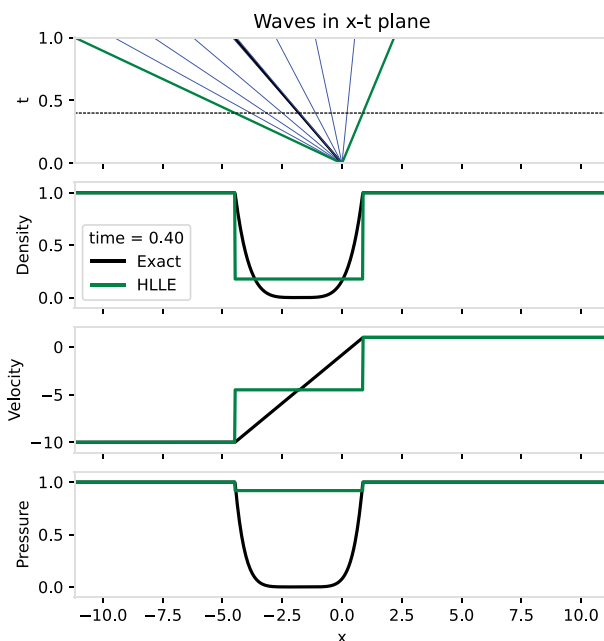
```
In [16]: left = State(Density = 1.,
                     Velocity = -10.,
                     Pressure = 1.)
```

```

right = State(Density = 1.,
              Velocity = 1.,
              Pressure = 1.)

states = compare_solutions(left, right, solvers=['Exact', 'HLL']);

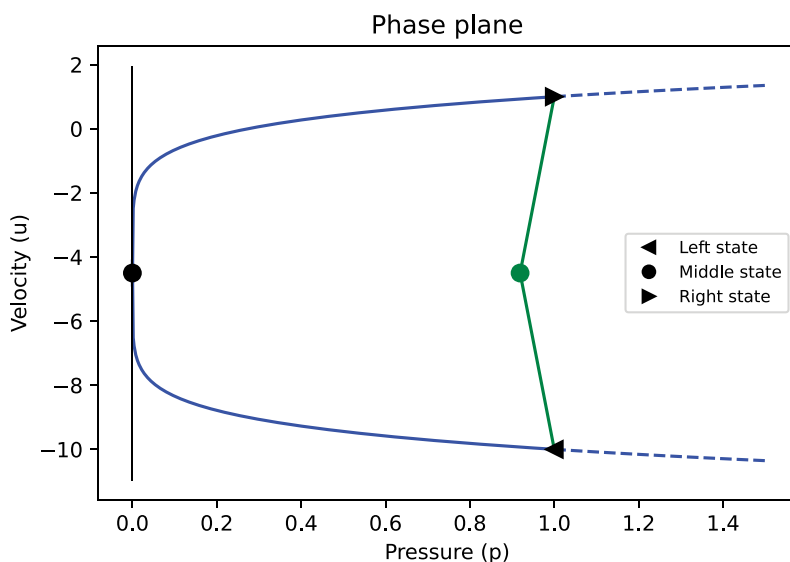
```



```

In [17]: euler.phase_plane_plot(left, right, approx_states=states['hll'])

```



Again recall that we are only considering a single Riemann solution in this chapter. In Chapter 14 we observe the effect of using these approximate solvers in a full discretization.

Chapter 14

Finite volume discretizations with approximate Riemann solvers

In the previous few chapters we introduced several approximate Riemann solvers for different nonlinear conservation laws and showed some comparisons of the approximate solution to a single Riemann problem with the true solution, where there often appear to be significant differences. However, their intended use is as a building block for finite volume methods such as Godunov's method or high-resolution extensions.

How do these solvers impact the solution accuracy when used within a finite volume discretization? To investigate, we will use them within PyClaw⁵¹ to solve several test problems.

In particular, for both the shallow water equations and the Euler equations, we will apply the numerical method to solve dam break and shock tube problems, which are themselves Riemann problems, so we can compute the exact solution, but once discretized they require the solution of a different set of Riemann problems at each cell interface at every time step.

For the Euler equations we will also consider the Woodward–Colella blast wave problem. The initial data consists of two Riemann problems, with resulting shock waves of differing strengths. These shock waves later interact with each other.

In this chapter we include extensive sections of code in the notebook. This is meant to more easily allow the reader to use these as templates for setting up other problems in PyClaw. For the code in this chapter we use approximate Riemann solvers from PyClaw that can be found in these files:

- `euler_1D_py.py`,⁵²
- `shallow_roe_with_efix_1D`.⁵³

⁵¹<http://www.clawpack.org/pyclaw/>

⁵²https://github.com/clawpack/riemann/blob/FA16/riemann/euler_1D_py.py

⁵³https://github.com/clawpack/riemann/blob/FA16/riemann/shallow_1D_py.py

14.1 ■ Shallow water equations

We compare results obtained with using the high-resolution wave propagation method implemented in PyClaw (with limiters to avoid nonphysical oscillations). We compare the results obtained when combined with two different approximate Riemann solvers: the Roe solver and HLLE.

```
In [3]: from exact_solvers import shallow_water as sw
        from clawpack import riemann
        from clawpack.riemann.shallow_roe_with_efix_1D_constants import \
            depth, momentum, num_eqn

def setup(riemann_solver='roe', N=20, IC='dam-break'):

    from clawpack import pyclaw

    if riemann_solver.lower() == 'roe':
        rs = riemann.shallow_roe_with_efix_1D
    elif riemann_solver.lower() == 'hlle':
        rs = riemann.shallow_hlle_1D

    solver = pyclaw.ClawSolver1D(rs)

    solver.bc_lower[0] = pyclaw.BC.extrap
    solver.bc_upper[0] = pyclaw.BC.extrap

    xlower = -5.0
    xupper = 5.0
    x = pyclaw.Dimension(xlower, xupper, N, name='x')
    domain = pyclaw.Domain(x)
    state = pyclaw.State(domain, num_eqn)

    # Gravitational constant
    state.problem_data['grav'] = 1.0
    state.problem_data['dry_tolerance'] = 1e-3
    state.problem_data['sea_level'] = 0.0

    xc = state.grid.x.centers

    x0=0.

    hl = 10.
    ul = 0.
    hr = 0.5
    ur = 0.
    state.q[depth,:] = hl * (xc <= x0) + hr * (xc > x0)
    state.q[momentum,:] = hl*ul * (xc <= x0) + hr*ur * (xc > x0)

    claw = pyclaw.Controller()
    claw.keep_copy = True
```

```

        claw.output_format = None
        claw.tfinal = 1.0
        claw.solution = pyclaw.Solution(state,domain)
        claw.solver = solver

    return claw

```

```

In [4]: N = 40 # number of grid cells to use
        q_l = [10,0]
        q_r = [0.5,0]

        # Roe solution
        roe_sw = setup(riemann_solver='roe',N=N)
        roe_sw.verbosity = 0
        status = roe_sw.run()
        xc_sw = roe_sw.grid.x.centers

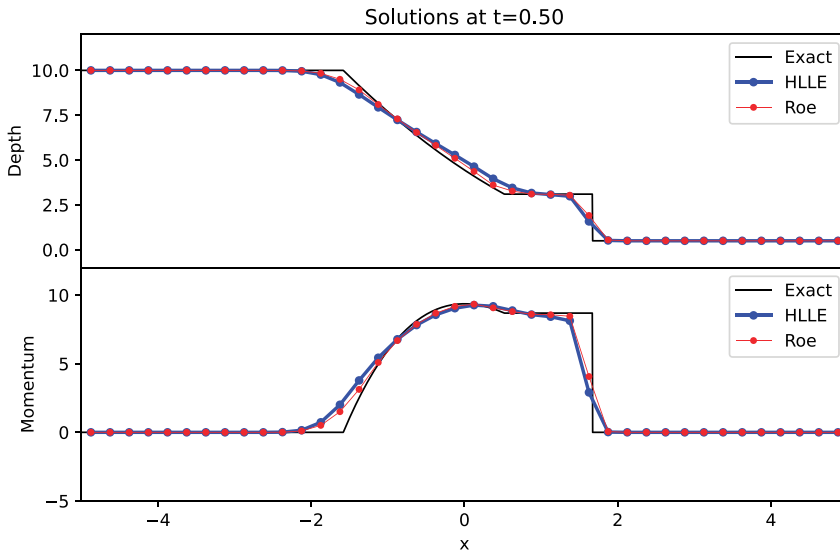
        # HLLC solution
        hllc_sw = setup(riemann_solver='hllc',N=N)
        hllc_sw.verbosity = 0
        status = hllc_sw.run()

        # Exact solution
        xc_exact_sw = np.linspace(-5,5,2000)
        states_sw, speeds_sw, reval_sw, wave_types_sw = \
            sw.exact_riemann_solution(q_l, q_r)

def plot_frame(i):
    t = roe_sw.frames[i].t+1.e-13
    fig, ax = plt.subplots(2,1, sharex=True, figsize=(8,5))
    variablenames = ["Depth", "Momentum"]
    variables = [depth, momentum]
    ylims = [[-1,12], [-5,12]]
    plt.subplots_adjust(hspace=0)
    ax[0].title.set_text('Solutions at t={:.2f}'.format(t))
    ax[0].set_xlim((-5,5))
    ax[1].set_xlabel = 'x')
    for j, variable in enumerate(variables):
        ax[j].set_ylim(ylims[j])
        ax[j].plot(xc_exact_sw,reval_sw/(t+1.e-16))[j],'-k',lw=1)
        ax[j].plot(xc_sw, hllc_sw.frames[i].q[variable,:],'-ob',lw=2,ms=4)
        ax[j].plot(xc_sw, roe_sw.frames[i].q[variable,:],'-or',lw=0.5,ms=3)
        ax[j].legend(['Exact', 'HLLC', 'Roe'],loc='best')
        ax[j].set_ylabel=variablenames[j])
    plt.show()

interact(plot_frame, i=widgets.IntSlider(min=0,max=10,value=5));

```



Clearly, both solvers give very similar results for this problem. They converge to the same solution, as you can verify by increasing the value of N above in the live notebook. You can also try varying the left and right states.

14.2 ■ Euler equations

We perform a similar test for the Euler equations with shock tube initial data.

14.2.1 ■ Sod shock tube problem

We first consider the classic shock tube problem proposed by Sod and already discussed in Chapter 9. This is a particular Riemann problem in which the initial velocity is zero on both sides of a discontinuity in pressure and/or density of a gas, so the exact Riemann solver for the Euler equations would provide the exact solution for all time, consisting of a right-going shock, a left-going rarefaction, and an intermediate contact discontinuity.

In the numerical experiments done in this notebook, we use this initial data for a more general finite volume method that could be used to approximate the solution for any initial data. In the first time step there is a single cell interface with nontrivial Riemann data, but as the solution evolves on the grid the Riemann problems that arise in subsequent time steps are very different from the single problem we started with. Depending on the accuracy of the numerical method, the resolution of the grid, and the choice of approximate Riemann solver to use at each grid cell at every time step, the numerical solution may deviate significantly from the exact solution to the original shock tube problem. This makes a good initial test problem for numerical methods because the exact solution can be computed for comparison purposes, and because it clearly shows whether the method introduces oscillations around discontinuities and/or smears them out.

```

In [5]: from clawpack.riemann.euler_with_efix_1D_constants \
import density, momentum, energy, num_eqn

def shocktube(q_l, q_r, N=50, riemann_solver='HLL',
              solver_type='classic'):

    from clawpack import pyclaw
    from clawpack import riemann

    if riemann_solver == 'Roe':
        rs = riemann.euler_1D_py.euler_roe_1D
    elif riemann_solver == 'HLL':
        rs = riemann.euler_1D_py.euler_hll_1D

    if solver_type == 'classic':
        solver = pyclaw.ClawSolver1D(rs)
        solver.limiters = pyclaw.limiters.tvd.MC
    else:
        solver = pyclaw.SharpClawSolver1D(rs)

    solver.kernel_language = 'Python'

    solver.bc_lower[0]=pyclaw.BC.extrap
    solver.bc_upper[0]=pyclaw.BC.extrap

    x = pyclaw.Dimension(-1.0,1.0,N,name='x')
    domain = pyclaw.Domain([x])
    state = pyclaw.State(domain,num_eqn)

    gamma = 1.4
    state.problem_data['gamma']= gamma
    state.problem_data['gamma1']= gamma-1.

    state.problem_data['efix'] = False

    xc = state.grid.p_centers[0]

    velocity = (xc<=0)*q_l[1] + (xc>0)*q_r[1]
    pressure = (xc<=0)*q_l[2] + (xc>0)*q_r[2]

    state.q[density,:] = (xc<=0)*q_l[0] + (xc>0)*q_r[0]
    state.q[momentum,:] = velocity * state.q[density,:]
    state.q[energy,:] = pressure/(gamma - 1.) + \
        0.5 * state.q[density,:] * velocity**2

    claw = pyclaw.Controller()
    claw.tfinal = 0.5
    claw.solution = pyclaw.Solution(state,domain)
    claw.solver = solver
    claw.num_output_times = 10

```

```
claw.keep_copy = True
claw.verbosity=0
```

```
return claw
```

In [6]: `N = 80` # number of grid cells to use

```
prim_l = [1.,0.,1.]
prim_r = [1./8,0.,1./10]
q_l = euler.conservative_to_primitive(*prim_l)
q_r = euler.conservative_to_primitive(*prim_r)
```

```
# Roe-based solution
```

```
roe_st = shocktube(q_l,q_r,N=N,riemann_solver='Roe')
roe_st.run()
xc_st = roe_st.solution.state.grid.p_centers[0]
```

```
# HLL-based solution
```

```
hll_st = shocktube(q_l,q_r,N=N,riemann_solver='HLL')
hll_st.run()
```

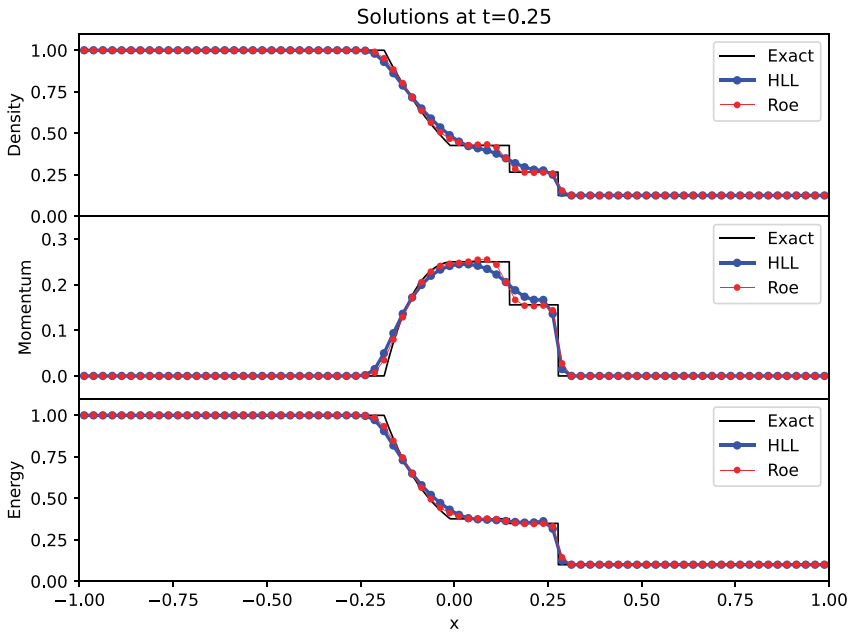
```
# Exact solution
```

```
xc_exact_st = np.linspace(-1,1,2000)
states_st, speeds_st, reval_st, wave_types_st = \
    euler.exact_riemann_solution(prim_l, prim_r)
```

```
def plot_frame(i):
```

```
    t = roe_st.frames[i].t
    fig, ax = plt.subplots(3,1, sharex=True, figsize=(8,6))
    variablenames = ["Density", "Momentum", "Energy"]
    variables = [density, momentum, energy]
    ylims = [[0,1.1], [-0.05,0.35], [0,1.1]]
    plt.subplots_adjust(hspace=0)
    ax[0].title.set_text('Solutions at t={:.2f}'.format(t))
    ax[0].set_xlim((-1,1))
    ax[2].set(xlabel = 'x')
    for j, variable in enumerate(variables):
        ax[j].set_ylim(ylims[j])
        ax[j].plot(xc_exact_st,reval_st(xc_exact_st/(t+1.e-16))[j],'-k',lw=1)
        ax[j].plot(xc_st,hll_st.frames[i].q[variable,:],'-ob',lw=2,ms=4)
        ax[j].plot(xc_st,roe_st.frames[i].q[variable,:],'-or',lw=0.5,ms=3)
        ax[j].legend(['Exact', 'HLL', 'Roe'],loc='best')
        ax[j].set(ylabel=variablenames[j])
    plt.show()
```

```
interact(plot_frame, i=widgets.IntSlider(min=0, max=10, value=5,
    description='Frame'));
```



As you might expect, the HLL solver smears the middle wave (contact discontinuity) significantly more than the Roe solver does. Perhaps surprisingly, it captures the shock just as accurately as the Roe solver does. In the live notebook you can refine the grid and observe the convergence.

14.2.2 ■ High-order WENO + Runge–Kutta

Next we look at the difference between the HLL and Roe solutions when these solvers are employed within a higher-order method of lines discretization using fifth-order weighted essentially nonoscillating (WENO) and a fourth-order Runge–Kutta scheme.

In [7]: `N = 80 # number of grid cells to use`

```
prim_l = [1., 0., 1.]
prim_r = [1./8, 0., 1./10]
q_l = euler.conservative_to_primitive(*prim_l)
q_r = euler.conservative_to_primitive(*prim_r)

roe_weno = shocktube(q_l, q_r, N=N, riemann_solver='Roe',
                     solver_type='sharpclaw')
roe_weno.run()
hll_weno = shocktube(q_l, q_r, N=N, riemann_solver='HLL',
                     solver_type='sharpclaw')
hll_weno.run()

xc_weno = roe_weno.solution.state.grid.p_centers[0]
```

```
# Exact solution
```

```
xc_exact_weno = np.linspace(-1,1,2000)
```

```
states_weno, speeds_weno, reval_weno, wave_types_weno =  
    euler.exact_riemann_solution(prim_l, prim_r)
```

```
def plot_frame(i):
```

```
    t = roe_weno.frames[i].t
```

```
    fig, ax = plt.subplots(3,1, sharex=True, figsize=(8,6))
```

```
    variablenames = ["Density", "Momentum", "Energy"]
```

```
    variables = [density, momentum, energy]
```

```
    ylims = [[0,1.1], [-0.05,0.35], [0,1.1]]
```

```
    plt.subplots_adjust(hspace=0)
```

```
    ax[0].title.set_text('Solutions at t={:2f}'.format(t))
```

```
    ax[0].set_xlim((-1,1))
```

```
    ax[2].set_xlabel = 'x'
```

```
    for j, variable in enumerate(variables):
```

```
        ax[j].set_ylim(ylims[j])
```

```
        ax[j].plot(xc_exact_weno,  
                   reval_weno(xc_exact_weno/(t+1.e-16))[j], '-k', lw=1)
```

```
        ax[j].plot(xc_weno, hll_weno.frames[i].q[variable,:], '-ob', lw=2, ms=4)
```

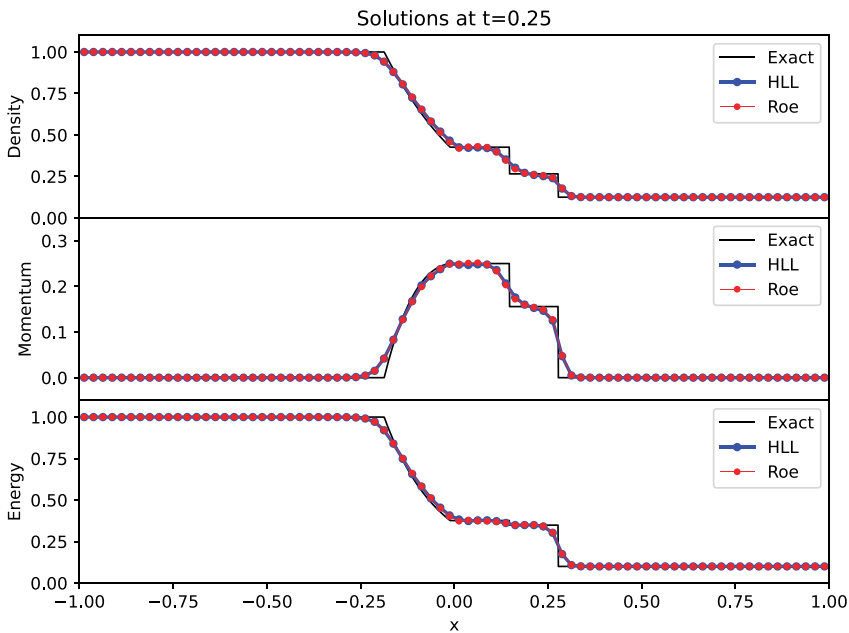
```
        ax[j].plot(xc_weno, roe_weno.frames[i].q[variable,:], '-or', lw=0.5, ms=3)
```

```
        ax[j].legend(['Exact', 'HLL', 'Roe'], loc='best')
```

```
        ax[j].set_ylabel=variablenames[j]
```

```
    plt.show()
```

```
interact(plot_frame, i=widgets.IntSlider(min=0, max=10, value=5,  
    description='Frame'));
```



With higher-order discretizations, the difference in solutions due to using different Riemann solvers is less significant. This is partly because these high-order

schemes use more accurate values as inputs to the Riemann problem, so that in smooth regions the jump between most cells is very small.

14.2.3 ■ Woodward–Colella blast wave

Finally, we consider the Woodward–Colella blast wave problem, which is discussed, for example, in [9]. Here the initial velocity is zero and the density is one everywhere. The pressure is

$$p_0(x) = \begin{cases} 1000, & 0 \leq x \leq 0.1, \\ 0.01, & 0.1 \leq x \leq 0.9, \\ 100, & 0.9 \leq x \leq 1. \end{cases} \quad (14.1)$$

The boundaries at $x = 0$ and $x = 1$ are solid walls. The solution involves a Riemann problem at $x = 0.1$ and another at $x = 0.9$. Later, the waves resulting from these Riemann problems interact with each other.

```
In [8]: from clawpack.riemann.euler_with_efix_1D_constants \
import density, momentum, energy, num_eqn

def blastwave(N=400, riemann_solver='HLL', solver_type='classic'):

    from clawpack import pyclaw
    from clawpack import riemann

    if riemann_solver == 'Roe':
        kernel_language = 'Fortran'
        rs = riemann.euler_with_efix_1D
    elif riemann_solver == 'HLL':
        kernel_language = 'Python'
        rs = riemann.euler_1D_py.euler_hll_1D

    if solver_type == 'classic':
        solver = pyclaw.ClawSolver1D(rs)
        solver.limiters = pyclaw.limiters.tvd.MC
    else:
        solver = pyclaw.SharpClawSolver1D(rs)

    solver.kernel_language = kernel_language

    solver.bc_lower[0]=pyclaw.BC.wall
    solver.bc_upper[0]=pyclaw.BC.wall

    x = pyclaw.Dimension(0.0,1.0,N,name='x')
    domain = pyclaw.Domain([x])
    state = pyclaw.State(domain,num_eqn)

    gamma = 1.4
    state.problem_data['gamma']= gamma
    state.problem_data['gamma1']= gamma-1.

    state.problem_data['efix'] = False
```

```

xc = state.grid.p_centers[0]

pressure = (xc<0.1)*1.e3 + (0.1<=xc)*(xc<0.9)*1.e-2 + (0.9<=xc)*1.e2

state.q[density,:] = 1.
state.q[momentum,:] = 0.
state.q[energy,:] = pressure / (gamma - 1.)

claw = pyclaw.Controller()
claw.tfinal = 0.038
claw.solution = pyclaw.Solution(state,domain)
claw.solver = solver
claw.num_output_times = 30
claw.keep_copy = True
claw.verbosity=0

return claw

```

In [9]: `N = 400 # number of grid cells to use`

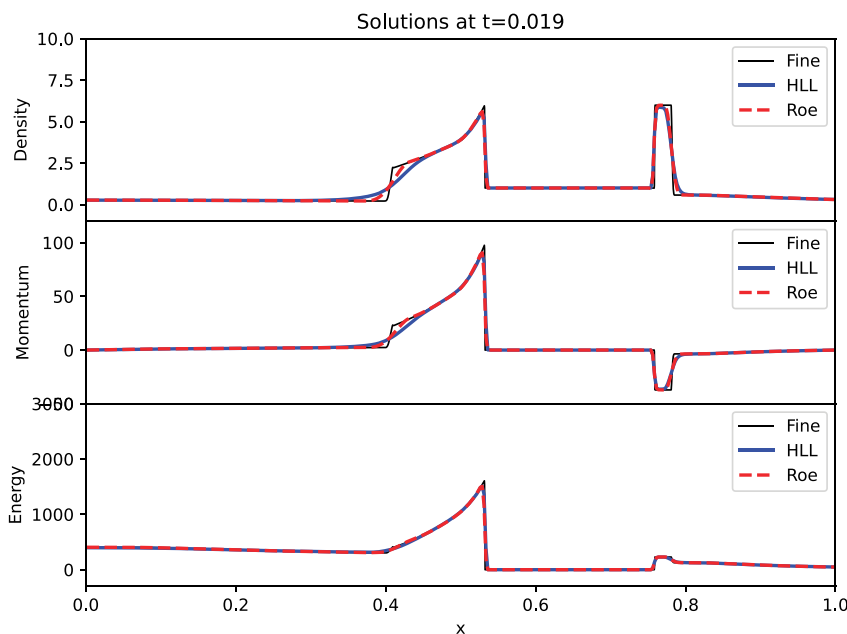
```

roe_bw = blastwave(N=N, riemann_solver='Roe')
roe_bw.run()
hll_bw = blastwave(N=N, riemann_solver='HLL')
hll_bw.run()
fine_bw = blastwave(N=4000,riemann_solver='Roe')
fine_bw.run();
xc_bw = roe_bw.solution.state.grid.p_centers[0]
xc_fine_bw = fine_bw.solution.state.grid.p_centers[0]

def plot_frame(i):
    t = roe_bw.frames[i].t
    fig, ax = plt.subplots(3,1, sharex=True, figsize=(8,6))
    variablenames = ["Density", "Momentum", "Energy"]
    variables = [density, momentum, energy]
    ylims = [[-1,10], [-50,120], [-300,3000]]
    plt.subplots_adjust(hspace=0)
    ax[0].title.set_text('Solutions at t={:.3f}'.format(t))
    ax[0].set_xlim((0,1))
    ax[2].set(xlabel = 'x')
    for j, variable in enumerate(variables):
        ax[j].set_ylim(ylims[j])
        ax[j].plot(xc_fine_bw,fine_bw.frames[i].q[variable,:],'-k',lw=1)
        ax[j].plot(xc_bw,hll_bw.frames[i].q[variable,:],'-b',lw=2)
        ax[j].plot(xc_bw,roe_bw.frames[i].q[variable,:], '--r',lw=2)
        ax[j].legend(['Fine', 'HLL', 'Roe'],loc='best')
        ax[j].set(ylabel=variablenames[j])
    plt.show()

interact(plot_frame, i=widgets.IntSlider(min=0, max=30, value=15,
    description='Frame'));

```



Here no exact solution is available, so we compare with a solution computed on a finer grid. Again the solutions are fairly similar, though the HLL solution is a bit more smeared.

One should not conclude from these tests that, for instance, the Roe solver is always *better* than the HLL solver. Many factors besides accuracy should be considered, including cost and robustness. As we have seen, the HLL solver is more robust in the presence of near-vacuum states.